

Optimización y Análisis de Redes

Apuntes de la
Asignatura

Grado en Matemáticas

AUTORES

- Víctor Aceña Gil
- Antonio Alonso Ayuso

2026-2027



Índice apuntes

Prefacio	7
Filosofía pedagógica del volumen	7
¿Qué aprenderás con este libro?	8
1 Introducción a la Investigación Operativa	10
1.1 Definición e Historia de la Investigación Operativa	11
1.1.1 Raíces Históricas y el Nacimiento de la Disciplina	11
1.1.2 Definiciones Formales	11
1.1.3 Herramientas de la Investigación Operativa	12
1.2 El Concepto de Modelo en Ciencias de la Decisión	12
1.2.1 Clasificación General de los Modelos	13
1.2.2 El Ciclo de Vida del Modelado Matemático	13
1.3 Formulación General de Modelos de Optimización	14
1.3.1 Clasificación de los Modelos de Optimización	15
1.4 La Estructura del Libro: Optimización No Lineal y en Redes	16
1.4.1 Bloque I: Optimización No Lineal (Temas 2 a 6)	16
1.4.2 Bloque II: Optimización en Redes (Temas 7 a 11)	17
2 Optimización No Lineal y Métodos Numéricos	18
2.1 Introducción y Formulación de Problemas No Lineales	19
2.1.1 Casos Particulares de la Optimización No Lineal	20
2.1.2 Caso de Estudio 1: El Monopolio de Cournot (1838)	20
2.1.3 Caso de Estudio 2: Selección de Cartera de Valores (Markowitz, 1952)	23
2.2 Tipos de Solución en Optimización No Lineal	23
2.3 Fundamentos Matemáticos y Condiciones de Optimalidad	24
2.3.1 Desarrollo de Taylor Multivariable	24
2.3.2 Caracterización de Curvatura y Clasificación de Matrices	25
2.3.3 Condiciones de Optimalidad y Puntos Estacionarios	26
2.3.4 Resolución Simbólica con Python y SymPy	27
2.4 Métodos Numéricos de Búsqueda Lineal (1D)	28
2.4.1 Métodos de Búsqueda Lineal sin Derivadas	28
2.4.2 Métodos de Búsqueda Lineal con Derivadas	37
2.5 Métodos Multivariantes de Optimización Sin Restricciones	41
2.5.1 Método de las Coordenadas Cíclicas	41
2.5.2 Método del Máximo Descenso (Steepest Descent)	44

2.5.3	Métodos de Newton Multivariable y BFGS	47
3	Análisis Convexo	48
3.1	Conjuntos Convexos	49
3.1.1	Definición y Conceptos Básicos	49
3.1.2	Ejemplos de Conjuntos Convexos Básicos	50
3.1.3	Operaciones que Conservan la Convexidad	50
3.1.4	Envoltura Convexa y Símplices	51
3.2	Teoremas de Separación e Hiperplano de Soporte	51
3.2.1	Teoremas de Separación de Hiperplanos	51
3.2.2	Teorema del Hiperplano de Soporte	52
3.3	Funciones Convexas	52
3.3.1	Definición y Significado Geométrico	52
3.3.2	Operaciones que Preservan la Convexidad de Funciones	53
3.3.3	Epígrafe y Subgradientes	53
3.4	Caracterización de Funciones Convexas Diferenciables	56
3.4.1	Caracterización de Primer Orden	56
3.4.2	Caracterización de Segundo Orden (Hessiana)	57
3.5	Óptimos de una Función Convexa	58
3.5.1	El Teorema Fundamental de la Optimización Convexa	58
3.5.2	Condición Variacional de Optimalidad	59
3.6	Maximización de Funciones Convexas	59
3.7	Generalizaciones de la Convexidad	60
3.7.1	Funciones Cuasi-convexas	60
3.7.2	Funciones Pseudo-convexas	60
3.7.3	Relaciones entre Categorías de Convexidad	61
4	Condiciones de Optimalidad y KKT	62
4.1	Optimización Sin Restricciones	62
4.1.1	Direcciones de Descenso	62
4.1.2	Condiciones Necesarias y Suficientes de Segundo Orden	63
4.2	Optimización Con Restricciones: Conceptos de Conos	64
4.2.1	Definiciones Geométricas de Conos	64
4.2.2	Caracterización Diferenciable mediante Semiespacios	66
4.3	Restricciones de Desigualdad y Cono de Restricciones Activas	66
4.4	Condiciones de Fritz-John (FJ)	67
4.4.1	Limitación de Fritz-John: El Caso de Multiplicador Nulo ($u_0 = 0$)	68
4.5	Condiciones de Karush-Kuhn-Tucker (KKT)	68
4.5.1	Historia de KKT	69
4.5.2	Cualificación de Restricciones (LICQ y otras)	69
4.5.3	Condiciones Necesarias de KKT	69
4.5.4	Condiciones Suficientes de KKT bajo Convexidad	70
4.6	Esquema General de Resolución KKT	71

4.7	Restricciones de Igualdad y Desigualdad	72
5	Aplicaciones de KKT: Programación Lineal y Cuadrática	74
5.1	Programación Lineal (LP)	75
5.1.1	Dualidad en Programación Lineal	75
5.1.2	Condiciones KKT en Programación Lineal	76
5.1.3	Interpretación Económica: Precios Sombra	77
5.1.4	Clasificación de Óptimos y Relaciones de Dualidad	78
5.2	Programación Cuadrática (QP)	79
5.2.1	El Sistema KKT para QP en Forma Matricial	79
5.2.2	Linealización y Algoritmo del Símplex de Wolfe	80
5.3	Caso Práctico Detallado: Algoritmo de Wolfe	80
5.3.1	Progresión de las Tablas del Símplex de Wolfe	81
5.4	Formulación en Python con CVXPY	84
6	Optimización en Ciencia de Datos: Regresión y SVM	87
6.1	Introducción a la Optimización en Aprendizaje Automático	87
6.2	El Problema de Ajuste de Datos en Diferentes Normas	88
6.2.1	Regresión Lineal Clásica (Norma ℓ_2)	88
6.2.2	Ajustes en Normas Robustas (ℓ_1 y ℓ_∞)	88
6.3	Regresión Lineal Múltiple y Selección de Características	90
6.3.1	Selección de Variables mediante Programación Entera Mixta	90
6.4	Máquinas de Vectores de Soporte (SVM)	91
6.5	Explicabilidad y Contrafacticos (Counterfactuals)	92
6.5.1	Formulación del Problema de Optimización	92
6.5.2	Formulación Entera Mixta para un Clasificador de Regresión Logística	93
6.6	Implementación Práctica en Python	93
7	Introducción a Grafos y Árboles Soporte	95
7.1	Origen Histórico y Definiciones Básicas	96
7.1.1	El Problema de Königsberg (Euler, 1736)	96
7.1.2	Definición Formal de un Grafo	96
7.1.3	Subgrafos, Conectividad y Grados	97
7.2	Representación de Grafos	97
7.2.1	Representación Geométrica y Grafos Planares	97
7.2.2	Representación en Ordenador	99
7.3	Estructura Matemática de los Árboles	99
7.4	Árbol Soporte de Mínimo Peso (MST)	99
7.4.1	Algoritmo de Kruskal (1956)	100
7.4.2	Algoritmo de Prim	100
7.5	Implementación en Python	101

8	El Problema del Camino Mínimo	103
8.1	Definición Formal y Condiciones de Existencia	104
8.2	El Principio de Optimalidad de Bellman	104
8.3	Redes Sin Circuitos (DAGs) y Ordenación Topológica	105
8.3.1	Ordenación Topológica	105
8.3.2	Algoritmo de Resolución	105
8.4	Redes con Pesos No Negativos: Algoritmo de Dijkstra	106
8.5	Redes con Pesos Negativos: Algoritmo de Bellman-Ford	106
8.5.1	Algoritmo Paso a Paso	106
8.6	Formulación en Programación Lineal Primal-Dual	107
8.6.1	Modelo Primal (Flujo de Coste Mínimo de Una Unidad)	107
8.6.2	Modelo Dual (Potenciales Nodales)	107
8.7	Camino Mínimo Entre Todos los Pares de Nodos: Floyd-Warshall	108
8.8	Implementación en Python con NetworkX	109
9	Problemas de Flujo en Redes	111
9.1	Estructura Algebraica de Flujos y Ciclos	111
9.1.1	Definición de Flujo y Circulación	112
9.2	El Lema de Coloración de Arcos de Minty (1966)	112
9.3	El Problema del Flujo Máximo	113
9.3.1	Cortes y Capacidad de un Corte	114
9.3.2	Algoritmo de Ford-Fulkerson (Camino Aumentante)	114
9.4	Problema de Flujo de Coste Mínimo y Transbordo	115
9.5	Implementación en Python con NetworkX	116
10	Problemas de Emparejamiento (Matching)	118
10.1	Conceptos Fundamentales de Emparejamiento	119
10.1.1	Definición de Emparejamiento	119
10.1.2	Cadenas Alternantes e Incrementales	120
10.2	Búsqueda mediante Árboles Alternantes	120
10.3	Problemas de Recubrimiento en Grafos	121
10.3.1	Definición de Recubrimientos y Conjuntos Independientes	121
10.4	Emparejamiento de Máximo Peso y Algoritmo Húngaro	122
10.5	Implementación en Python con NetworkX	122
11	Rutas Eulerianas, Hamiltonianas y TSP	124
11.1	Grafos Eulerianas	125
11.2	El Problema del Cartero Chino (CPP)	125
11.2.1	Estrategia de Resolución en Grafos No Dirigidos	125
11.3	Ciclos Hamiltonianos	126
11.3.1	Teoremas de Suficiencia de Hamiltonianidad	126
11.4	El Problema del Viajante (TSP)	127
11.4.1	Formulación Lineal Entera DFJ (Dantzig-Fulkerson-Johnson, 1954) . . .	128

11.4.2	Formulación Lineal Entera MTZ (Miller-Tucker-Zemlin, 1960)	128
11.4.3	Heurísticas y Algoritmos de Aproximación	129
11.5	Implementación en Python con NetworkX	130
Conclusiones		132
	Resumen del aprendizaje	132
	Reflexiones finales	133
	Proyección futura: El valor del rigor matemático	134
Bibliografía		135

Prefacio

La optimización matemática y el análisis de redes constituyen dos de las ramas más dinámicas y aplicadas de las matemáticas modernas. En un mundo cada vez más interconectado y dependiente del análisis de datos para la toma de decisiones, la capacidad de modelar sistemas complejos, optimizar recursos limitados y analizar la topología de flujos de información se ha convertido en una competencia fundamental para matemáticos, científicos de datos y profesionales de la investigación operativa. Este libro está diseñado para proporcionar una comprensión profunda, rigurosa y práctica de estas técnicas, sirviendo de guía de estudio para la asignatura de **Optimización y Análisis de Redes** en el **Grado en Matemáticas**.

A lo largo de los capítulos, exploraremos un recorrido metodológico completo. En la primera parte, nos sumergiremos en la optimización continua, analizando la optimización no lineal desde sus bases algorítmicas univariantes y multivariantes, el formalismo teórico del análisis convexo y las condiciones fundamentales de optimalidad (KKT), hasta su implementación práctica en problemas cuadráticos y de ciencia de datos mediante **CVXPY**. En la segunda parte, trasladaremos nuestro enfoque a las estructuras discretas de redes, resolviendo problemas clásicos de camino mínimo, flujos máximos, emparejamientos y enrutamiento óptimo (TSP) empleando la librería **NetworkX**.

El objetivo de este texto es doble: por un lado, asentar los fundamentos algebraicos, analíticos y geométricos que justifican la existencia y propiedades de los óptimos y los flujos; por otro, dotar al lector de las destrezas de programación para modelar y resolver de forma eficiente problemas del mundo real. Al finalizar esta obra, habrás adquirido un criterio sólido para analizar la complejidad de un problema de decisión, seleccionar el algoritmo idóneo para su resolución e interpretar los resultados con sentido crítico.

Filosofía pedagógica del volumen

Siguiendo el enfoque de nuestra obra de referencia, la filosofía de estos apuntes se sustenta en un método **“teórico-práctico”** sin concesiones. Creemos firmemente que la labor computacional y el desarrollo de código no deben reducirse a una mera aplicación ciega de funciones o recetas preestablecidas. El verdadero potencial de un matemático reside en comprender el *porqué* detrás de cada paso algorítmico, el comportamiento de las restricciones en el plano geométrico y la estructura algebraica que garantiza la convergencia, tal y como se promueve en los tratados clásicos de la disciplina (Hillier y Lieberman 2010).

Es por ello que cada concepto se introduce motivando su necesidad histórica y práctica, se formaliza mediante teoremas y lemas detallados, y finalmente se traduce a código ejecutable en Python, promoviendo una simbiosis natural entre el rigor teórico y la destreza computacional.

¿Qué aprenderás con este libro?

Al completar este volumen, habrás desarrollado habilidades clave para:

- **Modelar** problemas complejos de decisión y redes utilizando variables de decisión continuas, enteras y binarias.
- **Analizar** la convexidad de funciones y conjuntos para garantizar la existencia de óptimos globales.
- **Resolver e interpretar** sistemas de optimalidad mediante las condiciones de Karush-Kuhn-Tucker (KKT).
- **Implementar** modelos de optimización robustos en Python utilizando la librería de optimización convexa **CVXPY**.
- **Manipular y analizar** redes complejas mediante la librería **NetworkX**, comprendiendo la eficiencia computacional de los algoritmos implementados.
- **Formular e interpretar** los problemas duales asociados a caminos mínimos y flujos en redes, relacionando las variables duales con potenciales nodales y tensiones.
- **Diseñar y evaluar** heurísticas constructivas y de mejora local (como 2-opt) y algoritmos de aproximación (como Christofides) para problemas NP-duros.

! Grado en Matemáticas

Este libro presenta el material teórico y práctico de la asignatura de Optimización y Análisis de Redes del Grado en Matemáticas de la Universidad Rey Juan Carlos. Su contenido está estrechamente vinculado con las asignaturas de Álgebra Lineal, Cálculo Multivariable, Programación y Optimización Lineal.

! Conocimientos previos

Es altamente recomendable que los alumnos dominen los conceptos básicos de espacios vectoriales, matrices totalmente unimodulares, derivabilidad multivariable (gradientes, jacobianos y hessianas) y los fundamentos de la programación lineal (método Simplex y dualidad clásica).

i Sobre los autores

Víctor Aceña Gil es graduado en Matemáticas por la UNED, máster en Tratamiento Estadístico y Computacional de la Información por la UCM y la UPM, doctor en Tecno-

logías de la Información y las Comunicaciones por la URJC y profesor del departamento de Informática y Estadística de la URJC. Miembro del grupo de investigación de alto rendimiento en Fundamentos y Aplicaciones de la Ciencia de Datos, DSLAB, de la URJC. Pertenece al grupo de innovación docente, DSLAB-TI.

Antonio Alonso Ayuso es catedrático de Universidad en el área de Estadística e Investigación Operativa del departamento de Informática y Estadística de la Universidad Rey Juan Carlos. Es especialista en optimización matemática continua, entera y estocástica, con amplia experiencia en la resolución de problemas de redes, logística y toma de decisiones bajo incertidumbre.

Esta obra está bajo una licencia de Creative Commons Atribución-CompartirIgual 4.0 Internacional.

1 Introducción a la Investigación Operativa

La **Investigación Operativa** (conocida internacionalmente como *Operations Research* o *Management Science*) es la disciplina científica que aplica métodos analíticos avanzados para dar soporte a la toma de decisiones complejas en sistemas reales. Lejos de constituir un mero conjunto de herramientas analíticas o algoritmos aislados, representa una metodología sistemática y estructurada. Combina la modelización matemática, el análisis estadístico y la optimización para diseñar, gestionar y mejorar sistemas complejos bajo condiciones de recursos limitados (Hillier y Lieberman 2010).

En este capítulo de apertura, estableceremos las bases conceptuales y epistemológicas de la disciplina. Exploraremos el origen histórico de la Investigación Operativa, formalizaremos el concepto de modelo matemático, detallaremos las fases fundamentales del ciclo de modelización y describiremos los elementos estructurales de un problema de optimización. Finalmente, introduciremos las dos grandes vertientes que estructuran este volumen: la optimización no lineal y la optimización en redes.

! Objetivos de aprendizaje

Al finalizar este capítulo, serás capaz de:

1. **Definir** el alcance, el origen y las herramientas esenciales de la Investigación Operativa en el ámbito profesional y académico.
2. **Diferenciar** entre modelos físicos (concretos) y matemáticos (abstractos), comprendiendo el equilibrio crítico entre realismo y manejabilidad matemática.
3. **Describir y aplicar** las cuatro fases del ciclo de vida de un proyecto de modelización matemática.
4. **Estructurar formalmente** un problema de optimización matemática, identificando sus variables de decisión, restricciones y función objetivo.
5. **Clasificar** los problemas de optimización atendiendo a criterios de linealidad, dominio de variables y presencia de incertidumbre.

1.1. Definición e Historia de la Investigación Operativa

1.1.1. Raíces Históricas y el Nacimiento de la Disciplina

La Investigación Operativa no nació en laboratorios académicos aislados, sino en el fragor de la necesidad estratégica y militar durante la Segunda Guerra Mundial. A finales de la década de 1930, el mando militar británico se enfrentaba a la inminente amenaza aérea del régimen nazi. Para contrarrestarla, desarrollaron el sistema de radares costeros, pero se percataron rápidamente de que disponer de la tecnología no bastaba: era imperativo aprender a utilizarla de manera integrada y eficiente.

Fue entonces cuando el físico británico Patrick Blackett organizó un grupo multidisciplinar de científicos (que incluía matemáticos, físicos, astrofísicos y militares) conocido históricamente como el “**Blackett’s Circus**” (el Circo de Blackett). Este equipo analizó científicamente las operaciones militares reales, logrando hitos memorables como la determinación del tamaño óptimo de los convoyes navales de suministro para minimizar las pérdidas causadas por los submarinos alemanes, o la calibración de la profundidad de detonación de las cargas antisubmarinas en los bombarderos.

Paralelamente, en la Unión Soviética, el matemático y economista **Leonid Kantorovich** desarrolló de forma independiente técnicas de planificación matemática para la industria del carbón y del acero en 1939. Kantorovich sentó las bases de lo que hoy conocemos como Programación Lineal, trabajo que le valdría el Premio Nobel de Economía en 1975 por su teoría sobre la asignación óptima de recursos.

Tras la finalización del conflicto mundial, en 1947, el matemático estadounidense **George Dantzig** revolucionó la disciplina al proponer el **método del Símplex**, un algoritmo sistemático y extremadamente eficiente para resolver problemas de programación lineal. Este descubrimiento coincidió con la invención y el rápido desarrollo de las primeras computadoras electrónicas, lo que permitió aplicar las metodologías de la Investigación Operativa a la planificación industrial, la logística comercial, el sector financiero y la administración pública. La rápida expansión económica de la posguerra demandaba una gestión científica y optimizada de las grandes corporaciones, consolidando el éxito de esta disciplina (Bazaraa, Jarvis, y Sherali 2011).

1.1.2. Definiciones Formales

Existen dos definiciones clásicas promovidas por las instituciones profesionales de referencia a nivel mundial:

- **The Operational Research Society (Reino Unido):** La Investigación Operativa es la aplicación del método científico a problemas complejos en la dirección y gestión de grandes sistemas de personas, máquinas, materiales y recursos financieros en la industria, el comercio, la administración pública y la defensa. Su enfoque característico reside en

construir un modelo científico del sistema, incorporando factores de riesgo e incertidumbre, para predecir y comparar las consecuencias de diversas decisiones, estrategias y controles alternativos.

- **Institute for Operations Research and the Management Sciences (INFORMS, EE.UU.):** La Investigación Operativa se define como el empleo de la ciencia de la decisión y la modelización analítica avanzada para resolver problemas y mejorar el rendimiento de sistemas y procesos de negocio complejos.

1.1.3. Herramientas de la Investigación Operativa

Para dar respuesta a problemas de diversa naturaleza, la Investigación Operativa se sirve de un amplio catálogo de metodologías científicas:

- **Optimización matemática:** Modelos lineales, enteros, no lineales y dinámicos orientados a maximizar rendimientos o minimizar costes bajo restricciones de recursos.
- **Teoría de grafos y optimización en redes:** Algoritmos discretos para resolver problemas de conectividad, distribución, transporte y caminos óptimos.
- **Procesos estocásticos y teoría de colas:** Modelización del comportamiento de sistemas sujetos a aleatoriedad y análisis de líneas de espera en sistemas de servicios, logística y telecomunicaciones.
- **Simulación computacional:** Simulación analítica (como los métodos de Montecarlo o la simulación de eventos discretos) para evaluar el rendimiento de sistemas complejos cuando las soluciones analíticas cerradas resultan inviables.
- **Teoría de juegos y decisión multicriterio:** Análisis de interacciones estratégicas entre agentes competitivos y metodologías para tomar decisiones cuando existen múltiples objetivos contrapuestos.

1.2. El Concepto de Modelo en Ciencias de la Decisión

Un **modelo** es una representación simplificada, abstracta y selectiva de un sistema real destinada a facilitar la comprensión de su estructura interna y predecir su comportamiento bajo diversos escenarios.

La labor del modelador matemático es similar a la del cartógrafo. Tal y como enunció el filósofo Alfred Korzybski, *“el mapa no es el territorio”*. Un mapa a escala 1:1 sería perfectamente inútil por su inmanejabilidad. De igual modo, un modelo matemático que pretenda incluir cada mínimo detalle físico del sistema real resultará intratable computacionalmente. La clave reside en realizar una **abstracción selectiva**: identificar y capturar las variables esenciales del problema eliminando el ruido irrelevante, logrando un equilibrio óptimo entre fidelidad (realismo) y manejabilidad (capacidad de resolución).

1.2.1. Clasificación General de los Modelos

Atendiendo a su naturaleza física o abstracta, los modelos se dividen clásicamente en:

- **Modelos concretos o icónicos:** Representaciones físicas a escala del sistema real. Ejemplos de esto son las maquetas de aviones ensayadas en túneles de viento o los modelos tridimensionales de proteínas. Su análisis se realiza mediante experimentación directa.
- **Modelos abstractos o analíticos:** Representaciones formales donde las variables del sistema y sus relaciones lógicas se expresan mediante ecuaciones, inecuaciones y funciones matemáticas. Son el objeto de estudio de la optimización y permiten una manipulación analítica o numérica asistida por computadoras.

Desde la perspectiva de su propósito metodológico, podemos catalogar los modelos matemáticos en tres niveles de madurez analítica:

1. **Modelos Descriptivos:** Representan la estructura del sistema para responder a la pregunta “¿qué está ocurriendo?”.
2. **Modelos Predictivos:** Utilizan datos históricos y estadística para responder a la pregunta “¿qué podría ocurrir en el futuro?”.
3. **Modelos Prescriptivos:** Integran los niveles anteriores para responder a la pregunta “¿qué decisión debemos tomar para optimizar el rendimiento del sistema?”. La optimización matemática pertenece a esta última categoría.

1.2.2. El Ciclo de Vida del Modelado Matemático

La resolución de un problema mediante la Investigación Operativa no consiste en un proceso lineal, sino en un ciclo iterativo de retroalimentación constante que consta de cuatro fases diferenciadas:

1. **Observación y definición del problema:** Se estudian los límites del sistema real y se identifican las metas del decisor. Se recopilan, depuran y analizan los datos empíricos relevantes del problema.
2. **Formulación matemática:** Se eligen las variables que representan las decisiones controlables, se describen las limitaciones físicas y operativas mediante restricciones (ecuaciones e inecuaciones) y se formaliza la métrica de éxito o coste como una función objetivo.
3. **Validación y análisis de sensibilidad:** Se resuelve el modelo numéricamente y se contrasta si las soluciones obtenidas son coherentes con la realidad del sistema. Se evalúa la robustez del modelo mediante el análisis de sensibilidad, midiendo cómo varían las decisiones óptimas ante perturbaciones en los parámetros del modelo.

4. **Implementación e integración:** Se traduce la solución matemática en decisiones operativas ejecutables por la organización. Se diseñan sistemas informáticos de soporte a la decisión y se establecen mecanismos de supervisión para detectar cambios en el sistema real que exijan reformular el modelo.

1.3. Formulación General de Modelos de Optimización

Un modelo de optimización matemática busca seleccionar la mejor alternativa posible dentro de un conjunto de soluciones factibles. Su formulación matemática genérica se escribe formalmente como:

$$\begin{aligned} & \underset{x}{\text{mín}} \text{ o } \underset{x}{\text{máx}} && f(x) \\ & \text{sujeto a} && g_i(x) \leq 0, \quad i = 1, \dots, m \\ & && h_j(x) = 0, \quad j = 1, \dots, p \\ & && x \in X \subseteq \mathbb{R}^n \end{aligned}$$

Donde los componentes esenciales del modelo son:

- **Variables de decisión** (x): El vector $x = (x_1, x_2, \dots, x_n)^T$ de dimensiones n que representa los factores bajo control directo del decisor que se desea determinar.
- **Función objetivo** ($f(x)$): La función escalar de las variables de decisión que mide la efectividad, el coste, el tiempo o el beneficio del sistema y que se pretende optimizar (minimizar o maximizar).
- **Restricciones de desigualdad** ($g_i(x) \leq 0$) **e igualdad** ($h_j(x) = 0$): Ecuaciones o inecuaciones que representan límites físicos (capacidad, stock), económicos (presupuesto), técnicos o de balance de flujo del sistema. Definen el **espacio factible** (o región factible) $\mathcal{S}$ del problema.
- **Espacio de definición** (X): El dominio general donde viven las variables (por ejemplo, números reales, enteros, no negativos, etc.).

Ejemplo Ilustrativo: La Dieta Óptima (Formulación Paso a Paso)

Supongamos que deseamos diseñar una dieta de bajo coste utilizando dos alimentos básicos: Leche (x_1) y Verdura (x_2). Disponemos de los siguientes datos de coste y nutrición:

- Un litro de leche cuesta 0.80 € y aporta 3 unidades de calcio y 1 unidad de hierro.
- Un kilo de verdura cuesta 1.20 € y aporta 1 unidad de calcio y 2 unidades de hierro.
- Los requisitos mínimos diarios de un individuo son 6 unidades de calcio y 4 unidades de hierro.

Para formular el modelo matemático de este problema operativo:

1. **Variables de decisión:**

- x_1 : Litros de leche consumidos al día.
- x_2 : Kilos de verdura consumidos al día.

2. **Función objetivo (minimizar el coste total diario):**

$$\text{mín} \quad 0.80x_1 + 1.20x_2$$

3. **Restricciones:**

- Requisito de Calcio: $3x_1 + 1x_2 \geq 6 \implies 6 - 3x_1 - x_2 \leq 0$
- Requisito de Hierro: $1x_1 + 2x_2 \geq 4 \implies 4 - x_1 - 2x_2 \leq 0$
- No negatividad (no se puede consumir comida negativa): $x_1, x_2 \geq 0$

1.3.1. Clasificación de los Modelos de Optimización

Atendiendo a las propiedades algebraicas y analíticas de sus componentes, los problemas de optimización se clasifican de la siguiente manera:

▪ **Según la linealidad:**

- *Programación Lineal (LP)*: Tanto la función objetivo $f(x)$ como todas las funciones de restricciones $g_i(x)$ e $h_j(x)$ son funciones afines (de primer grado). Permiten la aplicación del algoritmo del Símplex y métodos de punto interior. Su resolución numérica es sumamente rápida, pudiendo manejar millones de variables y restricciones.
- *Programación No Lineal (NLP)*: Al menos una de las funciones involucradas (f , g_i o h_j) presenta curvatura o componentes no lineales (como potencias, exponenciales, logaritmos o funciones trigonométricas). La búsqueda del óptimo requiere herramientas de cálculo multivariable y análisis de curvatura.

▪ **Según el dominio de definición:**

- *Optimización Continua*: El espacio de búsqueda X es continuo ($x \in \mathbb{R}^n$). Esto nos permite explotar las derivadas de la función (gradiente, hessiana) para dirigir la búsqueda.
- *Optimización Entera o Discreta (IP)*: Las variables de decisión solo pueden tomar valores en el conjunto de los números enteros ($x \in \mathbb{Z}^n$). Estos problemas son combinatorios y presentan una alta dificultad de resolución (son NP-duros).
- *Optimización Binaria*: Las variables se restringen a $x_j \in \{0, 1\}$, representando decisiones lógicas excluyentes del tipo “sí” o “no” (por ejemplo, construir una planta o no).

- *Optimización Entera Mixta (MIP)*: Coexisten variables continuas y enteras/binarias dentro del mismo modelo, permitiendo modelar simultáneamente decisiones lógicas y cantidades físicas continuas.

■ **Según la presencia de incertidumbre:**

- *Optimización Determinista*: Todos los coeficientes y parámetros del modelo se conocen con absoluta certeza.
- *Optimización Estocástica*: Algunos parámetros del modelo se comportan como variables aleatorias con distribuciones de probabilidad conocidas. Se busca optimizar el valor esperado o cumplir restricciones de probabilidad (restricciones de azar).

1.4. La Estructura del Libro: Optimización No Lineal y en Redes

Este volumen se organiza en dos bloques metodológicos bien diferenciados por su naturaleza matemática, geométrica y algorítmica:

1.4.1. Bloque I: Optimización No Lineal (Temas 2 a 6)

Cuando abandonamos la hipótesis de linealidad, la optimización matemática gana sustancialmente en realismo, pero incrementa de manera drástica su complejidad teórica y computacional.

- **Métodos Numéricos (Tema 2)**: Estudiaremos las herramientas fundamentales de la optimización continua sin restricciones en una y múltiples dimensiones. Analizaremos algoritmos de búsqueda unidimensional (sección áurea, Fibonacci, bisección, Newton) y métodos multivariantes basados en el gradiente (descenso de gradiente, método de Newton y técnicas Quasi-Newton como BFGS).
- **Análisis Convexo (Tema 3)**: Estableceremos las propiedades de los conjuntos y funciones convexas. En este marco matemático se formula el resultado cardinal de la optimización: cualquier mínimo local es de forma garantizada un mínimo global, dotando a los algoritmos numéricos de una gran estabilidad. Introduciremos el concepto de subgradiente para manejar funciones no diferenciables.
- **Condiciones KKT (Temas 4 y 5)**: Analizaremos las condiciones necesarias y suficientes de optimalidad para problemas restringidos mediante los multiplicadores de Lagrange y las condiciones de Karush-Kuhn-Tucker. Estudiaremos su aplicación a la programación lineal y cuadrática (el algoritmo del Simplex de Wolfe).
- **Ciencia de Datos (Tema 6)**: Veremos la aplicación de la optimización continua en el aprendizaje automático moderno, analizando la regresión robusta bajo diferentes normas (ℓ_1 , ℓ_2 , ℓ_∞), las Máquinas de Vectores de Soporte (SVM) y la explicabilidad mediante contrafacticos.

1.4.2. Bloque II: Optimización en Redes (Temas 7 a 11)

La optimización en redes aprovecha la estructura combinatorial de los grafos para resolver problemas de gran envergadura de forma extremadamente eficiente.

- **Representación y Estructura (Tema 7):** Modelado formal de redes empleando matrices de adyacencia e incidencia. Analizaremos la propiedad de total unimodularidad, que permite resolver problemas discretos de redes como si fuesen continuos sin perder la integridad de las soluciones. Estudiaremos la obtención de árboles soporte mínimos (MST).
- **Caminos Mínimos (Tema 8):** Algoritmos clásicos de Dijkstra, Bellman-Ford y Floyd-Warshall. Abordaremos la formulación matemática dual del camino mínimo expresada como tensiones y potenciales de nodos.
- **Flujos en Redes (Tema 9):** Estudio del espacio vectorial de flujos. Analizaremos el Lema de Coloración de Minty y el teorema cardinal de Flujo Máximo - Corte Mínimo de Ford-Fulkerson.
- **Emparejamientos (Tema 10):** Teoría de emparejamientos y asignación óptima de tareas mediante el algoritmo Húngaro.
- **Rutas y TSP (Tema 11):** Planificación de rutas de reparto (el Cartero Chino y el TSP) mediante formulaciones lineales enteras con eliminación exponencial de subtours, heurísticas y algoritmos de aproximación.

2 Optimización No Lineal y Métodos Numéricos

La **optimización no lineal** (Nonlinear Programming, NLP) estudia problemas de decisión donde la función objetivo o alguna de las restricciones presentan comportamientos no lineales. A diferencia de la programación lineal, donde las soluciones óptimas se encuentran siempre en la frontera del conjunto factible (específicamente en sus puntos extremos o vértices), en optimización no lineal el óptimo puede situarse en el interior del espacio factible o en cualquier punto de su frontera. La presencia de curvatura en la función objetivo y en las restricciones exige herramientas avanzadas de análisis diferencial multivariable y algoritmos iterativos complejos.

En este capítulo, estudiaremos la formulación general de los problemas de optimización no lineal, su clasificación en familias estructurales específicas y su interpretación geométrica. Asimismo, desarrollaremos la fundamentación matemática basada en el gradiente y la matriz hessiana para caracterizar la curvatura local y establecer las condiciones de optimalidad. Por último, abordaremos de forma analítica y práctica los principales métodos numéricos de búsqueda unidimensional (1D) y multivariable sin restricciones, complementándolos con código numérico y simbólico en Python.

! Objetivos de aprendizaje

Al finalizar este capítulo, serás capaz de:

1. **Formular y clasificar** problemas de optimización no lineal en sus distintas familias estructurales.
2. **Visualizar e interpretar geométricamente** la diferencia entre la optimización lineal y la no lineal a través del análisis de conjuntos factibles convexos y curvas de nivel no lineales.
3. **Caracterizar la curvatura** de una función multivariable mediante el análisis del gradiente y de la matriz hessiana empleando el criterio de los menores principales.
4. **Clasificar puntos estacionarios** aplicando las condiciones de optimalidad de primer y segundo orden (FONC, SONC, SOSOC).
5. **Implementar y trazar numéricamente** algoritmos de búsqueda lineal en una dimensión, tanto sin derivadas como con derivadas.
6. **Comprender y programar** algoritmos de optimización multivariable sin restricciones, analizando el comportamiento de las coordenadas cíclicas, el fenómeno de zigzag

del máximo descenso y las ventajas de los métodos de Newton y Cuasi-Newton (BFGS).

2.1. Introducción y Formulación de Problemas No Lineales

En el mundo real, la linealidad es una excepción matemática. Los fenómenos físicos, económicos y tecnológicos se comportan de manera inherentemente no lineal. En la industria, por ejemplo, las **economías de escala** provocan que el coste unitario de producción disminuya a medida que aumenta el volumen debido al reparto de costes fijos o descuentos por cantidad, lo que rompe la relación de proporcionalidad lineal. En el ámbito económico, la **elasticidad de la demanda** determina que la cantidad demandada de un bien sea sensible a su precio de venta, por lo que la función de ingresos resultante ($I = P \cdot Q$) es cuadrática y no lineal. En el sector financiero, siguiendo el modelo de selección de carteras de Markowitz, el riesgo se mide mediante la varianza del rendimiento de los activos, que constituye una combinación cuadrática de los pesos invertidos, mientras que el rendimiento esperado se mantiene lineal. Finalmente, en la ciencia de datos, la estimación de modelos y algoritmos de aprendizaje automático suele penalizar los errores de ajuste de forma cuadrática (mínimos cuadrados), generando funciones objetivo no lineales.

Un **problema de optimización no lineal (PNL)** general se define matemáticamente sobre un espacio vectorial $\mathbb{R}^n$ como:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{sujeto a} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

Donde $f : \mathbb{R}^n \rightarrow \mathbb{R}$ es la función objetivo, $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$ representa las restricciones de desigualdad, e $h_j : \mathbb{R}^n \rightarrow \mathbb{R}$ representa las restricciones de igualdad. Para que el problema sea clasificado como no lineal, al menos una de estas funciones debe exhibir un comportamiento no lineal en el dominio de interés.

La transición de modelos lineales a no lineales altera profundamente la geometría del espacio de búsqueda. En programación lineal (LP), el conjunto factible es siempre un poliedro convexo y la solución óptima se localiza en uno de sus vértices. Esto permite que algoritmos como el Simplex resuelvan problemas de millones de variables explorando únicamente un número finito de candidatos. En optimización no lineal (NLP), las restricciones no lineales pueden deformar el conjunto factible, y el óptimo puede hallarse en el interior del espacio factible o en cualquier punto de la frontera. Además, la presencia de no convexidades puede generar múltiples óptimos locales, atrapando a los algoritmos numéricos antes de alcanzar el mínimo global.

2.1.1. Casos Particulares de la Optimización No Lineal

Para abordar la resolución de problemas no lineales de forma eficiente, la disciplina los clasifica en familias estructuradas según las propiedades algebraicas y analíticas de sus funciones:

La **Programación Cuadrática (QP)** representa problemas donde la función objetivo es cuadrática y las restricciones son afines (lineales), formulándose como $\min_x \frac{1}{2}x^T Qx - b^T x$ sujeto a $Ax \leq d$. Si la matriz Q es semidefinida positiva, el problema es convexo, lo que permite el uso de algoritmos especializados extremadamente eficientes como los métodos de conjuntos activos (*active-set*) o de punto interior, que resuelven el problema de forma exacta en pocas iteraciones.

La **Programación Convexa** constituye la clase de problemas más robusta en optimización no lineal. Un problema es convexo si la función objetivo $f(x)$ es convexa, las restricciones de desigualdad $g_i(x)$ son convexas y las restricciones de igualdad $h_j(x)$ son afines. La gran ventaja analítica de la programación convexa es que todo mínimo local es un mínimo global. Esto elimina la necesidad de realizar búsquedas globales exhaustivas y garantiza que métodos locales de gradiente converjan siempre a la solución óptima del sistema.

La **Programación Separable** se presenta cuando todas las funciones involucradas se pueden descomponer en la suma de funciones que dependen exclusivamente de una sola variable independiente, es decir, $f(x) = \sum_{j=1}^n f_j(x_j)$ y $g_i(x) = \sum_{j=1}^n g_{ij}(x_j) \leq 0$. Esta propiedad estructural es muy útil, ya que permite aproximar funciones no lineales arbitrarias mediante funciones lineales a trozos (*piecewise linear approximation*), transformando el problema en un modelo lineal de variables continuas que se puede resolver con variantes del algoritmo del Simplex.

La **Programación Geométrica** surge cuando la función objetivo y las restricciones están modeladas por posinomios (sumas de términos monomios con coeficientes reales positivos de la forma $cx_1^{a_1}x_2^{a_2}\dots x_n^{a_n}$). Aunque un problema geométrico es inherentemente no convexo en sus variables originales, la aplicación de un cambio de variables logarítmico de la forma $y_j = \ln(x_j)$ transforma el modelo en un problema convexo equivalente en el espacio y , permitiendo su resolución exacta de forma global.

La **Programación Fraccionaria** se presenta cuando la función objetivo se formula como el cociente de dos funciones reales, es decir, $\min_x f_1(x)/f_2(x)$. Aparece habitualmente al optimizar ratios de eficiencia, rentabilidad de inversiones o productividad. Bajo condiciones de concavidad y convexidad de las funciones individuales, la transformación de variables de Charnes-Cooper permite linealizar el modelo y resolverlo de forma exacta.

2.1.2. Caso de Estudio 1: El Monopolio de Cournot (1838)

Consideremos una empresa que opera en régimen de monopolio y desea maximizar su beneficio. La relación entre la cantidad demandada del mercado Q y el precio de venta P viene dada por

la función de demanda inversa del mercado: $P(Q) = 10 - Q$. El coste total de producción de la empresa es lineal con respecto a la cantidad fabricada: $C(Q) = 1 + 3Q$. La empresa está sujeta a una limitación de capacidad física máxima de producción de 9 unidades ($Q \leq 9$). Por su parte, el organismo regulador estatal establece que el precio de venta al público no puede superar las 8 unidades monetarias ($P \leq 8$).

Para determinar la política óptima de producción y precios, formulamos el problema como un modelo con dos variables de decisión (Q y P):

$$\begin{aligned} \max_{Q,P} \quad & B(Q, P) = P \cdot Q - 3Q - 1 \\ \text{sujeto a} \quad & Q \leq 10 - P \\ & 0 \leq P \leq 8 \\ & 0 \leq Q \leq 9 \end{aligned}$$

Este problema presenta una función objetivo no lineal debido al término cuadrático $P \cdot Q$. Dado que la empresa busca maximizar el beneficio y el precio está acotado por la demanda, la solución óptima operará exactamente sobre la frontera de la curva de demanda activa (donde la restricción $Q \leq 10 - P$ se cumple como igualdad: $P = 10 - Q$). Sustituyendo el precio en la función de beneficio, reducimos el problema a una única variable real Q :

$$B(Q) = (10 - Q)Q - 3Q - 1 = -Q^2 + 7Q - 1$$

El rango de factibilidad de Q se obtiene combinando las restricciones: $Q \leq 9$ y $P \leq 8 \implies 10 - Q \leq 8 \implies Q \geq 2$. Por lo tanto, el problema de una sola variable se reduce a:

$$\max_Q \quad -Q^2 + 7Q - 1 \quad \text{sujeto a} \quad 2 \leq Q \leq 9$$

Para hallar el óptimo, derivamos el beneficio con respecto a la cantidad de producción e igualamos a cero: $B'(Q) = -2Q + 7 = 0 \implies Q^* = 3.5$. Como $Q^* = 3.5$ pertenece al intervalo factible $[2, 9]$, constituye el óptimo global del problema. Sustituyendo, obtenemos el precio óptimo $P^* = 6.5$ y el beneficio máximo de la empresa: $B^* = -(3.5)^2 + 7(3.5) - 1 = 11.25$ u.m.

La región factible en el plano (Q, P) es un polígono convexo con vértices en $(0, 0)$, $(0, 8)$, $(2, 8)$, $(9, 1)$, y $(9, 0)$. Las curvas de nivel de la función objetivo (curvas de isobeneficio) son hipérbolas de la forma $P = 3 + (B + 1)/Q$. A medida que el beneficio B aumenta, las hipérbolas se desplazan hacia arriba y a la derecha. El punto óptimo $(Q^*, P^*) = (3.5, 6.5)$ es el punto donde la curva de isobeneficio correspondiente a $B = 11.25$ es exactamente tangente al segmento de la frontera $P + Q = 10$, tal y como se ilustra en la Figura 2.1.

Este resultado ilustra la diferencia fundamental con la programación lineal: la solución óptima de un problema no lineal con restricciones lineales puede situarse en el interior de una de las

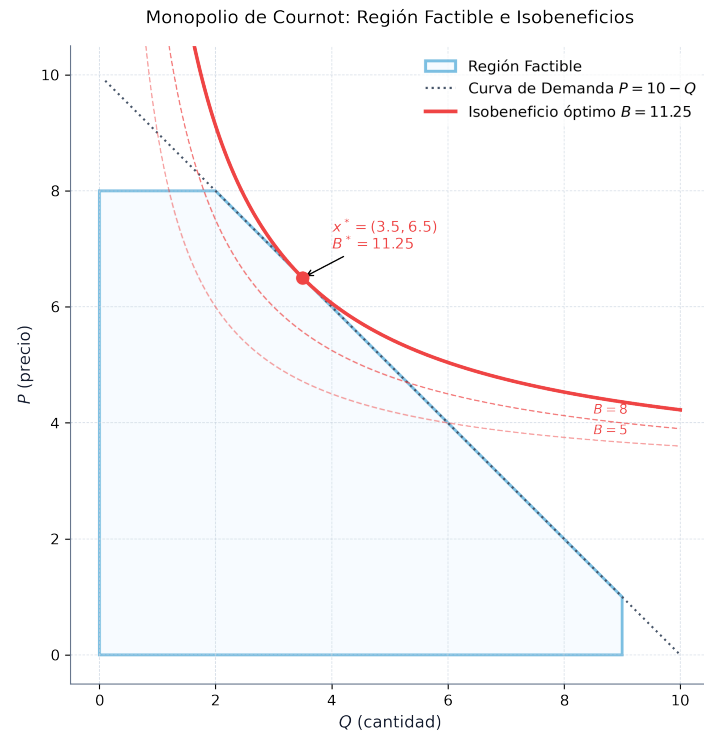


Figura 2.1: Geometría del Monopolio de Cournot: el óptimo de tangencia $(3.5, 6.5)$ se sitúa en la frontera y no coincide con ningún vértice de la región factible polimórfica

caras de la región factible y no en un vértice (los vértices adyacentes son $(2, 8)$ y $(9, 1)$, los cuales arrojan beneficios inferiores, de 9.0 y 8.0 u.m. respectivamente).

2.1.3. Caso de Estudio 2: Selección de Cartera de Valores (Markowitz, 1952)

Supongamos que disponemos de un capital C para invertir entre n activos financieros. Denotamos por x_i la cantidad monetaria asignada al activo i . Cada activo presenta un rendimiento medio esperado μ_i , y la relación de variabilidad conjunta del mercado se modela a través de la covarianza σ_{ij} entre los activos i y j . La varianza de la cartera (medida del riesgo o volatilidad) viene dada por la forma cuadrática $\sum_{i=1}^n \sum_{j=1}^n \sigma_{ij} x_i x_j$.

El inversor busca maximizar el rendimiento esperado de la cartera al tiempo que minimiza su riesgo, penalizándolo mediante un coeficiente de aversión al riesgo $\beta > 0$. El problema se formula como:

$$\begin{aligned} & \underset{x \in \mathbb{R}^n}{\text{máx}} && \sum_{i=1}^n \mu_i x_i - \beta \sum_{i=1}^n \sum_{j=1}^n \sigma_{ij} x_i x_j \\ \text{sujeto a} &&& \sum_{i=1}^n x_i \leq C \\ &&& x_i \geq 0, \quad i = 1, \dots, n \end{aligned}$$

Este problema posee una función objetivo cuadrática no lineal y restricciones lineales, lo que constituye un problema clásico de **Programación Cuadrática (QP)**. Su naturaleza estrictamente convexa (cuando la matriz de covarianzas es semidefinida positiva) asegura que cualquier óptimo local sea también el óptimo global del sistema (Boyd y Vandenberghe 2004; Hillier y Lieberman 2010).

2.2. Tipos de Solución en Optimización No Lineal

Sea $f : \mathbb{R}^n \rightarrow \mathbb{R}$ una función real definida sobre un conjunto factible $\mathcal{X} \subseteq \mathbb{R}^n$. Un punto $x^* \in \mathcal{X}$ es un **mínimo global** si no existe ningún otro punto factible con un valor inferior de la función objetivo, es decir, $f(x) \geq f(x^*)$ para todo $x \in \mathcal{X}$. Del mismo modo, x^* es un **mínimo global estricto** si se cumple que $f(x) > f(x^*)$ para cualquier $x \in \mathcal{X} \setminus \{x^*\}$.

Por otro lado, un punto x^* constituye un **mínimo local** si es la mejor solución dentro de un entorno local delimitado por una bola de radio $\epsilon > 0$ centrada en él, satisfaciendo $f(x) \geq f(x^*)$ para todo $x \in \mathcal{X}$ tal que $\|x - x^*\| \leq \epsilon$. Finalmente, x^* es un **mínimo local estricto** si existe un radio $\epsilon > 0$ tal que $f(x) > f(x^*)$ para todo $x \in \mathcal{X} \setminus \{x^*\}$ con $\|x - x^*\| \leq \epsilon$.

En optimización no lineal general, debido a la posible presencia de no convexidades, los algoritmos locales basados en derivadas pueden converger hacia mínimos locales mediocres en

lugar de alcanzar el óptimo global. Esta es la complejidad fundamental que diferencia al PNL de la programación lineal.

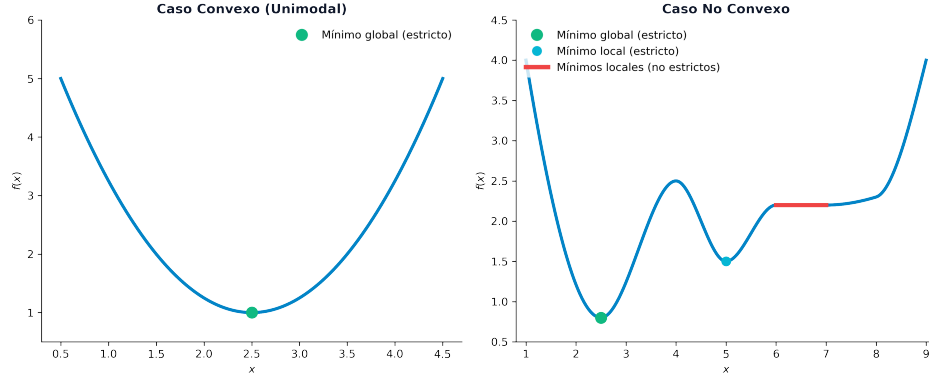


Figura 2.2: Ilustración de los distintos tipos de óptimos locales y globales (estrictos y no estrictos) sobre funciones convexas y no convexas

2.3. Fundamentos Matemáticos y Condiciones de Optimalidad

Para caracterizar y clasificar los puntos óptimos locales en múltiples dimensiones, recurrimos al cálculo diferencial. Consideremos una función $f : \mathbb{R}^n \rightarrow \mathbb{R}$ diferenciable de clase $\mathcal{C}^2$ (con segundas derivadas continuas). El **vector gradiente** representa la dirección de máxima pendiente de subida local de la función y se define como el vector columna de derivadas parciales de primer orden: $\nabla f(x) = \left(\frac{\partial f(x)}{\partial x_1}, \frac{\partial f(x)}{\partial x_2}, \dots, \frac{\partial f(x)}{\partial x_n} \right)^T$. Por su parte, la **matriz hessiana** modela la curvatura local de la función (análoga a la segunda derivada en una dimensión) y se define como la matriz simétrica de segundas derivadas parciales $[\nabla^2 f(x)]_{ij} = \frac{\partial^2 f(x)}{\partial x_i \partial x_j}$.

2.3.1. Desarrollo de Taylor Multivariable

El comportamiento local de una función no lineal alrededor de un punto de referencia $\bar{x}$ se aproxima mediante su desarrollo cuadrático de Taylor:

$$f(x) = f(\bar{x}) + \nabla f(\bar{x})^T (x - \bar{x}) + \frac{1}{2} (x - \bar{x})^T \nabla^2 f(\bar{x}) (x - \bar{x}) + o(\|x - \bar{x}\|^2)$$

Si la función $f(x)$ es cuadrática, esta aproximación de segundo orden es exacta para todo $x \in \mathbb{R}^n$.

2.3.2. Caracterización de Curvatura y Clasificación de Matrices

La curvatura local de la función está determinada por el carácter de su matriz hessiana $\nabla^2 f(x)$, el cual se clasifica analizando la forma cuadrática $z^T \nabla^2 f(x) z$:

- **Definida positiva** ($\nabla^2 f(x) \succ 0$): Si $z^T \nabla^2 f(x) z > 0$ para todo vector no nulo $z \in \mathbb{R}^n$. Todos sus autovalores son estrictamente positivos y todos sus menores principales son mayores que cero. La función se curva hacia arriba en forma de cuenco, lo que garantiza que un punto crítico sea un mínimo local estricto.
- **Semidefinida positiva** ($\nabla^2 f(x) \succeq 0$): Si $z^T \nabla^2 f(x) z \geq 0$ para todo vector $z \in \mathbb{R}^n$. Todos sus autovalores son no negativos y todos sus menores principales son no negativos.
- **Definida negativa** ($\nabla^2 f(x) \prec 0$): Si $z^T \nabla^2 f(x) z < 0$ para todo vector no nulo $z \in \mathbb{R}^n$. Sus menores principales alternan de signo comenzando por negativo ($M_1 < 0, M_2 > 0, M_3 < 0, \dots$) y todos sus autovalores son estrictamente negativos. La función se curva hacia abajo en forma de cúpula.
- **Semidefinida negativa** ($\nabla^2 f(x) \preceq 0$): Si $z^T \nabla^2 f(x) z \leq 0$ para todo vector z . Todos sus autovalores son no positivos y todos sus menores principales alternan de signo de forma no estricta o son nulos.
- **Indefinida**: Si existen autovalores estrictamente positivos y negativos. El punto crítico resultante constituye un **punto de silla** (con direcciones de subida y direcciones de bajada).

Ejemplos: Clasificación de Matrices

1. **Matriz Definida Positiva**: Sea $A_1 = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. Sus menores principales son $M_1 = |2| = 2 > 0$, y $M_2 = \det(A_1) = 4 - 1 = 3 > 0$. Al ser ambos menores principales estrictamente positivos, $A_1 \succ 0$. Resolviendo el polinomio característico $\det(A_1 - \lambda I) = (2 - \lambda)^2 - 1 = 0 \implies \lambda^2 - 4\lambda + 3 = 0$, obtenemos los autovalores $\lambda_1 = 3$ y $\lambda_2 = 1$, los cuales confirman que es definida positiva.
2. **Matriz Indefinida**: Sea $A_2 = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$. Sus menores principales son $M_1 = |1| = 1 > 0$, y $M_2 = \det(A_2) = 1 - 4 = -3 < 0$. Dado que el determinante es negativo en dimensión 2, la matriz es indefinida. Resolviendo $\det(A_2 - \lambda I) = (1 - \lambda)^2 - 4 = 0 \implies \lambda^2 - 2\lambda - 3 = 0$, hallamos los autovalores $\lambda_1 = 3$ (positivo) y $\lambda_2 = -1$ (negativo), lo que ratifica el carácter indefinido.
3. **Matriz Semidefinida Positiva**: Sea $A_3 = \begin{pmatrix} 2 & -3 & -1 \\ -3 & 12 & 6 \\ -1 & 6 & 3.2 \end{pmatrix}$. Sus menores principales son $M_1 = |2| = 2 > 0$, $M_2 = 24 - 9 = 15 > 0$, y $M_3 = \det(A_3) = 2(38.4 - 36) + 3(-9.6 + 6) - 1(-18 + 12) = 4.8 - 10.8 + 6 = 0$. Al ser todos los

menores principales no negativos y el determinante nulo, la matriz es semidefinida positiva y singular. Sus autovalores son $\lambda_1 \approx 15.75$, $\lambda_2 \approx 1.45$, y $\lambda_3 = 0$, confirmando la semidefinición ($A_3 \succeq 0$).

2.3.3. Condiciones de Optimalidad y Puntos Estacionarios

Las condiciones diferenciales para determinar extremos locales se enuncian en el siguiente teorema.

! Teorema: Condiciones de Optimalidad de Segundo Orden

Sea $f : \mathbb{R}^n \rightarrow \mathbb{R}$ una función diferenciable de clase $\mathcal{C}^2$:

1. **Condición Necesaria de Primer Orden (FONC)**: Si x^* es un mínimo local, entonces el gradiente de la función se anula en dicho punto:

$$\nabla f(x^*) = 0$$

Cualquier punto que verifique esta condición se denomina **punto estacionario** o punto crítico.

2. **Condición Necesaria de Segundo Orden (SONC)**: Si x^* es un mínimo local, entonces además de cumplirse la FONC, la matriz hessiana en dicho punto es semidefinida positiva:

$$\nabla^2 f(x^*) \succeq 0$$

3. **Condición Suficiente de Segundo Orden (SOSC)**: Si un punto x^* satisface simultáneamente que:

$$\nabla f(x^*) = 0 \quad \text{y} \quad \nabla^2 f(x^*) \succ 0$$

Entonces x^* es un **mínimo local estricto** de la función.

! Importancia de la Definición Positiva en SOSC

Es fundamental notar que en la condición suficiente (SOSC), la matriz hessiana debe ser **estrictamente definida positiva** ($\nabla^2 f(x^*) \succ 0$) y no simplemente semidefinida positiva. Si la hessiana es semidefinida con algún autovalor nulo (por ejemplo, en la función $f(x) = x^4$ en el origen $x^* = 0$, donde $f'(0) = 0$ y $f''(0) = 0 \succeq 0$), el análisis de segundo orden es insuficiente para garantizar analíticamente si el punto es un mínimo, un máximo o un punto de inflexión.

Ejemplos: Clasificación de Puntos Estacionarios

- **Punto de Silla:** Sea $f(x, y) = \frac{1}{2}x^2 + 2xy + \frac{1}{2}y^2 - y + 9$. Su gradiente es $\nabla f(x, y) = (x + 2y, 2x + y - 1)^T$. Igualando a cero obtenemos el punto crítico en $x^* = 2/3, y^* = -1/3$. La matriz hessiana es $\nabla^2 f = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$. Como su menor principal de segundo orden es $M_2 = -3 < 0$, la matriz es indefinida, lo que clasifica al punto $(2/3, -1/3)$ como un **punto de silla**.
- **Mínimo Local Estricto:** Sea $f(x, y) = (x - 2)^2 + (y - 1)^2$. Su gradiente es $\nabla f(x, y) = (2(x - 2), 2(y - 1))^T$, anulándose en $(x^*, y^*) = (2, 1)$. La matriz hessiana es $\nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$. Al ser definida positiva ($M_1 = 2 > 0, M_2 = 4 > 0$), el punto crítico constituye un **mínimo local estricto**.
- **Función Cuadrática Acoplada:** Sea $f(x, y) = 8x^2 + 3xy + 7y^2 - 25x + 31y - 29$. Su gradiente es $\nabla f(x, y) = (16x + 3y - 25, 3x + 14y + 31)^T$, el cual se anula en el punto crítico $x^* \approx 2.060, y^* \approx -2.656$. La matriz hessiana es $\nabla^2 f = \begin{pmatrix} 16 & 3 \\ 3 & 14 \end{pmatrix}$. Sus menores principales son $M_1 = 16 > 0$ y $M_2 = 215 > 0$, por lo que la matriz es definida positiva, clasificando al punto como un **mínimo local estricto**.

2.3.4. Resolución Simbólica con Python y SymPy

El análisis algebraico de gradientes, matrices hessianas y sistemas de ecuaciones no lineales puede automatizarse de forma exacta utilizando la biblioteca de computación simbólica **SymPy** de Python. El siguiente fragmento de código ilustra cómo definir la función acoplada del ejemplo anterior, calcular de forma analítica su gradiente y su matriz hessiana, y resolver el sistema $\nabla f(x, y) = 0$ para encontrar las coordenadas exactas de su punto crítico:

```
import sympy as sp

# 1. Definir las variables simbólicas reales
x, y = sp.symbols('x y', real=True)

# 2. Definir la función objetivo cuadrática acoplada
f = 8*x**2 + 3*x*y + 7*y**2 - 25*x + 31*y - 29

# 3. Calcular el gradiente simbólico (derivadas parciales de primer orden)
grad_f = [sp.diff(f, var) for var in (x, y)]
print("Gradiente Simbólico:")
print(f" df/dx = {grad_f[0]}")
print(f" df/dy = {grad_f[1]}\n")
```

```

# 4. Calcular la matriz Hessiana simbólica
H = sp.hessian(f, (x, y))
print("Matriz Hessiana:")
sp.pprint(H)
print()

# 5. Clasificar la matriz calculando sus menores principales
m1 = H[0, 0]
m2 = H.det()
print(f"Menor principal M1 = {m1}")
print(f"Menor principal M2 = {m2}")
if m1 > 0 and m2 > 0:
    print("La matriz es definida positiva.\n")

# 6. Resolver el sistema grad_f = 0 para hallar los puntos críticos
soluciones = sp.solve(grad_f, (x, y))
print(f"Punto crítico exacto: (x, y) = ({soluciones[x]}, {soluciones[y]})")
print(f"Punto crítico aproximado: (x, y) = ({float(soluciones[x]):.4f}, {float(soluciones[y]):.4f})")

```

La ejecución de este código devuelve de manera inmediata la solución exacta $(x^*, y^*) = (443/215, -571/215)$, lo que demuestra cómo las herramientas de software facilitan la validación rápida de resultados analíticos complejos.

2.4. Métodos Numéricos de Búsqueda Lineal (1D)

Los métodos de búsqueda unidimensional buscan minimizar una función real de una variable $f : \mathbb{R} \rightarrow \mathbb{R}$ en un intervalo acotado $[a, b]$. Son fundamentales porque resolver problemas multidimensionales complejos suele requerir resolver una sucesión de problemas unidimensionales a lo largo de direcciones de descenso: $\min_{\alpha > 0} f(x_k + \alpha d_k)$.

2.4.1. Métodos de Búsqueda Lineal sin Derivadas

Estos métodos solo requieren evaluar el valor de la función objetivo. Exigen que la función sea **unimodal** en el intervalo $[a, b]$, es decir, que posea un único mínimo local en dicho intervalo, decreciendo de forma monótona a la izquierda del mínimo y creciendo a su derecha.

La **Búsqueda Uniforme (Rejilla)** consiste en dividir el intervalo inicial $[a, b]$ en n subintervalos de igual anchura mediante una rejilla de puntos $x_k = a + k \frac{b-a}{n}$ para $k = 0, \dots, n$. Se evalúa la función en todos los puntos de la rejilla, se identifica el punto mínimo x_{k^*} y se define el nuevo

intervalo reducido como $[x_{k^*-1}, x_{k^*+1}]$. Aunque es un método muy robusto, resulta poco eficiente debido a que no aprovecha la información de las evaluaciones previas (ver la Figura 2.3).

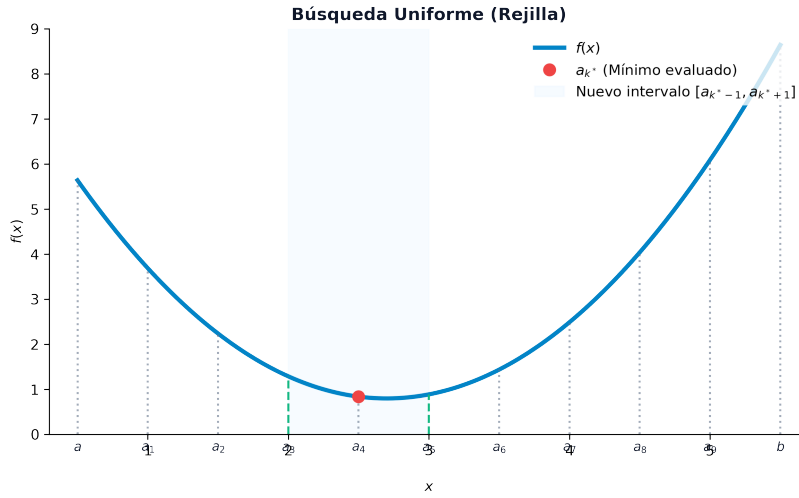


Figura 2.3: Concepto de la Búsqueda Uniforme en rejilla: subdivisión equidistante del intervalo y reducción basada en el punto mínimo de la muestra

La **Búsqueda Dicotómica** evalúa la función en el entorno del punto medio del intervalo actual. Para ilustrar por qué es necesario evaluar al menos dos puntos internos en lugar de uno solo, consideremos la Figura 2.4. Si únicamente evaluásemos la función en el punto medio exacto c , obtendríamos la misma información en ambos casos, haciendo imposible deducir si el mínimo se localiza en la mitad izquierda o derecha.

Para resolver este inconveniente, la Búsqueda Dicotómica calcula en cada iteración k dos puntos simétricos respecto al centro separados por una perturbación infinitesimal $\delta > 0$: $\lambda_k = \frac{a_k+b_k}{2} - \delta$ y $\mu_k = \frac{a_k+b_k}{2} + \delta$. Si $f(\lambda_k) < f(\mu_k)$, el mínimo debe situarse en el intervalo izquierdo, actualizándose a $[a_k, \mu_k]$, mientras que si $f(\lambda_k) > f(\mu_k)$, se actualiza a $[\lambda_k, b_k]$ (ver la Figura 2.5). Cada paso requiere dos evaluaciones y reduce el intervalo a prácticamente la mitad ($I_{k+1} \approx \frac{1}{2}I_k$).

La **Sección Áurea (Golden Section)** optimiza la búsqueda dicotómica al colocar los puntos de evaluación de manera que uno de ellos pueda ser reutilizado en la iteración siguiente. Para que se mantenga esta relación de escala constante entre iteraciones, la anchura de los intervalos decrece según la razón áurea $\alpha = (1 + \sqrt{5})/2 \approx 1.618$. Los puntos en cada paso k sobre el intervalo $[a_k, b_k]$ se definen como $\lambda_k = b_k - (b_k - a_k)/\alpha$ y $\mu_k = a_k + (b_k - a_k)/\alpha$. Este método reduce el intervalo de incertidumbre en cada paso en un factor constante de $1/\alpha \approx 0.618$. A partir de la segunda iteración, solo se requiere evaluar la función en un único punto nuevo. Dada una tolerancia $\epsilon > 0$, el número total de iteraciones necesarias se calcula mediante la expresión $N = \lceil (\ln(b - a) - \ln(\epsilon)) / \ln(\alpha) \rceil$.

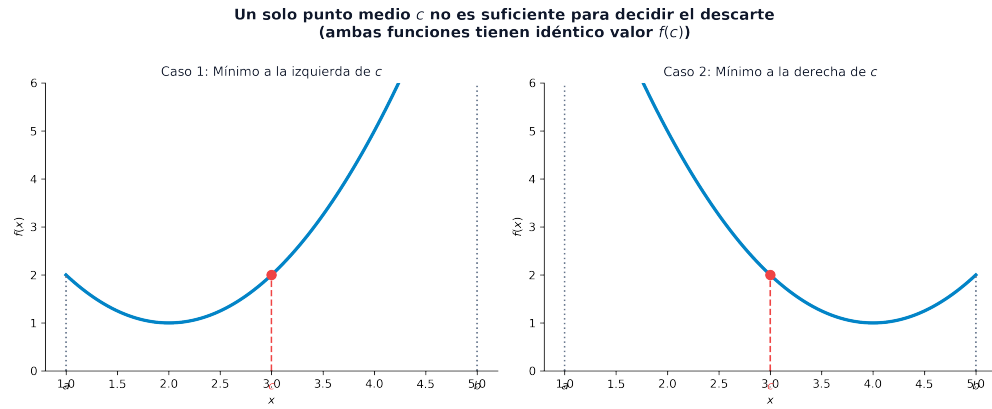


Figura 2.4: Insuficiencia de evaluar un único punto medio: dos funciones unimodales diferentes pueden tener idéntico valor en c teniendo sus mínimos globales en lados opuestos

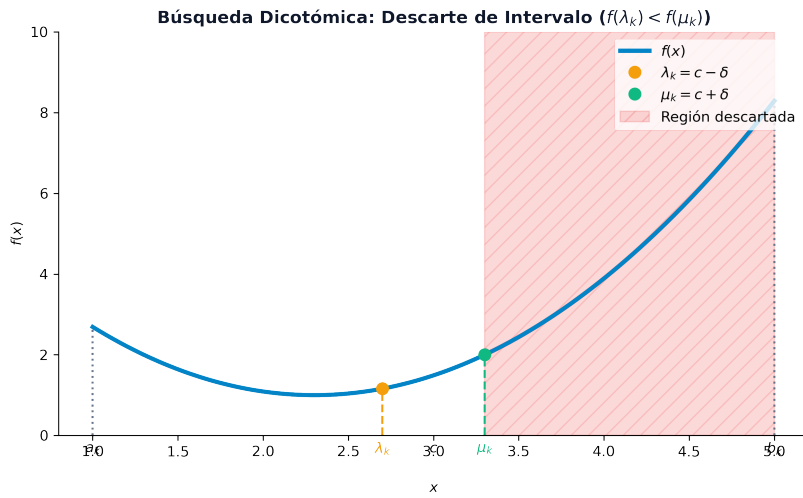


Figura 2.5: Concepto de la Búsqueda Dicotómica: la comparación de λ_k y μ_k descarta con certeza una de las mitades del intervalo

La **Búsqueda de Fibonacci** fija de antemano el número total de evaluaciones de función N y utiliza los términos de la sucesión de Fibonacci (F_k , donde $F_0 = 1, F_1 = 1, F_2 = 2, F_3 = 3, F_4 = 5, F_5 = 8, F_6 = 13, F_7 = 21, \dots$) para situar los puntos de evaluación. En la iteración k , los puntos se colocan en $\mu_k = a_k + \frac{F_{N-k}}{F_{N-k+1}}(b_k - a_k)$ y $\lambda_k = a_k + (1 - \frac{F_{N-k}}{F_{N-k+1}})(b_k - a_k)$. Proporciona la máxima reducción teórica del intervalo de incertidumbre para un número de pasos preestablecido. Aunque Fibonacci ofrece una reducción ligeramente mayor, la Sección Áurea suele ser preferida en la práctica porque no exige predeterminar el número total de iteraciones N .

A modo de ilustración de las trazas numéricas de estos algoritmos, consideremos la minimización en el intervalo $[1, 4]$ de la función de cuarto grado $f(x) = \frac{1}{4}x^4 - \frac{13}{3}x^3 + 27x^2 - 72x + 1$ (cuyo gráfico se muestra en la Figura 2.6) utilizando una tolerancia de precisión $\epsilon = 0.15$:

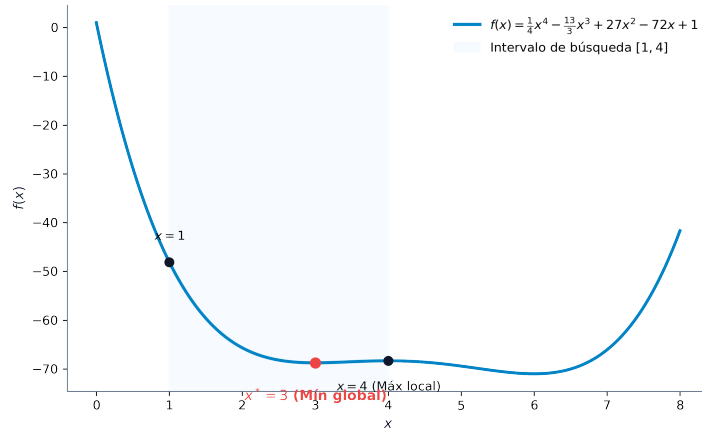


Figura 2.6: Minimización de la función de cuarto grado y su mínimo global en $x^* = 3.0$

■ *Traza de la Búsqueda Dicotómica:* Usamos una perturbación $\delta = 0.02$.

- *Iteración 1:* $[a_0, b_0] = [1, 4]$. El punto medio es $c = 2.5$. Evaluamos los puntos simétricos: $\lambda_0 = 2.5 - 0.02 = 2.48 \Rightarrow f(2.48) = -68.397$, y $\mu_0 = 2.5 + 0.02 = 2.52 \Rightarrow f(2.52) = -68.490$. Al cumplirse que $f(\lambda_0) > f(\mu_0)$, descartamos la porción izquierda $[a_0, \lambda_0]$. El nuevo intervalo es $[a_1, b_1] = [2.48, 4]$.
- *Iteración 2:* $[a_1, b_1] = [2.48, 4]$. El punto medio es $c = 3.24$. Evaluamos $\lambda_1 = 3.24 - 0.02 = 3.22 \Rightarrow f(3.22) = -68.730$, y $\mu_1 = 3.24 + 0.02 = 3.26 \Rightarrow f(3.26) = -68.683$. Al cumplirse que $f(\lambda_1) < f(\mu_1)$, descartamos la porción derecha $[\mu_1, b_1]$. El nuevo intervalo es $[a_2, b_2] = [2.48, 3.26]$.
- *Iteración 3:* $[a_2, b_2] = [2.48, 3.26]$. El punto medio es $c = 2.87$. Evaluamos $\lambda_2 = 2.87 - 0.02 = 2.85 \Rightarrow f(2.85) = -68.711$, y $\mu_2 = 2.87 + 0.02 = 2.89 \Rightarrow f(2.89) = -68.724$. Dado que $f(\lambda_2) > f(\mu_2)$, descartamos la izquierda. El nuevo intervalo es $[a_3, b_3] = [2.85, 3.26]$.
- *Iteración 4:* $[a_3, b_3] = [2.85, 3.26]$. El punto medio es $c = 3.055$. Evaluamos $\lambda_3 = 3.055 - 0.02 = 3.035 \Rightarrow f(3.035) = -68.744$, y $\mu_3 = 3.055 + 0.02 = 3.075 \Rightarrow$

$f(3.075) = -68.718$. Dado que $f(\lambda_3) < f(\mu_3)$, descartamos la derecha. El nuevo intervalo es $[a_4, b_4] = [2.85, 3.075]$.

- *Iteración 5:* $[a_4, b_4] = [2.85, 3.075]$. El punto medio es $c = 2.9625$. Evaluamos $\lambda_4 = 2.9625 - 0.02 = 2.9425 \Rightarrow f(2.9425) = -68.730$, y $\mu_4 = 2.9625 + 0.02 = 2.9825 \Rightarrow f(2.9825) = -68.748$. Como $f(\lambda_4) > f(\mu_4)$, descartamos la izquierda. El nuevo intervalo es $[a_5, b_5] = [2.9425, 3.075]$. La amplitud resultante es de $3.075 - 2.9425 = 0.1325 \leq 0.15$, finalizando el algoritmo.

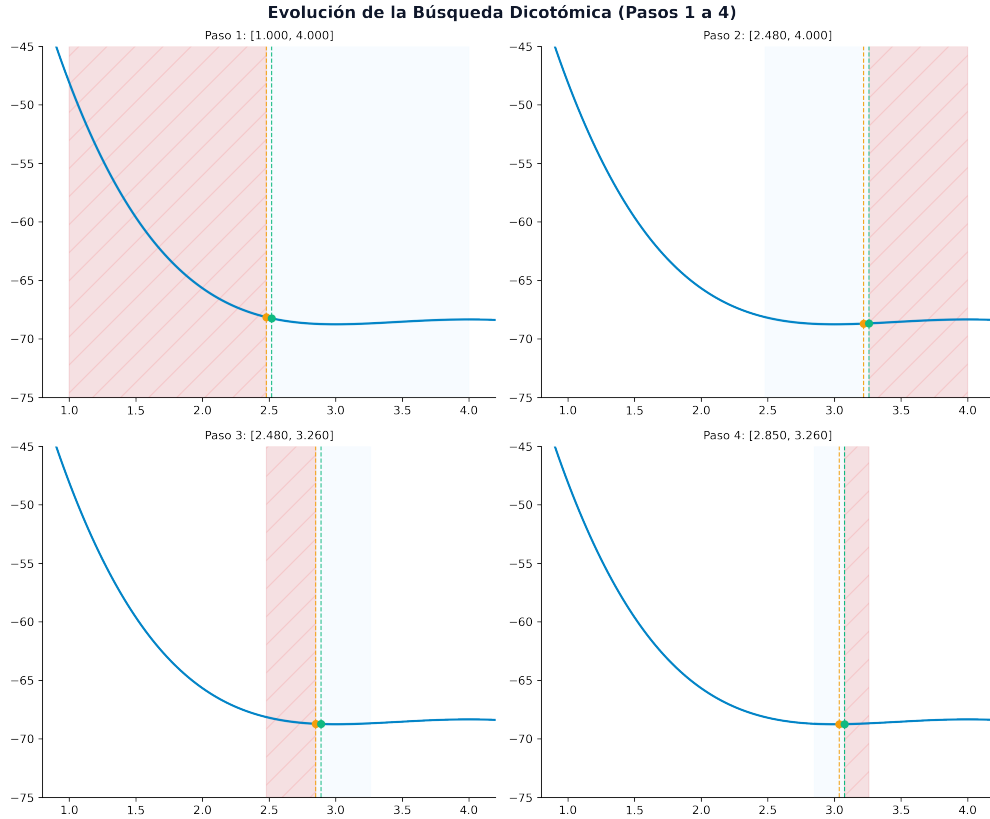


Figura 2.7: Evolución del intervalo en la Búsqueda Dicotómica (Pasos 1 a 4)

💡 Código Python: Búsqueda Dicotómica

```
f = lambda x: 0.25*x**4 - (13/3)*x**3 + 27*x**2 - 72*x + 1
a, b = 1.0, 4.0
tol = 0.15
delta = 0.02

print("Iteración | a_k      | b_k      | c_k      | lambda_k | mu_k      | f(lambda) | f(mu)")
print("-" * 80)
for k in range(1, 10):
    c = (a + b) / 2
    lmbda = c - delta
    mu = c + delta
    f_l = f(lmbda)
    f_m = f(mu)
    print(f"{k:9d} | {a:7.4f} | {b:7.4f} | {c:7.4f} | {lmbda:8.4f} | {mu:7.4f} | {f_l:9.4f} | {f_m:9.4f}")
    if f_l < f_m:
        b = mu
    else:
        a = lmbda
    if (b - a) < tol:
        break
```

- *Traza de la Sección Áurea:* El número de iteraciones requeridas es $N = \lceil (\ln(3.0) - \ln(0.15)) / \ln(1.618) \rceil = 7$.
 - *Iteración 1:* $[a_0, b_0] = [1, 4]$. Amplitud $I_1 = 3.0/1.618 = 1.8541$. Calculamos $\lambda_0 = 4 - 1.8541 = 2.1459 \Rightarrow f(2.1459) = -66.692$, y $\mu_0 = 1 + 1.8541 = 2.8541 \Rightarrow f(2.8541) = -68.714$. Como $f(\lambda_0) > f(\mu_0)$, descartamos la izquierda. Nuevo intervalo: $[2.1459, 4]$.
 - *Iteración 2:* $[a_1, b_1] = [2.1459, 4]$. Amplitud $I_2 = 1.8541/1.618 = 1.1459$. Reutilizamos $\lambda_1 = \mu_0 = 2.8541 \Rightarrow f(2.8541) = -68.714$. Calculamos el nuevo punto $\mu_1 = 2.1459 + 1.1459 = 3.2918 \Rightarrow f(3.2918) = -68.654$. Como $f(\lambda_1) < f(\mu_1)$, descartamos la derecha. Nuevo intervalo: $[2.1459, 3.2918]$.
 - *Iteración 7 (Última):* El proceso continúa hasta que en la iteración $k = 7$ la amplitud es $I_7 = 0.1033 \leq 0.15$, convergiendo en el intervalo final $[2.9574, 3.0608]$, que contiene el mínimo en $x^* = 3.0$.

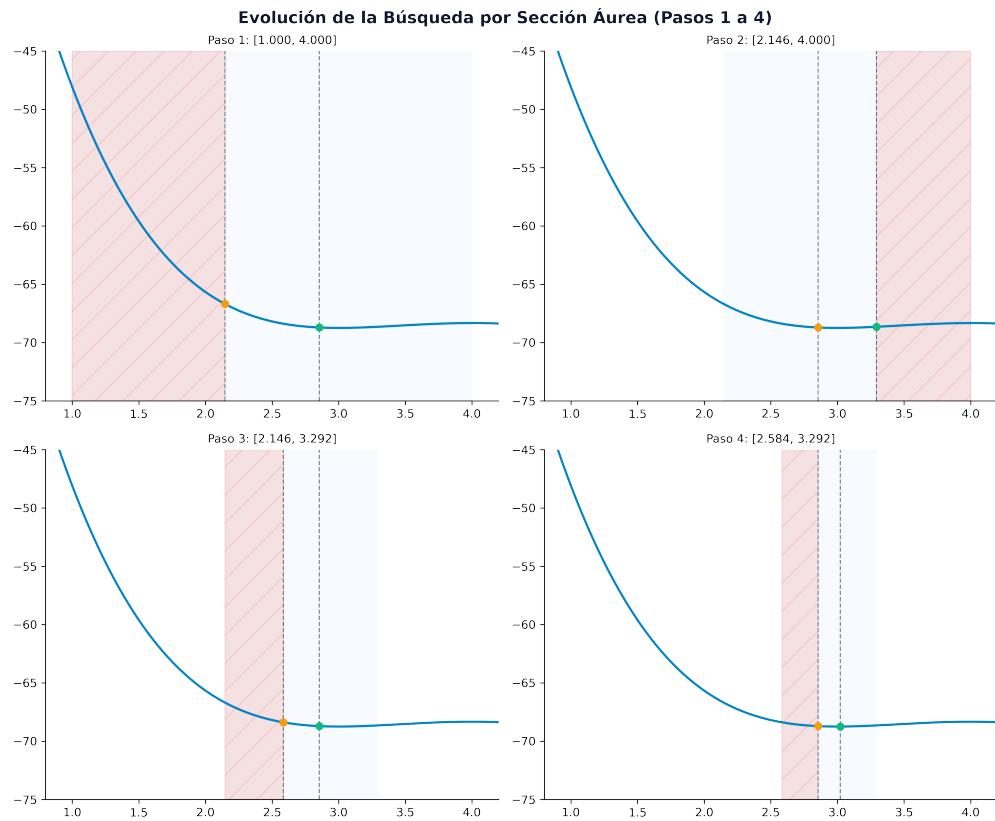


Figura 2.8: Evolución del intervalo en la búsqueda por Sección Áurea (Pasos 1 a 4)

💡 Código Python: Sección Áurea

```
import math
f = lambda x: 0.25*x**4 - (13/3)*x**3 + 27*x**2 - 72*x + 1
a, b = 1.0, 4.0
tol = 0.15
alpha = (1 + math.sqrt(5)) / 2

lmbda = b - (b - a) / alpha
mu = a + (b - a) / alpha
f_l = f(lmbda)
f_m = f(mu)

print("Iteración | a_k      | b_k      | lmbda_k | mu_k      | f(lmbda) | f(mu)")
print("-" * 75)
print(f"{{1:9d}} | {{a:7.4f}} | {{b:7.4f}} | {{lmbda:8.4f}} | {{mu:7.4f}} | {{f_l:9.4f}} | {{f_m:8.4f}}")

n_iter = int(math.ceil((math.log(b - a) - math.log(tol)) / math.log(alpha)))
for k in range(2, n_iter + 1):
    if f_l < f_m:
        b = mu
        mu = lmbda
        f_m = f_l
        lmbda = b - (b - a) / alpha
        f_l = f(lmbda)
    else:
        a = lmbda
        lmbda = mu
        f_l = f_m
        mu = a + (b - a) / alpha
        f_m = f(mu)
    print(f"{{k:9d}} | {{a:7.4f}} | {{b:7.4f}} | {{lmbda:8.4f}} | {{mu:7.4f}} | {{f_l:9.4f}} | {{f_m:8.4f}}")
```

- *Traza de Fibonacci*: Fijamos $N = 6$ evaluaciones de la función (usando $F_7 = 21$).
 - *Iteración 1*: $[a_0, b_0] = [1, 4]$. Amplitud del paso 1: $I_1 = \frac{13}{21} \cdot 3.0 \approx 1.8571$. Calculamos $\lambda_0 = 4 - 1.8571 = 2.1429 \Rightarrow f(2.1429) = -66.673$, y $\mu_0 = 1 + 1.8571 = 2.8571 \Rightarrow f(2.8571) = -68.715$. Como $f(\lambda_0) > f(\mu_0)$, descartamos la izquierda. Nuevo intervalo: $[2.1429, 4]$.
 - *Iteración 2*: $[a_1, b_1] = [2.1429, 4]$. Amplitud $I_2 = \frac{8}{21} \cdot 3.0 \approx 1.1429$. Reutilizamos $\lambda_1 = \mu_0 = 2.8571$. Calculamos $\mu_1 = 2.1429 + 1.1429 = 3.2857 \Rightarrow f(3.2857) = -68.657$. Como $f(\lambda_1) < f(\mu_1)$, descartamos la derecha. Nuevo intervalo: $[2.1429, 3.2857]$.

- *Iteración 6 (Última)*: La amplitud se reduce a $I_6 = 0.1429 \leq 0.15$, logrando la convergencia en $[2.8541, 3.001]$ con una iteración menos que la sección áurea gracias al uso de la serie de Fibonacci.

💡 Código Python: Búsqueda de Fibonacci

```
f = lambda x: 0.25*x**4 - (13/3)*x**3 + 27*x**2 - 72*x + 1
a, b = 1.0, 4.0
n = 6
fib = [1, 1]
for i in range(2, n + 2):
    fib.append(fib[-1] + fib[-2])

lmbda = a + (1 - fib[n] / fib[n+1]) * (b - a)
mu = a + (fib[n] / fib[n+1]) * (b - a)
f_l = f(lmbda)
f_m = f(mu)

print("Iteración | a_k      | b_k      | lmbda_k | mu_k      | f(lmbda) | f(mu)")
print("-" * 75)
print(f"{1:9d} | {a:7.4f} | {b:7.4f} | {lmbda:8.4f} | {mu:7.4f} | {f_l:9.4f} | {f_m:8.4f}")

for k in range(2, n):
    if f_l < f_m:
        b = mu
        mu = lmbda
        f_m = f_l
        lmbda = a + (1 - fib[n-k+1] / fib[n-k+2]) * (b - a)
        f_l = f(lmbda)
    else:
        a = lmbda
        lmbda = mu
        f_l = f_m
        mu = a + (fib[n-k+1] / fib[n-k+2]) * (b - a)
        f_m = f(mu)
    print(f"{k:9d} | {a:7.4f} | {b:7.4f} | {lmbda:8.4f} | {mu:7.4f} | {f_l:9.4f} | {f_m:8.4f}")
```

2.4.2. Métodos de Búsqueda Lineal con Derivadas

Los métodos con derivadas incorporan información analítica del gradiente para acelerar la convergencia o resolver ecuaciones.

El **Método de Bisección (Bolzano)** busca el cero de la derivada de primer orden ($f'(x) = 0$). Requiere que los signos de la derivada en los extremos sean opuestos ($f'(a) < 0$ y $f'(b) > 0$), lo que garantiza, por el teorema de Bolzano, la existencia de al menos un punto estacionario en el interior. En cada iteración, evalúa la derivada en el punto medio $c = (a + b)/2$. Si $f'(c) < 0$, actualiza $a = c$; si $f'(c) > 0$, actualiza $b = c$. El algoritmo reduce el intervalo a la mitad en cada paso.

El **Método de Newton 1D** aproxima la función objetivo en cada paso mediante un polinomio cuadrático de Taylor de segundo orden en el punto actual x_k y salta directamente al vértice de la parábola: $x_{k+1} = x_k - f'(x_k)/f''(x_k)$. Presenta una convergencia cuadrática local (orden 2), lo que significa que el número de dígitos significativos de precisión se duplica en cada iteración cerca del óptimo. Requiere calcular la segunda derivada $f''(x)$ en cada paso y exige que sea estrictamente positiva ($f''(x_k) > 0$).

El **Método de la Secante** evita el cálculo de la segunda derivada del método de Newton aproximándola numéricamente por diferencias finitas a partir de los dos últimos puntos evaluados: $f''(x_k) \approx (f'(x_k) - f'(x_{k-1})) / (x_k - x_{k-1})$. Sustituyendo esta aproximación en la fórmula de Newton, el nuevo punto se calcula como $x_{k+1} = x_k - f'(x_k) \frac{x_k - x_{k-1}}{f'(x_k) - f'(x_{k-1})}$. Presenta una convergencia superlineal de orden ≈ 1.618 .

Minimizamos de nuevo la función $f(x) = \frac{1}{4}x^4 - \frac{13}{3}x^3 + 27x^2 - 72x + 1$ en $[1, 4]$ partiendo del punto inicial $x_0 = 1.0$. Las derivadas son $f'(x) = x^3 - 13x^2 + 54x - 72$ y $f''(x) = 3x^2 - 26x + 54$.

■ *Iteración de Bisección:* $[a_0, b_0] = [1, 4]$.

- *Iteración 1:* Punto medio $c_0 = 2.5 \Rightarrow f'(2.5) = -2.6225 < 0$. Descartamos la izquierda, nuevo intervalo $[2.5, 4]$.
- *Iteración 2:* Punto medio $c_1 = 3.25 \Rightarrow f'(3.25) = 0.5156 > 0$. Descartamos la derecha, nuevo intervalo $[2.5, 3.25]$.
- *Iteración 3:* $c_2 = 2.875 \Rightarrow f'(2.875) = -0.4433 < 0 \Rightarrow$ nuevo intervalo $[2.875, 3.25]$.
- *Iteración 4:* $c_3 = 3.0625 \Rightarrow f'(3.0625) = 0.1760 > 0 \Rightarrow$ nuevo intervalo $[2.875, 3.0625]$.
- *Iteración 5:* $c_4 = 2.96875 \Rightarrow f'(2.96875) = -0.0964 < 0 \Rightarrow$ nuevo intervalo $[2.96875, 3.0625]$. La amplitud es de $0.0938 \leq 0.15$, finalizando el algoritmo.

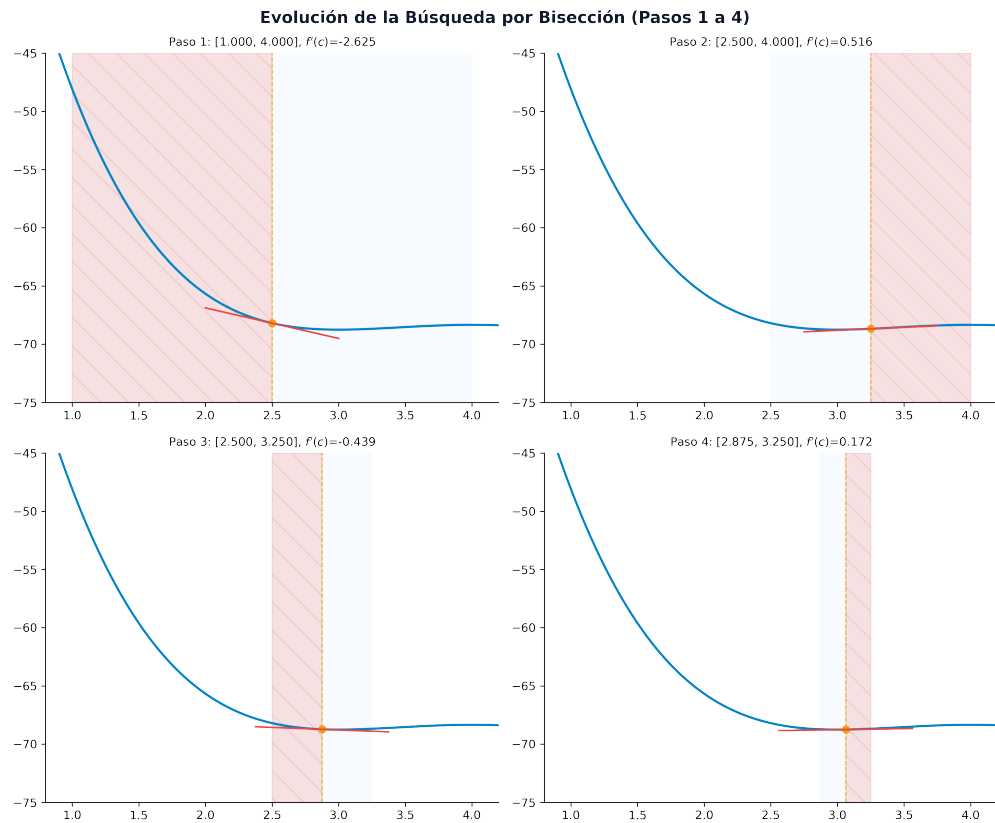


Figura 2.9: Evolución del intervalo en la búsqueda por Bisección (Pasos 1 a 4)

💡 Código Python: Método de Bisección

```
df = lambda x: x**3 - 13*x**2 + 54*x - 72
a, b = 1.0, 4.0
tol = 0.15

print("Iteración | a_k      | b_k      | c_k      | f'(c)")
print("-" * 55)
for k in range(1, 10):
    c = (a + b) / 2
    val_df = df(c)
    print(f"{k:9d} | {a:7.4f} | {b:7.4f} | {c:7.4f} | {val_df:8.4f}")
    if val_df < 0:
        a = c
    else:
        b = c
    if (b - a) < tol:
        break
```

■ *Iteración de Newton 1D:* Partimos de $x_0 = 1.0$.

- *Iteración 1:* $f'(1.0) = -30.0$, $f''(1.0) = 31.0 \Rightarrow x_1 = 1.0 - (-30.0/31.0) \approx 1.9677$.
- *Iteración 2:* $f'(1.9677) \approx -8.459$, $f''(1.9677) \approx 14.450 \Rightarrow x_2 = 1.9677 - (-8.459/14.450) \approx 2.5529$.
- *Iteración 3:* $f'(2.5529) \approx -2.235$, $f''(2.5529) \approx 7.177 \Rightarrow x_3 = 2.5529 - (-2.235/7.177) \approx 2.8637$.
- *Iteración 4:* $f'(2.8637) \approx -0.485$, $f''(2.8637) \approx 4.146 \Rightarrow x_4 = 2.8637 - (-0.485/4.146) \approx 2.9808$.
- *Iteración 5:* $f'(2.9808) \approx -0.059$, $f''(2.9808) \approx 3.154 \Rightarrow x_5 = 2.9808 - (-0.059/3.154) \approx 2.9995 \approx 3.0$. Convergencia en 5 iteraciones.

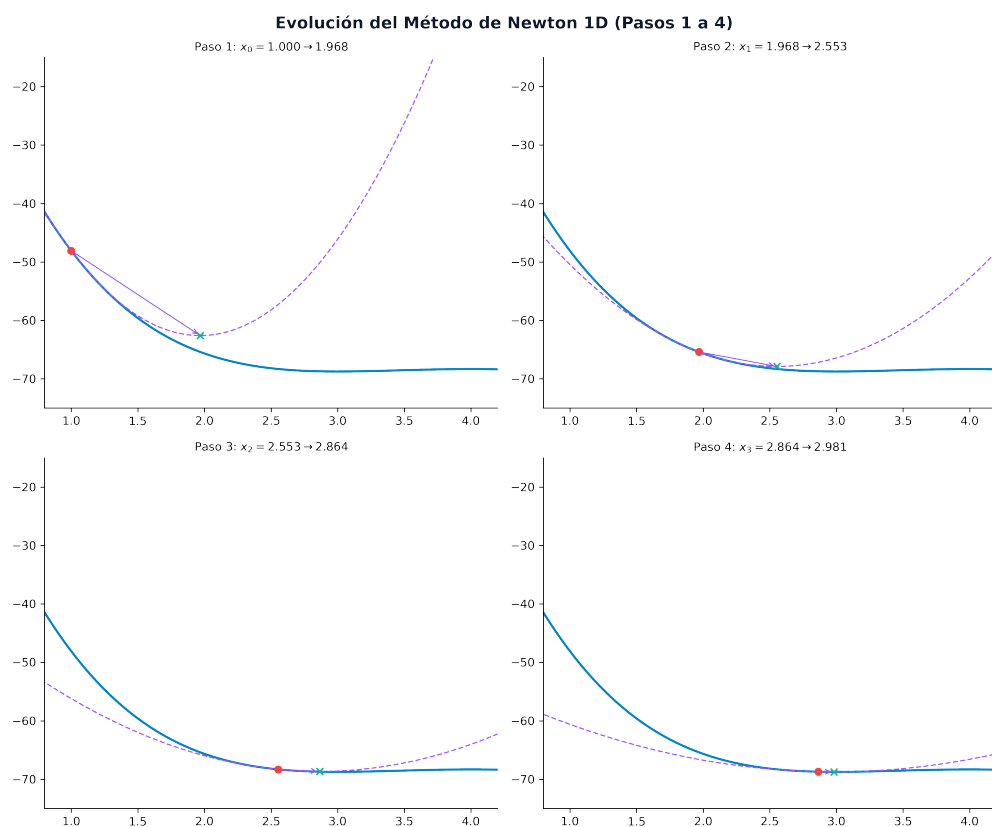


Figura 2.10: Aproximaciones cuadráticas en el método de Newton 1D (Pasos 1 a 4)

💡 Código Python: Método de Newton 1D

```
df = lambda x: x**3 - 13*x**2 + 54*x - 72
ddf = lambda x: 3*x**2 - 26*x + 54
x = 1.0
tol = 1e-5

print("Iteración | x_k      | f'(x_k) | f''(x_k) | x_{k+1}")
print("-" * 55)
for k in range(1, 10):
    val_df = df(x)
    val_ddf = ddf(x)
    x_new = x - val_df / val_ddf
    print(f"{k:9d} | {x:7.4f} | {val_df:8.4f} | {val_ddf:8.4f} | {x_new:7.4f}")
    if abs(x_new - x) < tol:
        break
    x = x_new
```

2.5. Métodos Multivariantes de Optimización Sin Restricciones

Para minimizar funciones multivariantes $f: \mathbb{R}^n \rightarrow \mathbb{R}$ sin restricciones, construimos una sucesión iterativa a partir de una dirección de búsqueda d_k y un tamaño de paso $\alpha_k > 0$: $x_{k+1} = x_k + \alpha_k d_k$. Para garantizar que el algoritmo avance reduciendo la función en cada paso, la dirección d_k debe ser de descenso, lo que exige matemáticamente que forme un ángulo obtuso con el gradiente de la función en el punto actual: $\nabla f(x_k)^T d_k < 0$.

2.5.1. Método de las Coordenadas Cíclicas

Es uno de los algoritmos de optimización multivariable más sencillos. Evita por completo el cálculo de derivadas y gradientes. En su lugar, avanza secuencialmente minimizando la función respecto a una única variable de decisión a la vez, manteniendo constantes las restantes $n - 1$ variables. Las direcciones de búsqueda coinciden cíclicamente con los vectores de la base canónica de $\mathbb{R}^n$: $d_k = e_{(k \bmod n)+1}$, donde e_i es el i -ésimo vector unitario. En cada paso, se resuelve un subproblema de optimización lineal 1D.

i Ejemplo: Coordenadas Cíclicas

- **Variables Desacopladas:** Minimizamos la función de distancia $f(x, y) = (x - 2)^2 + (y - 1)^2$ (ver Figura 2.11) partiendo de $x_0 = (0, 0)$.

- *Paso 1:* Minimizamos respecto a x : $\min_x f(x, 0) = (x - 2)^2 + 1 \Rightarrow x_1 = 2 \Rightarrow x_1 = (2, 0)$.
- *Paso 2:* Minimizamos respecto a y : $\min_y f(2, y) = (y - 1)^2 \Rightarrow y_1 = 1 \Rightarrow x_2 = (2, 1)$. Al no existir términos de acoplamiento cruzado entre las variables (la matriz hessiana es diagonal), el algoritmo localiza el mínimo exacto en una única iteración de $n = 2$ pasos ortogonales.

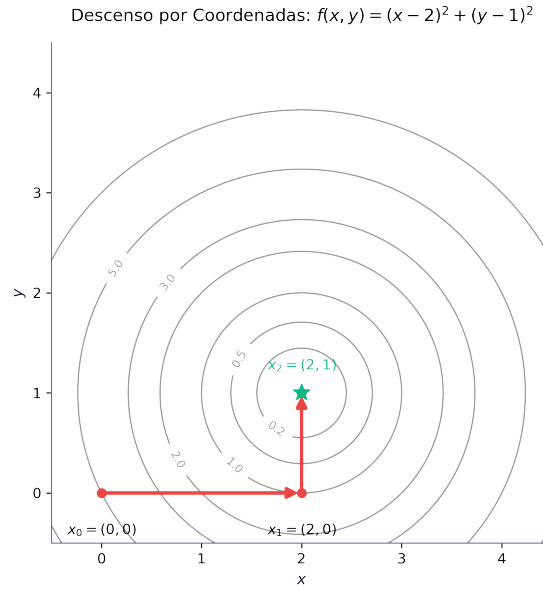


Figura 2.11: Descenso por coordenadas cíclicas sobre curvas de nivel circulares independientes

- **Variables Acopladas:** Minimizamos $f(x, y) = 8x^2 + 3xy + 7y^2 - 25x + 31y - 29$ (ver Figura 2.12) partiendo de $x_0 = (0, 0)$.
 - *Paso 1:* Minimizamos respecto a x : $\min_x f(x, 0) = 8x^2 - 25x - 29 \Rightarrow 16x - 25 = 0 \Rightarrow x_1 = (1.5625, 0)$.
 - *Paso 2:* Minimizamos respecto a y manteniendo $x = 1.5625$: $\min_y f(1.5625, y) = 7y^2 + 35.6875y + ct \Rightarrow 14y + 35.6875 = 0 \Rightarrow y = -571/224 \approx -2.5491 \Rightarrow x_2 = (1.5625, -2.5491)$.
 - *Paso 3:* Minimizamos de nuevo respecto a x con $y = -2.5491$: $\min_x f(x, -2.5491) = 8x^2 - 32.6473x + ct \Rightarrow 16x - 32.6473 = 0 \Rightarrow x_3 = (2.0405, -2.5491)$. Debido al acoplamiento bilineal inducido por el término $3xy$, la trayectoria dibuja un patrón de “escalera”, requiriendo múltiples ciclos para aproximarse al mínimo global situado en $x^* = (2.060, -2.656)$.

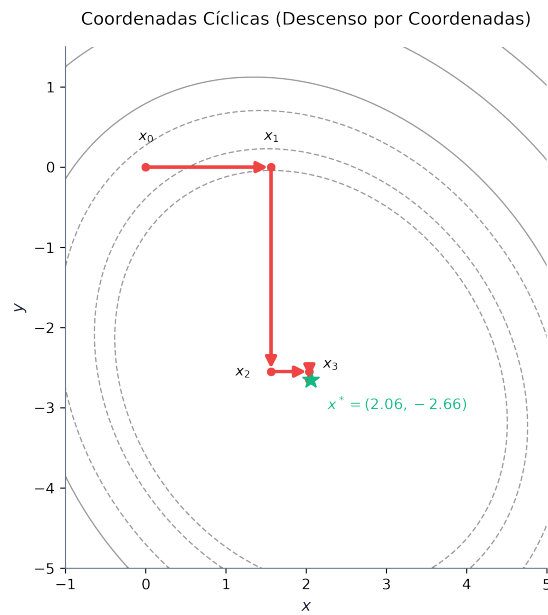


Figura 2.12: Trayectoria en escalera del descenso por coordenadas sobre una función cuadrática acoplada

Código Python: Descenso por Coordenadas

```

import numpy as np

# Función del ejemplo
f = lambda x: 8*x[0]**2 + 3*x[0]*x[1] + 7*x[1]**2 - 25*x[0] + 31*x[1] - 29

# Punto inicial
x = np.array([0.0, 0.0])
print(f"Punto inicial x_0: {x}")

# Paso 1: optimizar x (componente 0) manteniendo y = 0
x[0] = 25 / 16
print(f"Paso 1 (optimizar x): {x} -> f(x) = {f(x):.4f}")

# Paso 2: optimizar y (componente 1) manteniendo x = 1.5625
x[1] = - (3*x[0] + 31) / 14
print(f"Paso 2 (optimizar y): {x} -> f(x) = {f(x):.4f}")

# Paso 3: optimizar x (componente 0) manteniendo y = -2.5491
x[0] = - (3*x[1] - 25) / 16
print(f"Paso 3 (optimizar x): {x} -> f(x) = {f(x):.4f}")

```

2.5.2. Método del Máximo Descenso (Steepest Descent)

La dirección de búsqueda en el método de máximo descenso se define como la dirección en la que la función decrece más rápidamente a nivel local, dada por el opuesto del vector gradiente: $d_k = -\nabla f(x_k)$. El tamaño de paso óptimo α_k se calcula mediante búsqueda de línea exacta: $\alpha_k = \arg \min_{\alpha > 0} f(x_k - \alpha \nabla f(x_k))$.

Una propiedad matemática de la búsqueda lineal exacta en el método de máximo descenso es que las direcciones de búsqueda en iteraciones consecutivas son estrictamente ortogonales entre sí: $d_k^T d_{k+1} = 0$.

Para demostrarlo, definimos la función unidimensional $\phi(\alpha) = f(x_k + \alpha d_k)$ que se minimiza para hallar α_k . La condición de óptimo exige que su derivada con respecto al tamaño de paso sea nula en α_k : $\phi'(\alpha_k) = \nabla f(x_k + \alpha_k d_k)^T d_k = 0$. Dado que el nuevo punto es $x_{k+1} = x_k + \alpha_k d_k$, se tiene que $\nabla f(x_{k+1})^T d_k = 0$. Puesto que las direcciones consecutivas son proporcionales a los gradientes ($d_{k+1} = -\nabla f(x_{k+1})$ y $d_k = -\nabla f(x_k)$), se concluye de forma directa la ortogonalidad: $d_{k+1}^T d_k = 0$.

Esta ortogonalidad geométrica provoca que, si la función presenta curvas de nivel elípticas muy alargadas (problemas mal acondicionados), el algoritmo realice continuos giros de 90° ,

generando un patrón en **zigzag** que ralentiza la convergencia, tal y como se observa en la Figura 2.13.

i Ejemplo: Máximo Descenso

Minimizamos $f(x, y) = 8x^2 + 3xy + 7y^2 - 25x + 31y - 29$ partiendo de $x_0 = (0, 0)$.
 * *Gradiente en x_0* : $\nabla f(x_0) = (16x - 25 + 3y, 3x + 14y + 31)_{x=(0,0)}^T = (-25, 31)^T$.
 La dirección de descenso es $d_0 = -\nabla f(x_0) = (25, -31)^T$. * *Búsqueda de línea*: Minimizamos $\phi(\alpha) = f(25\alpha, -31\alpha) = 9402\alpha^2 - 1586\alpha - 29$. Derivando e igualando a cero: $\phi'(\alpha) = 18804\alpha - 1586 = 0 \implies \alpha_0 \approx 0.0843$. * *Actualización*: $x_1 = x_0 + \alpha_0 d_0 = (0, 0)^T + 0.0843(25, -31)^T = (2.109, -2.615)^T$. El nuevo gradiente en x_1 es $\nabla f(x_1) \approx (0.8925, 0.7225)^T$. Su producto escalar con la dirección anterior es $d_0^T \nabla f(x_1) = 25(0.8925) - 31(0.7225) \approx 0$, lo que ilustra la ortogonalidad.

Código Python: Máximo Descenso

```
import numpy as np

# Función y gradiente del ejemplo
f = lambda x: 8*x[0]**2 + 3*x[0]*x[1] + 7*x[1]**2 - 25*x[0] + 31*x[1] - 29
grad_f = lambda x: np.array([16*x[0] + 3*x[1] - 25, 3*x[0] + 14*x[1] + 31])

# Punto inicial
x = np.array([0.0, 0.0])
g = grad_f(x)
d = -g
print(f"Punto inicial x_0: {x}")
print(f"Gradiente g_0: {g} -> Dirección de descenso d_0: {d}")

# Búsqueda lineal exacta: minimizar f(x + alpha * d)
# alpha_0 = 1586 / 18804 approx 0.0843
alpha = 1586 / 18804
x_new = x + alpha * d
g_new = grad_f(x_new)
print(f"Paso 1: x_1 = {x_new} -> f(x_1) = {f(x_new):.4f}")
print(f"Nuevo gradiente g_1: {g_new}")
print(f"Producto escalar d_0^T g_1: {np.dot(d, g_new):.6f} (Ortogonalidad)")
```

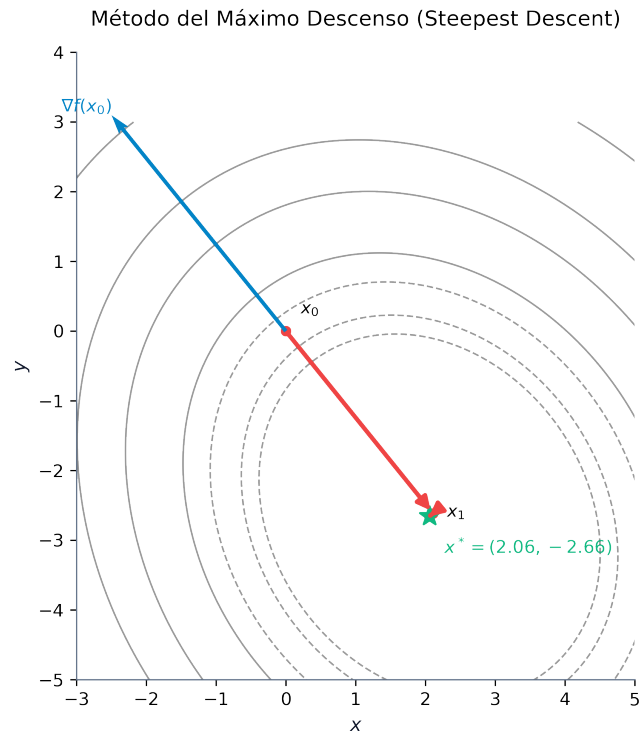


Figura 2.13: Trayectoria ortogonal de máximo descenso: giros de 90 grados sobre las curvas de nivel elípticas de la función cuadrática

2.5.3. Métodos de Newton Multivariable y BFGS

El **Método de Newton Multivariable** incorpora información de segundo orden utilizando la aproximación de Taylor cuadrática en el punto x_k , calculando el nuevo punto mediante la expresión: $x_{k+1} = x_k - [\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$. Si la función objetivo es cuadrática, el método de Newton localiza el mínimo global del sistema en una sola iteración desde cualquier punto inicial. Si la función es no lineal general, presenta una convergencia cuadrática local de orden 2.

A pesar de su velocidad de convergencia, el método de Newton tiene importantes limitaciones prácticas. Presenta un **coste computacional elevado** debido a que en cada iteración se debe calcular la matriz hessiana y resolver un sistema de ecuaciones lineales, lo que supone un coste de $O(n^3)$ operaciones. Asimismo, exige que la matriz hessiana sea **estrictamente definida positiva** para garantizar que la dirección obtenida sea de descenso; si el algoritmo entra en una región cóncava o se topa con un punto de silla, puede diverger. Por último, su convergencia es de **carácter local**, estando garantizada solo si el punto inicial x_0 se encuentra cerca del óptimo.

Los **Métodos Cuasi-Newton (BFGS)** eliminan estos inconvenientes estimando iterativamente una aproximación simétrica y definida positiva de la inversa de la hessiana, denotada por $H_k \approx [\nabla^2 f(x_k)]^{-1}$. Esta matriz se actualiza utilizando únicamente la información de los gradientes de los dos puntos consecutivos, forzando a que se cumpla la **ecuación de la secante**: $H_{k+1}y_k = s_k$, donde $s_k = x_{k+1} - x_k$ y $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$. El algoritmo BFGS actualiza la matriz aproximada mediante la expresión $H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$, donde $\rho_k = 1/(y_k^T s_k)$. Garantiza que la aproximación se mantenga definida positiva en cada paso si la búsqueda de línea satisface las condiciones de Wolfe, proporcionando una convergencia superlineal con un coste por iteración de tan solo $O(n^2)$ operaciones (Nocedal y Wright 2006; Luenberger y Ye 2015).

3 Análisis Convexo

El **análisis convexo** constituye la columna vertebral de la optimización matemática moderna. Históricamente, la optimización estuvo dominada por el paradigma de la linealidad frente a la no linealidad. Sin embargo, a finales del siglo XX se produjo un cambio de paradigma fundamental, resumido por el matemático R. Tyrrell Rockafellar: *“la divisoria fundamental en optimización no es entre linealidad y no linealidad, sino entre convexidad y no convexidad”* (Rockafellar 1970). Bajo la hipótesis de convexidad, cualquier óptimo local se convierte de manera natural en un óptimo global (Boyd y Vandenberghe 2004). Esta propiedad crucial no solo simplifica la búsqueda teórica de soluciones, sino que garantiza la convergencia fiable y veloz de los algoritmos numéricos, evitando que queden atrapados en mínimos locales subóptimos.

En este capítulo, estudiaremos los conjuntos convexos y las operaciones que preservan esta propiedad, las caracterizaciones de primer y segundo orden para funciones convexas diferenciables y no diferenciables (subgradientes), las propiedades fundamentales de sus extremos y las generalizaciones teóricas de la convexidad (cuasi-convexidad y pseudo-convexidad).

! Objetivos de aprendizaje

Al finalizar este capítulo, serás capaz de:

1. **Identificar y demostrar** si un conjunto dado en $\mathbb{R}^n$ es convexo aplicando la definición o las propiedades algebraicas de conservación.
2. **Diferenciar** entre combinaciones lineales, afines, cónicas y convexas, analizando la estructura geométrica de sus envolturas.
3. **Comprender e interpretar** los Teoremas de Separación de Hiperplanos y del Hiperplano de Soporte.
4. **Caracterizar la convexidad de funciones** mediante las propiedades del epígrafe, los conjuntos de nivel y las caracterizaciones de primer y segundo orden (gradiente y hessiana).
5. **Calcular el subdiferencial** de funciones convexas no diferenciables en puntos singulares de esquina o codo, y aplicar el cálculo de subgradientes.
6. **Aplicar el Teorema Fundamental de la Optimización Convexa** para deconstruir y verificar la globalidad y unicidad de los mínimos.
7. **Clasificar y relacionar** las funciones cuasi-convexas, pseudo-convexas y sus variantes, comprendiendo sus implicaciones en la optimización restringida.

3.1. Conjuntos Convexos

3.1.1. Definición y Conceptos Básicos

Un conjunto es convexo si no presenta “entrantes” o cavidades, de modo que la conexión recta entre cualquier pareja de sus puntos está totalmente contenida en él.

- **Conjunto Convexo:** Un conjunto $\mathcal{S} \subseteq \mathbb{R}^n$ es **convexo** si:

$$\forall x_1, x_2 \in \mathcal{S}, \quad \forall \lambda \in (0, 1) \implies \lambda x_1 + (1 - \lambda)x_2 \in \mathcal{S}$$

- **Combinación Lineal Convexa:** Un punto x es una combinación lineal convexa de un conjunto finito de puntos $\{x_1, \dots, x_k\} \subset \mathbb{R}^n$ si puede expresarse de la forma:

$$x = \sum_{i=1}^k \lambda_i x_i \quad \text{con} \quad \lambda_i \geq 0 \quad \text{y} \quad \sum_{i=1}^k \lambda_i = 1$$

- **Combinación Afín:** Se obtiene eliminando la condición de no negatividad de los coeficientes ($\lambda_i \in \mathbb{R}$ con $\sum_{i=1}^k \lambda_i = 1$). Genera subespacios afines (rectas, planos, etc.).
- **Combinación Conica:** Se define exigiendo únicamente la no negatividad de los coeficientes ($\lambda_i \geq 0$ sin exigir que sumen 1). Genera conos convexos.
- **Combinación Lineal:** Se define eliminando ambas condiciones ($\lambda_i \in \mathbb{R}$ sin restricciones). Genera subespacios vectoriales.

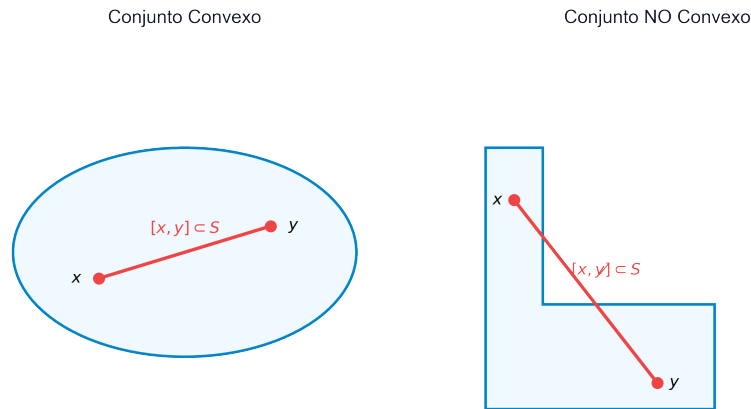


Figura 3.1: Comparación entre un conjunto convexo (izquierda) y un conjunto no convexo (derecha)

3.1.2. Ejemplos de Conjuntos Convexos Básicos

- **Hiperplano:** Dado un vector normal no nulo $p \in \mathbb{R}^n$ y un escalar $\alpha \in \mathbb{R}$, el hiperplano es:

$$\mathcal{H} = \{x \in \mathbb{R}^n \mid p^T x = \alpha\}$$

Representa una superficie plana de dimensión $n - 1$.

- **Semiespacio Cerrado:** La región del espacio situada en uno de los lados de un hiperplano:

$$\mathcal{H}^- = \{x \in \mathbb{R}^n \mid p^T x \leq \alpha\}$$

- **Poliedro o Politopo Convexo:** La intersección de un número finito de semiespacios cerrados:

$$\mathcal{P} = \{x \in \mathbb{R}^n \mid Ax \leq b\}$$

Constituye la región factible típica de la programación lineal. Sus esquinas o vértices se denominan **puntos extremos**.

- **Bola Métrica:** Una bola cerrada en $\mathbb{R}^n$ con centro en x_0 y radio $r \geq 0$ definida bajo cualquier norma $\|\cdot\|$:

$$\mathcal{B}_r(x_0) = \{x \in \mathbb{R}^n \mid \|x - x_0\| \leq r\}$$

3.1.3. Operaciones que Conservan la Convexidad

i Lema: Conservación de la Convexidad

Sean $\mathcal{S}_1$ y $\mathcal{S}_2$ dos conjuntos convexos en $\mathbb{R}^n$. Las siguientes estructuras también son conjuntos convexos:

1. **Intersección:**

$$\mathcal{S}_1 \cap \mathcal{S}_2$$

(Esta propiedad se extiende a la intersección de una familia infinita o arbitraria de conjuntos convexos).

2. **Suma Directa:**

$$\mathcal{S}_1 \oplus \mathcal{S}_2 = \{x_1 + x_2 \mid x_1 \in \mathcal{S}_1, x_2 \in \mathcal{S}_2\}$$

3. **Diferencia Directa:**

$$\mathcal{S}_1 \ominus \mathcal{S}_2 = \{x_1 - x_2 \mid x_1 \in \mathcal{S}_1, x_2 \in \mathcal{S}_2\}$$

3.1.4. Envoltura Convexa y Símplices

- **Envoltura Convexa** ($\mathcal{H}(\mathcal{S})$): El conjunto de todas las combinaciones lineales convexas posibles de puntos pertenecientes al conjunto $\mathcal{S}$:

$$\mathcal{H}(\mathcal{S}) = \left\{ x = \sum_{j=1}^k \lambda_j x_j \mid x_j \in \mathcal{S}, \lambda_j \geq 0, \sum_{j=1}^k \lambda_j = 1 \right\}$$

! Teorema: Minimalidad de la Envoltura Convexa

La envoltura convexa $\mathcal{H}(\mathcal{S})$ es el menor conjunto convexo que contiene al conjunto $\mathcal{S}$.

Demostración: Sea $\mathcal{C}$ cualquier conjunto convexo que contiene a $\mathcal{S}$ (es decir, $\mathcal{S} \subseteq \mathcal{C}$). Supongamos por reducción al absurdo que existe un punto $x \in \mathcal{H}(\mathcal{S})$ tal que $x \notin \mathcal{C}$. Como $x \in \mathcal{H}(\mathcal{S})$, se puede escribir como una combinación lineal convexa $x = \sum_{j=1}^k \lambda_j x_j$ donde cada $x_j \in \mathcal{S}$. Dado que $\mathcal{S} \subseteq \mathcal{C}$, se deduce que todos los puntos generadores x_j pertenecen a $\mathcal{C}$. Como $\mathcal{C}$ es convexo por hipótesis, cualquier combinación lineal convexa de sus elementos debe pertenecer a él, lo que obliga a que $x \in \mathcal{C}$, contradiciendo la suposición inicial. ■

- **Símplex:** Un politopo convexo generado por la envoltura convexa de $n + 1$ puntos afínmente independientes en $\mathbb{R}^n$. Un simplex en $\mathbb{R}^1$ es un segmento, en $\mathbb{R}^2$ es un triángulo y en $\mathbb{R}^3$ es un tetraedro.

3.2. Teoremas de Separación e Hiperplano de Soporte

La geometría convexa descansa sobre dos teoremas fundamentales que fundamentan la teoría de la dualidad y los multiplicadores de Lagrange.

3.2.1. Teoremas de Separación de Hiperplanos

Estos teoremas establecen las condiciones bajo las cuales es posible “trazar una frontera plana” (un hiperplano) que separe dos conjuntos convexas disjuntos.

! Teorema de Separación Débil

Sean $\mathcal{C}$ y $\mathcal{D}$ dos conjuntos convexas disjuntos no vacíos en $\mathbb{R}^n$. Entonces existe un hiperplano que los separa débilmente. Es decir, existen un vector no nulo $p \in \mathbb{R}^n$ y un escalar $\alpha \in \mathbb{R}$ tales que:

$$p^T x \leq \alpha, \quad \forall x \in \mathcal{C} \quad \text{y} \quad p^T y \geq \alpha, \quad \forall y \in \mathcal{D}$$

! Teorema de Separación Estricta

Sean $\mathcal{C}$ y $\mathcal{D}$ dos conjuntos convexos disjuntos no vacíos en $\mathbb{R}^n$. Si además $\mathcal{C}$ es cerrado y $\mathcal{D}$ es cerrado y acotado (compacto), entonces existe un hiperplano que los separa estrictamente. Es decir, existen un vector no nulo $p \in \mathbb{R}^n$ y un escalar $\alpha \in \mathbb{R}$ tales que:

$$p^T x < \alpha, \quad \forall x \in \mathcal{C} \quad \text{y} \quad p^T y > \alpha, \quad \forall y \in \mathcal{D}$$

3.2.2. Teorema del Hiperplano de Soporte

Un hiperplano de soporte para un conjunto $\mathcal{S}$ en un punto de su frontera $x_0 \in \partial\mathcal{S}$ es un hiperplano que pasa por x_0 y deja a todo el conjunto $\mathcal{S}$ contenido en uno de sus semiespacios cerrados.

! Teorema del Hiperplano de Soporte

Sea $\mathcal{S}$ un conjunto convexo no vacío en $\mathbb{R}^n$, y sea x_0 un punto perteneciente a la frontera de $\mathcal{S}$ ($x_0 \in \partial\mathcal{S}$). Entonces existe un hiperplano de soporte para $\mathcal{S}$ en x_0 . Es decir, existe un vector no nulo $p \in \mathbb{R}^n$ tal que:

$$p^T x \leq p^T x_0, \quad \forall x \in \mathcal{S}$$

Este teorema garantiza que para cualquier conjunto convexo siempre podemos “apoyar” un plano sobre cualquier punto de su superficie exterior sin llegar a cruzar el interior del conjunto.

3.3. Funciones Convexas

3.3.1. Definición y Significado Geométrico

- **Función Convexa:** Sea $\mathcal{S} \subseteq \mathbb{R}^n$ un conjunto convexo no vacío. Una función $f : \mathcal{S} \rightarrow \mathbb{R}$ es **convexa** si para todo par de puntos $x_1, x_2 \in \mathcal{S}$ y cualquier parámetro $\lambda \in (0, 1)$ se cumple que:

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$$

- **Función Estrictamente Convexa:** Si la desigualdad anterior es estricta ($<$) para todo $x_1 \neq x_2$.
- **Función Cóncava:** Una función f es cóncava si su opuesta $-f$ es convexa.

Geométricamente, una función es convexa si el segmento de cuerda que une los puntos de la gráfica $(x_1, f(x_1))$ y $(x_2, f(x_2))$ queda por encima o sobre la gráfica de la función en dicho intervalo.

3.3.2. Operaciones que Preservan la Convexidad de Funciones

- **Suma ponderada positiva:** Si $f_1, \dots, f_k$ son convexas y $\alpha_1, \dots, \alpha_k \geq 0$, entonces la combinación $\sum_j \alpha_j f_j$ es convexa.
- **Supremo puntual (Máximo):** Si $f_1, \dots, f_k$ son convexas, la función envoltura superior $f(x) = \max_j \{f_j(x)\}$ es convexa.
- **Composición afín:** Si $g : \mathbb{R}^m \rightarrow \mathbb{R}$ es convexa y $h(x) = Ax + b$ es una función afín, entonces la composición $f(x) = g(Ax + b)$ es convexa.

! Teorema: Convexidad de los Conjuntos de Nivel

Sea $f : \mathcal{S} \rightarrow \mathbb{R}$ una función convexa sobre un dominio convexo $\mathcal{S}$. Para cualquier constante $\alpha \in \mathbb{R}$, el conjunto de nivel inferior:

$$\mathcal{S}_\alpha = \{x \in \mathcal{S} \mid f(x) \leq \alpha\}$$

es un conjunto convexo.

Demostración: Sean $x_1, x_2 \in \mathcal{S}_\alpha$. Por definición del conjunto de nivel, se cumple que $f(x_1) \leq \alpha$ y $f(x_2) \leq \alpha$. Sea $\lambda \in (0, 1)$. Como el dominio $\mathcal{S}$ es convexo, el punto intermedio $\lambda x_1 + (1 - \lambda)x_2$ pertenece a $\mathcal{S}$. Aplicando la definición de convexidad de f :

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2) \leq \lambda \alpha + (1 - \lambda)\alpha = \alpha$$

Por tanto, $\lambda x_1 + (1 - \lambda)x_2 \in \mathcal{S}_\alpha$, lo que demuestra la convexidad del conjunto de nivel. ■

Nota: El recíproco no es cierto en general (una función con conjuntos de nivel convexos no tiene por qué ser convexa; esta propiedad define a las funciones cuasi-convexas).

3.3.3. Epígrafe y Subgradiientes

- **Epígrafe (epi f):** El conjunto de puntos en $\mathbb{R}^{n+1}$ situados sobre o por encima de la gráfica de la función:

$$\text{epi } f = \{(x, y) \in \mathcal{S} \times \mathbb{R} \mid y \geq f(x)\}$$

! Teorema: Relación entre Epígrafe y Convexidad

Una función $f : \mathcal{S} \rightarrow \mathbb{R}$ es convexa si y solo si su epígrafe $\text{epi } f$ es un conjunto convexo en $\mathbb{R}^{n+1}$.

Demostración: ($\Rightarrow$) Supongamos que f es convexa. Sean $(x_1, y_1), (x_2, y_2) \in \text{epi } f$ y $\lambda \in (0, 1)$. Por definición de epígrafe, $y_1 \geq f(x_1)$ y $y_2 \geq f(x_2)$. Evaluando la combinación lineal:

$$\lambda y_1 + (1 - \lambda)y_2 \geq \lambda f(x_1) + (1 - \lambda)f(x_2) \geq f(\lambda x_1 + (1 - \lambda)x_2)$$

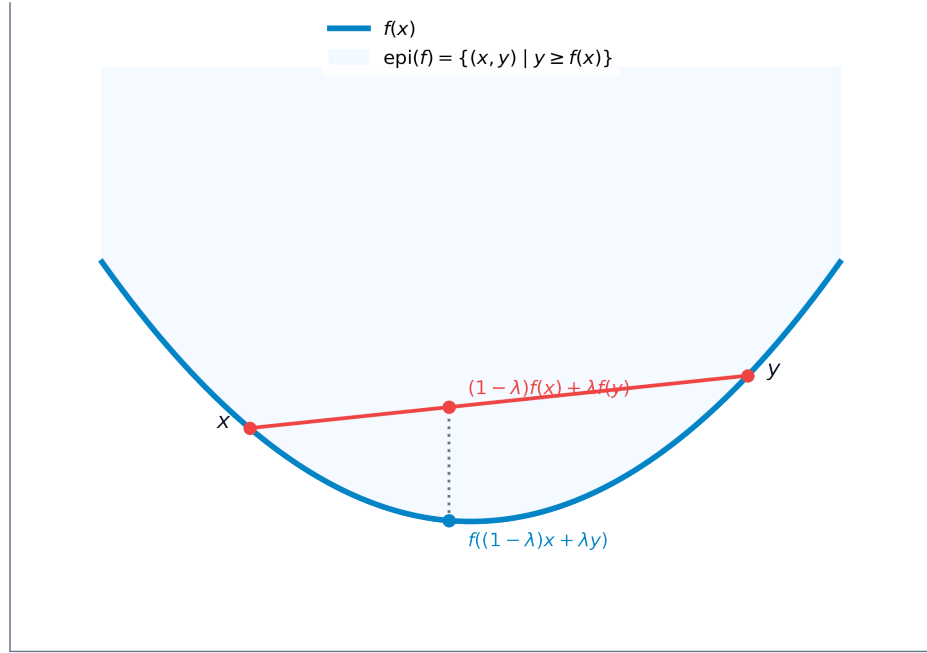


Figura 3.2: Representación gráfica del epígrafe de una función convexa y su secante

Como $\mathcal{S}$ es convexo, el punto combinado pertenece a $\mathcal{S}$, y el par combinado pertenece a $\text{epi } f$, demostrando su convexidad.

($\Leftarrow$) Supongamos que $\text{epi } f$ es un conjunto convexo. Sean $x_1, x_2 \in \mathcal{S}$. Los puntos $(x_1, f(x_1))$ y $(x_2, f(x_2))$ pertenecen a $\text{epi } f$. Por convexidad del epígrafe, para todo $\lambda \in (0, 1)$:

$$\lambda(x_1, f(x_1)) + (1 - \lambda)(x_2, f(x_2)) = (\lambda x_1 + (1 - \lambda)x_2, \lambda f(x_1) + (1 - \lambda)f(x_2)) \in \text{epi } f$$

Lo que implica directamente que $\lambda f(x_1) + (1 - \lambda)f(x_2) \geq f(\lambda x_1 + (1 - \lambda)x_2)$, es decir, f es convexa. ■

- **Subgradiente y Subdiferencial:** Para funciones no diferenciables (que presentan picos o codos), el gradiente no existe en ciertos puntos. Generalizamos este concepto mediante el **subgradiente**. Un vector $\xi \in \mathbb{R}^n$ es un subgradiente de la función convexa f en el punto $x_0 \in \mathcal{S}$ si:

$$f(x) \geq f(x_0) + \xi^T(x - x_0), \quad \forall x \in \mathcal{S}$$

Geométricamente, define un hiperplano que apoya a la función desde abajo en x_0 . El conjunto de todos los subgradientes en x_0 se denomina **subdiferencial** y se denota por $\partial f(x_0)$.

El subdiferencial $\partial f(x_0)$ es siempre un conjunto convexo y cerrado, cuyo análisis deta-

llado e implicaciones en optimización no diferenciable se estudian extensamente en el análisis convexo clásico (Rockafellar 1970; Boyd y Vandenberghe 2004). Si la función es diferenciable en x_0 , el subdiferencial contiene un único elemento que es el gradiente: $\partial f(x_0) = \{\nabla f(x_0)\}$.

i Propiedades y Cálculo del Subdiferencial

El cálculo con subgradientes sigue reglas análogas al cálculo diferencial estándar:

1. **Regla de la Suma:** Si f_1, f_2 son funciones convexas, el subdiferencial de su suma cumple:

$$\partial(f_1(x) + f_2(x)) = \partial f_1(x) \oplus \partial f_2(x)$$

2. **Multiplicación Escalar:** Para cualquier constante $\alpha \geq 0$:

$$\partial(\alpha f(x)) = \alpha \partial f(x)$$

3. **Máximo de Funciones:** Si $f(x) = \max\{f_1(x), f_2(x)\}$ con f_1, f_2 convexas:

$$\partial f(x) = \mathcal{H}(\cup_{i \in \mathcal{J}(x)} \partial f_i(x))$$

donde $\mathcal{J}(x)$ es el conjunto de índices de las funciones que alcanzan el máximo en el punto x .

💡 Caso de Estudio: Subdiferencial en Puntos Singulares (No Diferenciabilidad)

Consideremos la función convexa definida por tramos:

$$f(x) = \max\{|x| - 4, (x - 2)^2 - 4\}$$

Analicemos su subdiferencial $\partial f(x_0)$ en diferentes regiones de la recta real:

- **Región diferenciable lineal:** Para $x_0 \in (1, 4)$, la función coincide con la recta $x - 4$. Su derivada es constante e igual a 1. Por tanto, el subdiferencial es el conjunto unitario: $\partial f(x_0) = \{1\}$.
- **Región diferenciable cuadrática:** Para $x_0 < 1$ o $x_0 > 4$, la función coincide con la parábola $(x - 2)^2 - 4$. La derivada es $2(x_0 - 2)$. Así, $\partial f(x_0) = \{2(x_0 - 2)\}$.
- **Punto singular** $x_0 = 1$: La función presenta un codo donde intersecan la parábola y la recta. La derivada lateral izquierda (cuadrática) es $2(1 - 2) = -2$, y la derivada lateral derecha (lineal) es 1. El subdiferencial está formado por todas las pendientes intermedias posibles (combinación convexa de las derivadas laterales):

$$\partial f(1) = [-2, 1]$$

- **Punto singular** $x_0 = 4$: La derivada lateral izquierda es 1 y la derecha es $2(4-2) = 4$. El subdiferencial es el intervalo:

$$\partial f(4) = [1, 4]$$

3.4. Caracterización de Funciones Convexas Diferenciables

3.4.1. Caracterización de Primer Orden

Cuando una función es diferenciable, podemos caracterizar su convexidad analizando su plano tangente.

! Teorema: Caracterización de Primer Orden para Funciones Diferenciables

Sea $f : \mathcal{S} \rightarrow \mathbb{R}$ una función diferenciable en un conjunto convexo y abierto $\mathcal{S}$.

1. f es convexa si y solo si:

$$f(x) \geq f(x_0) + \nabla f(x_0)^T(x - x_0), \quad \forall x, x_0 \in \mathcal{S}$$

2. f es estrictamente convexa si y solo si:

$$f(x) > f(x_0) + \nabla f(x_0)^T(x - x_0), \quad \forall x, x_0 \in \mathcal{S} \text{ con } x \neq x_0$$

Significado geométrico: La aproximación lineal de primer orden (plano tangente en cualquier punto x_0) es un estimador inferior global de la función. Es decir, la gráfica de la función se apoya en su totalidad sobre el plano tangente.

! Teorema: Monotonía del Gradiente

Una función diferenciable f es convexa en un conjunto convexo abierto $\mathcal{S}$ si y solo si su gradiente es un operador monótono:

$$(\nabla f(x_2) - \nabla f(x_1))^T(x_2 - x_1) \geq 0, \quad \forall x_1, x_2 \in \mathcal{S}$$

Demostración: ($\Rightarrow$) Si f es convexa, por la caracterización de primer orden se cumplen simultáneamente:

$$f(x_1) \geq f(x_2) + \nabla f(x_2)^T(x_1 - x_2)$$

$$f(x_2) \geq f(x_1) + \nabla f(x_1)^T(x_2 - x_1)$$

Sumando ambas inecuaciones miembro a miembro:

$$f(x_1) + f(x_2) \geq f(x_2) + f(x_1) + \nabla f(x_2)^T(x_1 - x_2) + \nabla f(x_1)^T(x_2 - x_1)$$

Cancelando términos comunes y reordenando:

$$0 \geq (\nabla f(x_1) - \nabla f(x_2))^T(x_2 - x_1) \implies (\nabla f(x_2) - \nabla f(x_1))^T(x_2 - x_1) \geq 0$$

($\Leftarrow$) Se demuestra de manera análoga aplicando el teorema del valor medio. ■

3.4.2. Caracterización de Segundo Orden (Hessiana)

La convexidad puede verificarse localmente analizando la concavidad o curvatura de segundo orden dada por la matriz hessiana en todo el dominio.

! Teorema: Caracterización de Segundo Orden

Sea $f : \mathcal{S} \rightarrow \mathbb{R}$ una función dos veces diferenciable en un conjunto convexo abierto $\mathcal{S}$.

1. f es convexa si y solo si su matriz hessiana es semidefinida positiva en todo punto de $\mathcal{S}$:

$$\nabla^2 f(x) \succeq 0, \quad \forall x \in \mathcal{S}$$

2. Si la matriz hessiana es definida positiva en todo el dominio ($\nabla^2 f(x) \succ 0, \forall x \in \mathcal{S}$), entonces f es estrictamente convexa.

💡 Ejemplos de Verificación de Convexidad con Matrices Hessianas

Analicemos la convexidad de tres funciones de prueba:

■ Ejemplo 1:

$$f(x_1, x_2, x_3) = x_1x_2 + 2x_1^2 + x_2^2 + 2x_3^2 - 6x_1x_3$$

La matriz hessiana de esta función cuadrática es constante:

$$\nabla^2 f = \begin{pmatrix} 4 & 1 & -6 \\ 1 & 2 & 0 \\ -6 & 0 & 4 \end{pmatrix}$$

Calculamos los menores principales líderes para aplicar el criterio de Sylvester:

- $M_1 = 4 > 0$.
- $M_2 = 4 \cdot 2 - 1^2 = 7 > 0$.
- $M_3 = \det(\nabla^2 f) = 4(8) - 1(4) - 6(12) = -20 < 0$. Al ser el determinante negativo, la matriz es indefinida, por lo que la función no es ni cóncava ni convexa en $\mathbb{R}^3$.

■ Ejemplo 2:

$$f(x_1, x_2) = 10 - 2(x_2 - x_1^2)^2$$

La matriz hessiana es variable y depende del punto:

$$\nabla^2 f(x) = \begin{pmatrix} 8x_2 - 24x_1^2 & 8x_1 \\ 8x_1 & -4 \end{pmatrix}$$

Para que sea cóncava se requiere $\nabla^2 f(x) \preceq 0$, lo que exige que los menores verifiquen:

$$8x_2 - 24x_1^2 \leq 0 \quad \text{y} \quad -4(8x_2 - 24x_1^2) - 64x_1^2 \geq 0 \implies 32(x_1^2 - x_2) \geq 0$$

Esto define una región del plano donde $x_2 \leq x_1^2$.

■ **Ejemplo 3:**

$$f(x, y, z) = x^2 + 3y^2 + z^2 + xy - 4z - 5x + 14$$

Su hessiana es constante:

$$\nabla^2 f = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 6 & 0 \\ 0 & 0 & 2 \end{pmatrix}$$

Menores principales: $M_1 = 2 > 0$, $M_2 = 11 > 0$, $M_3 = 22 > 0$. Al ser definida positiva, la función es estrictamente convexa en todo el espacio.

3.5. Óptimos de una Función Convexa

3.5.1. El Teorema Fundamental de la Optimización Convexa

La convexidad simplifica drásticamente la búsqueda de óptimos al eliminar la distinción entre mínimos locales y globales.

! Teorema Fundamental de la Optimización Convexa

Sea $f : \mathcal{S} \rightarrow \mathbb{R}$ una función convexa sobre un conjunto convexo $\mathcal{S}$.

1. **Cualquier mínimo local de f en $\mathcal{S}$ es también un mínimo global.**
2. **Si la función f es estrictamente convexa, el mínimo global (si existe) es único.**

Demostración:

1. Sea x^* un mínimo local de f en $\mathcal{S}$. Por definición, existe un radio $\epsilon > 0$ tal que $f(x) \geq f(x^*)$ para todo $x \in \mathcal{S}$ con $\|x - x^*\| < \epsilon$. Supongamos por contradicción que x^* no es un mínimo global, lo que implica que existe un punto $y \in \mathcal{S}$ tal que $f(y) < f(x^*)$. Construimos una combinación lineal convexa de ambos puntos: $x(\lambda) = \lambda y + (1 - \lambda)x^*$ para $\lambda \in (0, 1)$. Como el dominio $\mathcal{S}$ es convexo, $x(\lambda) \in \mathcal{S}$.

Elegimos un λ suficientemente pequeño de forma que el punto $x(\lambda)$ entre dentro de la bola de radio ϵ alrededor de x^* . Aplicando la propiedad de convexidad de f :

$$f(x(\lambda)) \leq \lambda f(y) + (1 - \lambda)f(x^*) < \lambda f(x^*) + (1 - \lambda)f(x^*) = f(x^*)$$

Obtenemos que $f(x(\lambda)) < f(x^*)$, lo cual contradice directamente la suposición de que x^* es un mínimo local. Por tanto, no puede existir tal punto y , y x^* es un mínimo global.

2. Supongamos que f es estrictamente convexa y que existen dos mínimos globales distintos, x^* e y^* , con $f(x^*) = f(y^*) = \alpha$. Sea el punto medio $z = \frac{1}{2}x^* + \frac{1}{2}y^* \in \mathcal{S}$. Por la convexidad estricta de f :

$$f(z) < \frac{1}{2}f(x^*) + \frac{1}{2}f(y^*) = \alpha$$

Resulta que $f(z) < \alpha$, lo que contradice que α sea el valor mínimo de la función. Por tanto, el mínimo global debe ser único. ■

3.5.2. Condición Variacional de Optimalidad

- **Caracterización con subgradientes:** Un punto x^* es mínimo global de la función convexa f en $\mathcal{S}$ si y solo si el vector nulo 0 es un subgradiente de f en x^* :

$$0 \in \partial f(x^*)$$

- **Caracterización variacional diferenciable:** Si f es diferenciable, un punto x^* es mínimo global si y solo si:

$$\nabla f(x^*)^T(x - x^*) \geq 0, \quad \forall x \in \mathcal{S}$$

Interpretación geométrica: El gradiente $\nabla f(x^*)$ forma un ángulo agudo u obtuso con cualquier dirección que apunte hacia el interior del conjunto factible desde x^* , lo que impide la existencia de direcciones de descenso admisibles.

3.6. Maximización de Funciones Convexas

La maximización de una función convexa es un problema no convexo y, por tanto, sumamente complejo.

- Las condiciones necesarias de primer orden (gradiente nulo en el interior) suelen identificar mínimos locales, no máximos.

- **Propiedad de los Puntos Extremos:** Si maximizamos una función convexa f sobre un politopo convexo y acotado (compacto) $\mathcal{S}$, la teoría matemática garantiza que el máximo global se sitúa en alguno de los **puntos extremos** (vértices) del politopo. Esto justifica que los algoritmos de maximización convexa se centren exclusivamente en explorar los vértices del espacio de soluciones.

3.7. Generalizaciones de la Convexidad

Para mantener las ventajas de la optimalidad global sobre una familia de problemas más amplia, relajamos las exigencias de la convexidad pura:

3.7.1. Funciones Cuasi-convexas

Una función es cuasi-convexa si todos sus conjuntos de nivel inferior $\mathcal{S}_\alpha$ son convexos. De manera formal:

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \max\{f(x_1), f(x_2)\}, \quad \forall x_1, x_2 \in \mathcal{S}, \quad \forall \lambda \in [0, 1]$$

- **Cuasi-convexidad Estricta:**

$$f(\lambda x_1 + (1 - \lambda)x_2) < \max\{f(x_1), f(x_2)\}, \quad \forall x_1, x_2 \in \mathcal{S} \text{ con } f(x_1) \neq f(x_2)$$

! Teorema: Mínimos en Funciones Estrictamente Cuasi-convexas

Si $f : \mathcal{S} \rightarrow \mathbb{R}$ es una función estrictamente cuasi-convexa definida en un conjunto convexo $\mathcal{S}$, entonces cualquier mínimo local es también un mínimo global.

Demostración: Supongamos por contradicción que x^* es un mínimo local que no es global. Entonces existe un punto $y \in \mathcal{S}$ tal que $f(y) < f(x^*)$. Para todo $\lambda \in (0, 1)$, consideramos el punto intermedio $x(\lambda) = \lambda y + (1 - \lambda)x^*$ para $\lambda \in (0, 1)$. Como x^* es mínimo local, para valores suficientemente pequeños de λ se cumple que $f(x^*) \leq f(x(\lambda))$. Sin embargo, al ser f estrictamente cuasi-convexa y cumplirse que $f(y) < f(x^*)$:

$$f(x(\lambda)) < \max\{f(y), f(x^*)\} = f(x^*)$$

Obtenemos que $f(x(\lambda)) < f(x^*)$, contradiciendo de forma directa que x^* sea un mínimo local. ■

3.7.2. Funciones Pseudo-convexas

Para funciones diferenciables, la pseudo-convexidad exige que si la función presenta una dirección de subida local, la función crece globalmente en esa dirección:

$$\nabla f(x_1)^T (x_2 - x_1) \geq 0 \implies f(x_2) \geq f(x_1)$$

Esta propiedad es de gran valor práctico: **cualquier punto estacionario** ($\nabla f(x^*) = 0$) **en una función pseudo-convexa es de forma garantizada un mínimo global.**

▪ **Pseudo-convexidad Estricta:**

$$\nabla f(x_1)^T(x_2 - x_1) \geq 0 \implies f(x_2) > f(x_1) \quad \text{para } x_1 \neq x_2$$

3.7.3. Relaciones entre Categorías de Convexidad

Las relaciones de implicación lógica entre estas definiciones se estructuran de la siguiente manera:

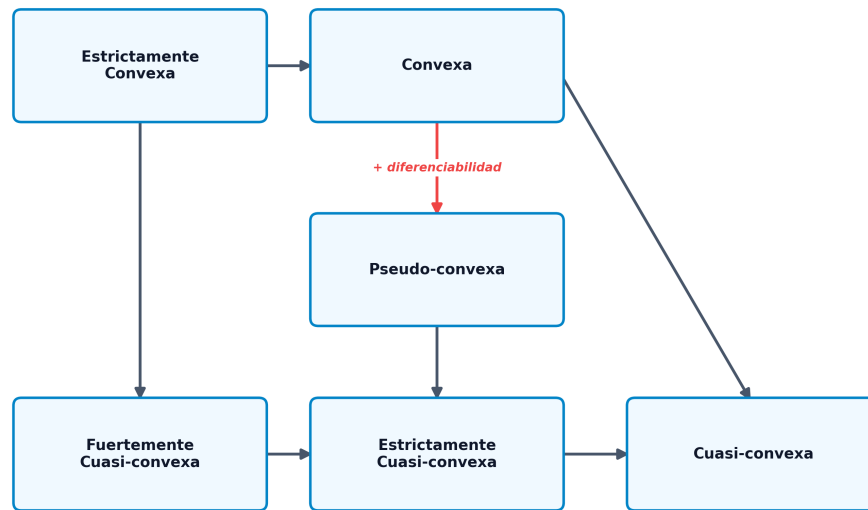


Figura 3.3: Relaciones entre categorías de convexidad

4 Condiciones de Optimalidad y KKT

En las aplicaciones prácticas de la optimización, la inmensa mayoría de los problemas incorporan restricciones sobre los recursos, el presupuesto o las condiciones físicas del sistema. En este capítulo, estudiaremos la caracterización analítica y geométrica de los puntos óptimos. Comenzaremos analizando los problemas sin restricciones para asentar las bases diferenciales, introduciremos el formalismo geométrico de los conos de direcciones factibles y de descenso, y derivaremos las célebres condiciones necesarias de Fritz-John (FJ) y las condiciones necesarias y suficientes de Karush-Kuhn-Tucker (KKT).

4.1. Optimización Sin Restricciones

El estudio de problemas de optimización sin restricciones ($\min f(x)$ para $x \in \mathbb{R}^n$) nos permite comprender cómo se comporta localmente una función basándonos en la información de sus derivadas.

4.1.1. Direcciones de Descenso

Una dirección de descenso es un vector que apunta hacia regiones del espacio donde el valor de la función disminuye localmente.

! Teorema: Condición de Dirección de Descenso

Dada una función $f : \mathbb{R}^n \rightarrow \mathbb{R}$ diferenciable en $\bar{x}$, si existe un vector $d \in \mathbb{R}^n$ tal que:

$$\nabla f(\bar{x})^T d < 0$$

Entonces existe un escalar $\delta > 0$ tal que:

$$f(\bar{x} + \lambda d) < f(\bar{x}), \quad \forall \lambda \in (0, \delta)$$

Demostración: Por la diferenciabilidad de f en el punto $\bar{x}$, podemos escribir el desarrollo de Taylor de primer orden como:

$$f(\bar{x} + \lambda d) = f(\bar{x}) + \lambda \nabla f(\bar{x})^T d + \lambda \|d\| \alpha(\bar{x}, \lambda d)$$

Donde $\alpha(\bar{x}, \lambda d) \rightarrow 0$ cuando $\lambda \rightarrow 0$. Dividiendo ambos miembros por el parámetro $\lambda > 0$, obtenemos:

$$\frac{f(\bar{x} + \lambda d) - f(\bar{x})}{\lambda} = \nabla f(\bar{x})^T d + \|d\| \alpha(\bar{x}, \lambda d)$$

Puesto que $\nabla f(\bar{x})^T d < 0$ por hipótesis, y dado que el término residual $\alpha(\bar{x}, \lambda d)$ tiende a cero conforme λ se desvanece, existe un radio de entorno $\delta > 0$ tal que para todo $\lambda \in (0, \delta)$ el lado derecho de la igualdad permanece estrictamente negativo. Esto implica que $f(\bar{x} + \lambda d) - f(\bar{x}) < 0$, lo que demuestra la tesis. ■

4.1.2. Condiciones Necesarias y Suficientes de Segundo Orden

! Teorema: Condiciones de Optimalidad de Segundo Orden (Sin Restricciones)

Sea $f : \mathbb{R}^n \rightarrow \mathbb{R}$ una función dos veces diferenciable.

1. **Condición Necesaria de Primer Orden (FONC):** Si $\bar{x}$ es un extremo local, entonces el gradiente en dicho punto se anula:

$$\nabla f(\bar{x}) = 0$$

2. **Condición Necesaria de Segundo Orden (SONC):** Si $\bar{x}$ es un mínimo local, entonces la matriz hessiana es semidefinida positiva:

$$\nabla^2 f(\bar{x}) \succeq 0$$

3. **Condición Suficiente de Segundo Orden (SOSC):** Si en un punto $\bar{x}$ se cumple que el gradiente es nulo y la matriz hessiana es definida positiva:

$$\nabla f(\bar{x}) = 0 \quad \text{y} \quad \nabla^2 f(\bar{x}) \succ 0$$

Entonces $\bar{x}$ es un mínimo local estricto de la función.

💡 Ejemplo Ilustrativo: Análisis de Puntos Estacionarios

Consideremos la función univariante $f(x) = (x^2 - 1)^3$. Sus dos primeras derivadas son:

$$f'(x) = 6x(x^2 - 1)^2, \quad f''(x) = 24x^2(x^2 - 1) + 6(x^2 - 1)^2$$

Igualando la primera derivada a cero para localizar los puntos estacionarios, obtenemos los candidatos:

$$x^* = 0, \quad x^* = 1, \quad x^* = -1$$

Analicemos la curvatura de segundo orden en cada candidato:

- **En $x^* = 0$:** $f''(0) = 6(0 - 1) + 6(-1)^2 = 6 > 0$. Al ser la segunda derivada estrictamente positiva, se cumple la condición suficiente de segundo orden, confirmando que $x^* = 0$ es un mínimo local.
- **En $x^* = \pm 1$:** $f''(\pm 1) = 0$. Al anularse la segunda derivada, no podemos concluir mediante las condiciones suficientes habituales. Un análisis de derivadas de orden superior revela que se trata de puntos de inflexión y no de extremos locales.

! Teorema: Suficiencia de Optimalidad bajo Pseudo-convexidad

Sea $f : \mathcal{S} \rightarrow \mathbb{R}$ una función pseudo-convexa definida sobre un conjunto convexo $\mathcal{S}$. Un punto $\bar{x} \in \mathcal{S}$ es un mínimo global de f si y solo si es un punto estacionario:

$$\nabla f(\bar{x}) = 0$$

4.2. Optimización Con Restricciones: Conceptos de Conos

Cuando el movimiento está limitado a una región factible $\mathcal{S} \subseteq \mathbb{R}^n$, la caracterización de los extremos exige analizar la interacción entre las direcciones que reducen la función y las que permanecen dentro del conjunto.

4.2.1. Definiciones Geométricas de Conos

- **Cono de Direcciones Factibles ($\mathcal{F}(\bar{x})$):** El conjunto de direcciones a lo largo de las cuales un paso infinitesimal no abandona la región factible $\mathcal{S}$:

$$\mathcal{F}(\bar{x}) = \{d \in \mathbb{R}^n \mid d \neq 0, \exists \delta_d > 0 \text{ tal que } \bar{x} + \lambda d \in \mathcal{S}, \forall \lambda \in (0, \delta_d)\}$$

- **Cono de Direcciones de Mejora/Descenso ($\mathcal{D}(\bar{x})$):** El conjunto de direcciones a lo largo de las cuales el valor de la función objetivo disminuye localmente:

$$\mathcal{D}(\bar{x}) = \{d \in \mathbb{R}^n \mid d \neq 0, \exists \delta_d > 0 \text{ tal que } f(\bar{x} + \lambda d) < f(\bar{x}), \forall \lambda \in (0, \delta_d)\}$$

Una condición fundamental de optimalidad geométrica establece que si un punto $\bar{x}$ es un mínimo local, no puede existir ninguna dirección que sea simultáneamente factible y de descenso:

$$\bar{x} \text{ es mínimo local} \implies \mathcal{F}(\bar{x}) \cap \mathcal{D}(\bar{x}) = \emptyset$$

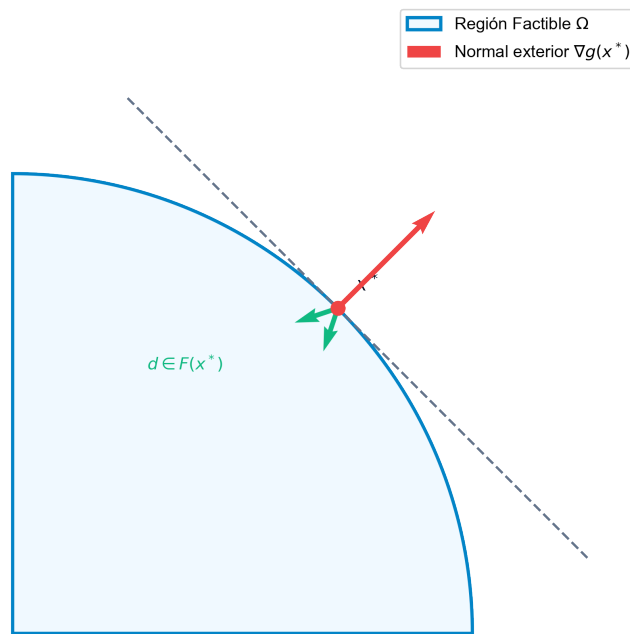


Figura 4.1: Cono de direcciones factibles y cono tangente en un punto frontera

4.2.2. Caracterización Diferenciable mediante Semiespacios

Si la función f es diferenciable en $\bar{x}$, definimos el semiespacio abierto de direcciones que forman un ángulo obtuso con el gradiente (direcciones de descenso analítico) como:

$$\mathcal{D}_0(\bar{x}) = \{d \in \mathbb{R}^n \mid \nabla f(\bar{x})^T d < 0\}$$

! Teorema: Exclusión de Direcciones de Descenso y Factibilidad

Si $\bar{x} \in \mathcal{S}$ es un mínimo local de una función diferenciable f , entonces el cono de direcciones factibles y el semiespacio de descenso analítico son disjuntos:

$$\mathcal{D}_0(\bar{x}) \cap \mathcal{F}(\bar{x}) = \emptyset$$

4.3. Restricciones de Desigualdad y Cono de Restricciones Activas

Consideremos un conjunto factible delimitado por un conjunto de restricciones de desigualdad:

$$\mathcal{S} = \{x \in \mathcal{X} \mid g_i(x) \leq 0, i = 1, \dots, m\}$$

Donde $\mathcal{X}$ es un conjunto abierto.

- **Restricciones Activas ($\mathcal{J}$):** Dado un punto factible $\bar{x} \in \mathcal{S}$, es el conjunto de índices de las restricciones que se cumplen con igualdad estricta:

$$\mathcal{J} = \{i \in \{1, \dots, m\} \mid g_i(\bar{x}) = 0\}$$

Las restricciones inactivas ($g_i(\bar{x}) < 0$) no limitan el movimiento local en un entorno pequeño del punto.

Definimos el cono abierto de direcciones de descenso para las restricciones activas como:

$$\mathcal{G}_0(\bar{x}) = \{d \in \mathbb{R}^n \mid \nabla g_i(\bar{x})^T d < 0, \forall i \in \mathcal{J}\}$$

! Teorema: Intersección Vacía con el Cono de Restricciones Activas

Sea $\bar{x} \in \mathcal{S}$ un mínimo local de f . Si f y las restricciones activas g_i ($i \in \mathcal{J}$) son diferenciables en $\bar{x}$, y las restricciones inactivas son continuas, entonces no existe ninguna dirección en la intersección:

$$\mathcal{D}_0(\bar{x}) \cap \mathcal{G}_0(\bar{x}) = \emptyset$$

Demostración: Supongamos por contradicción que existe un vector $d \in \mathcal{D}_0(\bar{x}) \cap \mathcal{G}_0(\bar{x})$.

Como $d \in \mathcal{G}_0(\bar{x})$, se cumple que $\nabla g_i(\bar{x})^T d < 0$ para toda $i \in \mathcal{I}$. Por el teorema de dirección de descenso, existe un paso $\delta_1 > 0$ tal que $g_i(\bar{x} + \lambda d) < g_i(\bar{x}) = 0$ para todo $\lambda \in (0, \delta_1)$.

Para las restricciones inactivas ($j \notin \mathcal{I}$), dado que $g_j(\bar{x}) < 0$ y por la continuidad de g_j , existe un paso $\delta_2 > 0$ tal que $g_j(\bar{x} + \lambda d) < 0$ para todo $\lambda \in (0, \delta_2)$. Como el dominio $\mathcal{X}$ es abierto, existe un paso $\delta_3 > 0$ tal que $\bar{x} + \lambda d \in \mathcal{X}$ para todo $\lambda \in (0, \delta_3)$.

Tomando el paso mínimo $\delta = \min\{\delta_1, \delta_2, \delta_3\} > 0$, se concluye que $\bar{x} + \lambda d$ es factible para todo $\lambda \in (0, \delta)$, lo que implica que $d \in \mathcal{F}(\bar{x})$. Sin embargo, dado que $d \in \mathcal{D}_0(\bar{x})$ por suposición, se cumple que $\nabla f(\bar{x})^T d < 0$, lo que implicaría que existe un paso de mejora factible, contradiciendo que $\bar{x}$ sea un mínimo local (puesto que $\mathcal{D}_0(\bar{x}) \cap \mathcal{F}(\bar{x}) = \emptyset$). Por tanto, la intersección es vacía. ■

4.4. Condiciones de Fritz-John (FJ)

El hecho geométrico de que $\mathcal{D}_0(\bar{x}) \cap \mathcal{G}_0(\bar{x}) = \emptyset$ se puede traducir algebraicamente, aplicando teoremas de alternativa (como el Teorema de Gordan o el Lema de Farkas), en una condición de dependencia lineal de los vectores gradiente. El matemático Fritz John formuló estas condiciones en 1948.

! Teorema: Condiciones Necesarias de Fritz-John (1948)

Sea $\bar{x}$ un mínimo local del problema restringido por desigualdades. Si f y las restricciones activas g_i ($i \in \mathcal{I}$) son diferenciables en $\bar{x}$, y las inactivas son continuas, existen escalares $u_0, u_1, \dots, u_m$ tales que:

1. Factibilidad dual (Ecuación de Fritz-John):

$$u_0 \nabla f(\bar{x}) + \sum_{i=1}^m u_i \nabla g_i(\bar{x}) = 0$$

2. Holgura complementaria:

$$u_i g_i(\bar{x}) = 0, \quad \forall i = 1, \dots, m$$

3. No negatividad:

$$u_0, u_i \geq 0, \quad \forall i = 1, \dots, m$$

4. No trivialidad:

$$(u_0, u_1, \dots, u_m) \neq (0, 0, \dots, 0)$$

4.4.1. Limitación de Fritz-John: El Caso de Multiplicador Nulo ($u_0 = 0$)

Las condiciones de Fritz-John presentan un inconveniente analítico: es posible que en el óptimo el multiplicador asociado a la función objetivo sea cero ($u_0 = 0$). En este escenario, la relación se reduce a:

$$\sum_{i \in \mathcal{J}} u_i \nabla g_i(\bar{x}) = 0$$

Esto sucede si los gradientes de las restricciones activas son linealmente dependientes en el punto de interés. En tal caso, las condiciones de Fritz-John se satisfacen independientemente de cuál sea la función objetivo a optimizar, perdiendo todo su poder selectivo sobre el óptimo real del problema.

Caso de Estudio: Fallo de Fritz-John por Multiplicador Nulo

Consideremos el problema:

$$\begin{aligned} \min_x \quad & -x_1 \\ \text{sueto a} \quad & g_1(x_1, x_2) = x_2 - (1 - x_1)^3 \leq 0 \\ & g_2(x_1, x_2) = -x_2 \leq 0 \end{aligned}$$

La solución óptima de este problema es $\bar{x} = (1, 0)$, donde ambas restricciones son activas ($\mathcal{J} = \{1, 2\}$). Planteando las ecuaciones de Fritz-John:

$$u_0 \begin{pmatrix} -1 \\ 0 \end{pmatrix} + u_1 \begin{pmatrix} 3(1 - x_1)^2 \\ 1 \end{pmatrix} + u_2 \begin{pmatrix} 0 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Evaluando en el punto óptimo $\bar{x} = (1, 0)$:

$$u_0 \begin{pmatrix} -1 \\ 0 \end{pmatrix} + u_1 \begin{pmatrix} 0 \\ 1 \end{pmatrix} + u_2 \begin{pmatrix} 0 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies \begin{aligned} -u_0 &= 0 \\ u_1 - u_2 &= 0 \end{aligned}$$

De donde se deduce obligatoriamente que $u_0 = 0$, e $u_1 = u_2 = \alpha > 0$. La función objetivo no influye en las ecuaciones debido a la geometría singular de la región factible en dicho punto (la tangencia de las curvas de restricción).

4.5. Condiciones de Karush-Kuhn-Tucker (KKT)

Para garantizar que la función objetivo participe activamente en la caracterización de optimalidad ($u_0 > 0$), debemos imponer una condición de regularidad sobre la geometría de las restricciones en el óptimo, denominada **cualificación de restricciones** (Constraint Qualification).

4.5.1. Historia de KKT

Las condiciones KKT tienen una historia singular de redescubrimiento. Fueron formuladas por primera vez en 1939 por **William Karush** en su tesis de maestría inédita en la Universidad de Chicago. Doce años después, en 1951, los matemáticos **Harold W. Kuhn** y **Albert W. Tucker** las redescubrieron de manera independiente, publicándolas en un célebre artículo de congreso. La comunidad científica finalmente reconoció la contribución primordial de Karush renombrando las condiciones como Karush-Kuhn-Tucker, constituyendo una de las bases analíticas de la optimización matemática moderna (Bazaraa, Sherali, y Shetty 2013; Boyd y Vandenberghe 2004; Nocedal y Wright 2006).

4.5.2. Cualificación de Restricciones (LICQ y otras)

La cualificación de restricciones más habitual es la **Independencia Lineal de las Restricciones Activas (LICQ)**: los gradientes de las restricciones activas $\{\nabla g_i(\bar{x}), \forall i \in \mathcal{I}\}$ deben ser linealmente independientes.

Existen otras cualificaciones de restricciones importantes en la literatura:

- **LICQ (Linear Independence Constraint Qualification)**: Gradientes de restricciones activas linealmente independientes. Satisface e implica todas las demás cualificaciones.
- **MFCQ (Mangasarian-Fromovitz Constraint Qualification)**: Los gradientes de restricciones de igualdad son linealmente independientes y existe una dirección factible estrictamente interior para las desigualdades.
- **Condición de Slater**: Para problemas convexos, existe al menos un punto x estrictamente factible ($g_i(x) < 0$ para toda i).

4.5.3. Condiciones Necesarias de KKT

! Teorema: Condiciones Necesarias de KKT

Sea $\bar{x}$ un mínimo local del problema. Si se verifica la cualificación de restricciones (LICQ) en $\bar{x}$, entonces existen multiplicadores únicos $u_1, \dots, u_m \geq 0$ tales que:

1. **Estacionariedad (Gradiente nulo de la Lagrangiana)**:

$$\nabla f(\bar{x}) + \sum_{i=1}^m u_i \nabla g_i(\bar{x}) = 0$$

2. **Factibilidad primal**:

$$g_i(\bar{x}) \leq 0, \quad \forall i = 1, \dots, m$$

3. Holgura complementaria:

$$u_i g_i(\bar{x}) = 0, \quad \forall i = 1, \dots, m$$

4. No negatividad (Factibilidad dual):

$$u_i \geq 0, \quad \forall i = 1, \dots, m$$

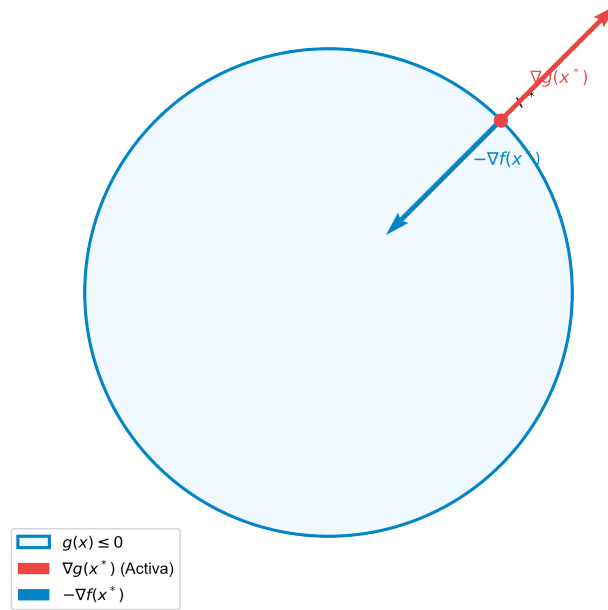


Figura 4.2: Interpretación geométrica de las condiciones KKT con restricciones activas

4.5.4. Condiciones Suficientes de KKT bajo Convexidad

! Teorema: Condiciones Suficientes de KKT bajo Convexidad

Sea $\bar{x}$ un punto factible que satisface las condiciones necesarias de KKT con multiplicadores u_i . Si:

1. La función objetivo $f(x)$ es **pseudo-convexa** en $\bar{x}$.
2. Las restricciones activas $g_i(x)$ ($i \in \mathcal{I}$) son **cuasi-convexas** y diferenciables en $\bar{x}$.

Entonces $\bar{x}$ es un mínimo global del problema.

4.6. Esquema General de Resolución KKT

El procedimiento analítico para resolver problemas mediante KKT se fundamenta en los siguientes pasos:

1. **Construir la función Lagrangiana:**

$$L(x, u) = f(x) + \sum_{i=1}^m u_i g_i(x)$$

2. **Plantear las ecuaciones del sistema KKT:**

- $\nabla_x L(x, u) = 0$.
- $g_i(x) \leq 0$, para todo i .
- $u_i g_i(x) = 0$, para todo i .
- $u_i \geq 0$, para todo i .

3. **Análisis por casos combinatorios:** Para cada restricción i , se evalúan los casos:

- *Caso A:* La restricción está inactiva ($g_i(x) < 0 \implies u_i = 0$).
- *Caso B:* La restricción está activa ($g_i(x) = 0 \implies u_i \geq 0$).

4. **Descartar candidatos no factibles o con multiplicadores negativos.**
5. **Comprobar la cualificación LICQ y aplicar la suficiencia por convexidad.**

💡 Caso Práctico: Resolución Analítica Detallada

Consideremos el problema:

$$\begin{aligned} \min_{x,y} \quad & f(x, y) = (x - 3)^2 + (y - 2)^2 \\ \text{sujeto a} \quad & g_1(x, y) = x^2 + y^2 - 5 \leq 0 \\ & g_2(x, y) = x + 2y - 4 \leq 0 \end{aligned}$$

La función objetivo mide el cuadrado de la distancia al punto $(3, 2)$. Construimos la

Lagrangiana:

$$L(x, y, u_1, u_2) = (x - 3)^2 + (y - 2)^2 + u_1(x^2 + y^2 - 5) + u_2(x + 2y - 4)$$

El sistema de derivadas parciales con respecto a x e y es:

1. $\frac{\partial L}{\partial x} = 2(x - 3) + 2u_1x + u_2 = 0$
2. $\frac{\partial L}{\partial y} = 2(y - 2) + 2u_1y + 2u_2 = 0$

Analizamos el caso donde ambas restricciones son activas ($g_1 = 0$ y $g_2 = 0$):

- $x^2 + y^2 = 5$
- $x + 2y = 4 \implies x = 4 - 2y$

Sustituyendo en la primera:

$$(4 - 2y)^2 + y^2 = 5 \implies 5y^2 - 16y + 11 = 0$$

Las soluciones son:

- $y = 1 \implies x = 2$.
- $y = 2.2 \implies x = -0.4$ (este punto viola la restricción $g_1 \leq 0$, por lo que se descarta).

Evaluamos el candidato $\bar{x} = (2, 1)$ en el sistema de derivadas para hallar los multiplicadores:

$$2(2 - 3) + 4u_1 + u_2 = 0 \implies 4u_1 + u_2 = 2$$

$$2(1 - 2) + 2u_1 + 4u_2 = 0 \implies 2u_1 + 4u_2 = 2 \implies u_1 + 2u_2 = 1$$

Resolviendo el sistema lineal de multiplicadores:

$$u_1 = \frac{1}{3} \geq 0, \quad u_2 = \frac{2}{3} \geq 0$$

Al ser ambos multiplicadores no negativos, el punto $(2, 1)$ satisface KKT. Como la función objetivo es cuadrática estrictamente convexa y las restricciones son convexas, se cumplen las condiciones suficientes, lo que garantiza que $(2, 1)$ es el **mínimo global único** del problema.

4.7. Restricciones de Igualdad y Desigualdad

El marco analítico se extiende al incorporar restricciones de igualdad ($h_j(x) = 0$).

! Teorema: Condiciones Necesarias Generales de KKT

Sea $\bar{x}$ un mínimo local del problema restringido por desigualdades y ecuaciones. Si se cumple LICQ (los gradientes de las restricciones de igualdad y de las restricciones activas de desigualdad son linealmente independientes en $\bar{x}$), existen multiplicadores únicos $u_1, \dots, u_m \geq 0$ y $v_1, \dots, v_\ell \in \mathbb{R}$ tales que:

1. **Estacionariedad:**

$$\nabla f(\bar{x}) + \sum_{i=1}^m u_i \nabla g_i(\bar{x}) + \sum_{j=1}^{\ell} v_j \nabla h_j(\bar{x}) = 0$$

2. **Factibilidad primal:**

$$g_i(\bar{x}) \leq 0 \quad (\forall i), \quad h_j(\bar{x}) = 0 \quad (\forall j)$$

3. **Holgura complementaria:**

$$u_i g_i(\bar{x}) = 0, \quad \forall i = 1, \dots, m$$

4. **No negatividad (desigualdades):**

$$u_i \geq 0, \quad \forall i = 1, \dots, m$$

5 Aplicaciones de KKT: Programación Lineal y Cuadrática

Las condiciones de Karush-Kuhn-Tucker (KKT) adquieren una relevancia excepcional al aplicarse sobre problemas con estructuras matemáticas específicas, como la **Programación Lineal (LP)** y la **Programación Cuadrática (QP)**. En ambos casos, las restricciones son afines, lo que garantiza que las cualificaciones de restricciones (como LICQ) se verifiquen de forma natural en cualquier punto factible. Además, cuando la función objetivo es convexa, las condiciones necesarias de KKT se convierten también en suficientes, proporcionando un marco algebraico cerrado para hallar y certificar el óptimo global.

En este capítulo, analizaremos la relación entre las condiciones KKT y la dualidad lineal, explicaremos la interpretación económica de las variables duales como precios sombra, clasificaremos los tipos de óptimos lineales, describiremos el método del Símples de Wolfe para programación cuadrática y mostraremos su resolución práctica mediante **CVXPY**.

! Objetivos de aprendizaje

Al finalizar este capítulo, serás capaz de:

1. **Deducir y justificar** la equivalencia entre los multiplicadores KKT de un problema de programación lineal y las variables óptimas de su problema dual asociado.
2. **Demostrar formalmente** el teorema de dualidad débil en programación lineal.
3. **Interpretar económicamente** las variables duales como precios sombra (shadow prices) de los recursos limitados.
4. **Clasificar los tipos de óptimos** en programación lineal atendiendo a la unicidad, degeneración y acotación tanto del problema primal como del dual.
5. **Formular el sistema KKT** para problemas de programación cuadrática (QP) en forma matricial.
6. **Aplicar el algoritmo del Símples de Wolfe** (regla de entrada restringida) para resolver de forma exacta problemas cuadráticos convexos.
7. **Implementar y resolver** modelos LP y QP en Python empleando la librería de optimización convexa **CVXPY**.

5.1. Programación Lineal (LP)

La Programación Lineal consiste en optimizar una función lineal sujeta a restricciones lineales de igualdad y desigualdad. Su simplicidad estructural permite deducir propiedades de dualidad muy potentes, cuyas bases matemáticas e históricas se detallan exhaustivamente en los textos de programación matemática (Bazaraa, Jarvis, y Sherali 2011; Luenberger y Ye 2015).

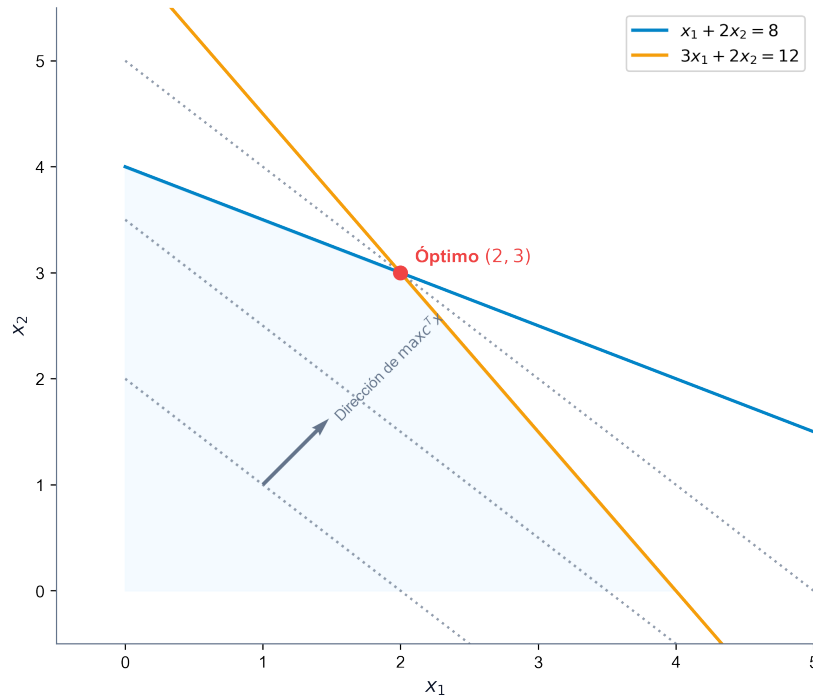


Figura 5.1: Región factible y dirección de optimización en un problema de programación lineal

5.1.1. Dualidad en Programación Lineal

Consideremos el par de problemas primal (P) y dual (D) en su forma canónica estándar:

■ **Problema Primal (P):**

$$\begin{aligned} & \min_x c^T x \\ & \text{sujeto a } Ax \geq b \\ & x \geq 0 \end{aligned}$$

■ **Problema Dual (D):**

$$\begin{aligned} & \underset{w}{\text{máx}} && b^T w \\ & \text{sujeto a} && A^T w \leq c \\ & && w \geq 0 \end{aligned}$$

! **Teorema: Teorema de Dualidad Débil en LP**

Para cualquier solución factible x del problema primal (P) y cualquier solución factible w del problema dual (D), el valor de la función objetivo dual es una cota inferior del valor de la función objetivo primal:

$$b^T w \leq c^T x$$

Demostración: Dado que x es factible en (P), se cumple que $Ax \geq b$. Multiplicando por la izquierda por el vector dual no negativo $w^T \geq 0$, preservamos el sentido de la desigualdad:

$$w^T Ax \geq w^T b = b^T w$$

De forma análoga, dado que w es factible en (D), se tiene que $A^T w \leq c$, lo que equivale a $w^T A \leq c^T$. Multiplicando por la derecha por el vector primal no negativo $x \geq 0$:

$$w^T Ax \leq c^T x$$

Combinando ambas desigualdades encadenadas, obtenemos:

$$b^T w \leq w^T Ax \leq c^T x \implies b^T w \leq c^T x. \quad \blacksquare$$

5.1.2. Condiciones KKT en Programación Lineal

Para aplicar el teorema de KKT, reescribimos el problema primal (P) en formato estándar de minimización con restricciones de tipo ≤ 0 :

$$\begin{aligned} & \underset{x}{\text{mín}} && c^T x \\ & \text{sujeto a} && b - Ax \leq 0 \\ & && -x \leq 0 \end{aligned}$$

Asociamos el vector de multiplicadores $u \in \mathbb{R}^m$ ($u \geq 0$) a las restricciones de recursos ($b - Ax \leq 0$) y el vector $v \in \mathbb{R}^n$ ($v \geq 0$) a las de no negatividad ($-x \leq 0$). La función Lagrangiana es:

$$L(x, u, v) = c^T x + u^T (b - Ax) - v^T x$$

Las condiciones de optimalidad de KKT para este problema son:

- **Estacionariedad:**

$$\nabla_x L(x, u, v) = c - A^T u - v = 0$$

- **Factibilidad Primal:**

$$Ax \geq b, \quad x \geq 0$$

- **Factibilidad Dual:**

$$u \geq 0, \quad v \geq 0$$

- **Holgura Complementaria:**

$$u_i(b_i - a_i^T x) = 0, \quad \forall i = 1, \dots, m$$

$$v_j x_j = 0, \quad \forall j = 1, \dots, n$$

De la ecuación de estacionariedad despejamos $c = A^T u + v$. Multiplicando por x^T por la izquierda:

$$c^T x = u^T A x + v^T x$$

Aplicando las condiciones de holgura complementaria ($v^T x = 0$ y $u^T A x = u^T b$):

$$c^T x = u^T b = b^T u$$

Esto demuestra que el vector de multiplicadores KKT u es factible en el problema dual (D) y proporciona exactamente el mismo valor objetivo que la solución primal x . Por el teorema de dualidad fuerte, **los multiplicadores KKT u del problema primal son idénticos a las variables óptimas w^* del problema dual.**

5.1.3. Interpretación Económica: Precios Sombra

En el ámbito de la economía y la gestión de operaciones, los problemas de optimización lineal suelen representar la asignación de recursos limitados (como horas de mano de obra, materias primas o presupuesto) para maximizar un beneficio o minimizar un coste de producción. En este marco:

- Las variables primales x_j representan los **niveles de actividad** (las cantidades de cada producto a fabricar).
- Los límites b_i representan la **disponibilidad disponible** de cada recurso i .
- Los multiplicadores duales u_i (o variables duales w_i) representan los **precios sombra** (shadow prices) de los recursos.

El precio sombra u_i mide la tasa de cambio del beneficio óptimo ante un incremento unitario en la disponibilidad del recurso b_i :

$$u_i^* = \frac{\partial(c^T x^*)}{\partial b_i}$$

La condición de holgura complementaria $u_i(b_i - a_i^T x^*) = 0$ posee entonces una interpretación económica intuitiva y elegante:

1. Si al alcanzar el plan de producción óptimo sobra parte del recurso i ($b_i - a_i^T x^* > 0$), dicho recurso no es un factor limitante (es abundante). Por tanto, añadir más cantidad del mismo no aumentará el beneficio, lo que obliga a que su precio sombra sea cero ($u_i^* = 0$).
2. Si un recurso posee un precio sombra estrictamente positivo ($u_i^* > 0$), significa que dicho recurso es valioso y está limitando la producción. En consecuencia, se consumirá por completo en el óptimo, dejando la holgura en cero ($b_i - a_i^T x^* = 0$).

5.1.4. Clasificación de Óptimos y Relaciones de Dualidad

Atendiendo a la geometría del espacio y la unicidad de las soluciones, se clasifican los siguientes escenarios recíprocos entre primal y dual (analizados sobre un espacio en $\mathbb{R}^2$):

■ **Caso 1: Solución Única y No Degenerada:**

- *Primal*: Mínimo único situado en un vértice donde se cruzan exactamente n restricciones activas.
- *Dual*: Solución única y no degenerada. Los multiplicadores asociados a las restricciones activas del primal son estrictamente positivos ($u_i > 0$).

■ **Caso 2: Solución Única Degenerada:**

- *Primal*: Mínimo situado en un vértice donde coinciden más de n restricciones activas (sobredeterminación). Los gradientes de estas restricciones son linealmente dependientes.
- *Dual*: El dual presenta múltiples soluciones óptimas alternas.

■ **Caso 3: Múltiples Soluciones No Degeneradas:**

- *Primal*: La función de coste es paralela a una de las restricciones activas. El óptimo es el segmento que conecta dos vértices.
- *Dual*: La solución del dual es única, pero degenerada (presenta algún multiplicador nulo en las restricciones activas del primal).

■ **Caso 4: Múltiples Soluciones Degeneradas:**

- *Primal*: El óptimo es una cara o segmento factible, pero alguno de los vértices extremos está sobredeterminado por restricciones activas redundantes.
- *Dual*: El dual presenta infinitas soluciones óptimas, muchas de ellas degeneradas.

■ **Caso 5: Primal No Acotado:**

- *Primal*: El valor objetivo decrece indefinidamente hacia $-\infty$ a lo largo de una dirección extrema de recesión factible. No existe punto KKT.
- *Dual*: El problema dual es no factible.

■ **Caso 6: Primal No Factible:**

- *Primal*: La región factible es vacía (restricciones incompatibles).
- *Dual*: El dual es no factible, o posee solución no acotada.

5.2. Programación Cuadrática (QP)

La programación cuadrática optimiza una función objetivo cuadrática sujeta a un conjunto de restricciones lineales de igualdad o desigualdad:

$$\begin{aligned} \min_x \quad & f(x) = c^T x + \frac{1}{2} x^T Q x \\ \text{sujeto a} \quad & Ax \geq b \\ & x \geq 0 \end{aligned}$$

Para garantizar que el problema sea convexo (y que las condiciones KKT sean necesarias y suficientes para alcanzar el óptimo global), la matriz simétrica $Q \in \mathbb{R}^{n \times n}$ debe ser **semidefinida positiva** ($Q \succeq 0$).

5.2.1. El Sistema KKT para QP en Forma Matricial

Asociando multiplicadores $u \geq 0$ para las restricciones de recursos y $v \geq 0$ para las de no negatividad, la Lagrangiana es:

$$L(x, u, v) = c^T x + \frac{1}{2} x^T Q x + u^T (b - Ax) - v^T x$$

Derivando con respecto a x y planteando KKT, obtenemos:

$$\begin{aligned} Ax - y &= b, \quad y \geq 0 \\ Qx + c - A^T u - v &= 0, \quad x \geq 0, \quad v \geq 0 \\ u &\geq 0, \quad y \geq 0 \\ u^T y &= 0, \quad v^T x = 0 \end{aligned}$$

donde $y \geq 0$ es el vector de variables de holgura del primal.

Podemos agrupar la parte lineal de este sistema en forma de sistema matricial ampliado:

$$\begin{pmatrix} Q & -A^T & -I & 0 \\ A & 0 & 0 & -I \end{pmatrix} \begin{pmatrix} x \\ u \\ v \\ y \end{pmatrix} = \begin{pmatrix} -c \\ b \end{pmatrix}$$

donde recordamos que además deben satisfacerse las restricciones de no negatividad para todas las variables y las condiciones lógicas de complementariedad ($u^T y + v^T x = 0$).

5.2.2. Linealización y Algoritmo del Símplex de Wolfe

El sistema KKT es lineal en todas sus partes excepto por la condición de complementariedad $u^T y + v^T x = 0$. Philip Wolfe propuso en 1959 adaptar el algoritmo del Símplex lineal para resolver este sistema utilizando una técnica de **entrada restringida** (restricted entry rule):

1. Se relaja temporalmente la complementariedad $u^T y + v^T x = 0$.
2. Se introducen variables artificiales no negativas $z_j \geq 0$ en las ecuaciones de estacionariedad para obtener una solución básica inicial obvia.
3. Se minimiza la suma de las variables artificiales $\min \sum z_j$ usando las operaciones de pivote del Símplex, pero imponiendo una restricción en la elección de la variable que entra en la base:
 - La variable dual u_i no puede entrar en la base si su holgura primal asociada y_i es actualmente una variable básica.
 - La holgura dual v_j no puede entrar en la base si la variable primal correspondiente x_j es actualmente básica.
4. Si el algoritmo finaliza con un coste óptimo de cero (todas las artificiales fuera de la base), el punto resultante satisface KKT y es el óptimo global del problema cuadrático.

5.3. Caso Práctico Detallado: Algoritmo de Wolfe

Consideremos el problema de minimizar la distancia euclídea al punto $(4, 4)$ en una región delimitada por tres restricciones lineales:

$$\begin{aligned} \min \quad & (x_1 - 4)^2 + (x_2 - 4)^2 \equiv \frac{1}{2} x^T \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} x + \begin{pmatrix} -8 \\ -8 \end{pmatrix}^T x + 32 \\ \text{sujeto a} \quad & x_1 + x_2 \leq 4 \\ & -x_1 + x_2 \leq 2 \\ & 2x_1 + x_2 \leq 7 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Formulamos el sistema lineal de Wolfe agregando variables primales de holgura y_i , multiplicadores duales u_i , holguras duales v_j y variables artificiales z_j :

- Ecuaciones primales:
 - $x_1 + x_2 + y_1 = 4$
 - $-x_1 + x_2 + y_2 = 2$
 - $2x_1 + x_2 + y_3 = 7$
- Ecuaciones de estacionariedad (evaluando $Qx + c - A^T u - v + z = 0$):
 - $2x_1 - 8 + u_1 - u_2 + 2u_3 - v_1 + z_1 = 0 \implies 2x_1 + u_1 - u_2 + 2u_3 - v_1 + z_1 = 8$
 - $2x_2 - 8 + u_1 + u_2 + u_3 - v_2 + z_2 = 0 \implies 2x_2 + u_1 + u_2 + u_3 - v_2 + z_2 = 8$

5.3.1. Progresión de las Tablas del Símplex de Wolfe

5.3.1.1. Tabla Inicial (Iteración 1)

- **Base:** $\{y_1, y_2, y_3, z_1, z_2\}$
- **Valor del Objetivo:** 16 (Suma de artificiales $z_1 + z_2 = 8 + 8 = 16$)

Ba- se	Sol.	x_1	x_2	y_1	y_2	y_3	u_1	u_2	u_3	v_1	v_2	z_1	z_2
y_1	4	1	1	1	0	0	0	0	0	0	0	0	0
y_2	2	-1	1	0	1	0	0	0	0	0	0	0	0
y_3	7	2	1	0	0	1	0	0	0	0	0	0	0
z_1	8	2	0	0	0	0	1	-1	2	-1	0	1	0
z_2	8	0	2	0	0	0	1	1	1	0	-1	0	1
Cos- te	16	-2	-2	0	0	0	-2	0	-3	1	1	0	0

Análisis: Los costes reducidos más negativos corresponden a las variables x_1, x_2, u_1, u_3 . Por la regla de entrada restringida, no podemos introducir u_1 ni u_3 porque sus holguras asociadas y_1, y_3 se encuentran actualmente en la base (son básicas). Por tanto, seleccionamos x_2 para entrar en la base. Aplicando el criterio de la razón mínima, la variable saliente es y_2 (pivote = 1).

5.3.1.2. Tabla 2 (Iteración 2)

- **Base:** $\{y_1, x_2, y_3, z_1, z_2\}$
- **Valor del Objetivo:** 12

Ba- se	Sol.	x_1	x_2	y_1	y_2	y_3	u_1	u_2	u_3	v_1	v_2	z_1	z_2
y_1	2	2	0	1	-1	0	0	0	0	0	0	0	0
x_2	2	-1	1	0	1	0	0	0	0	0	0	0	0
y_3	5	3	0	-1	-1	1	0	0	0	0	0	0	0
z_1	8	2	0	0	0	0	1	-1	2	-1	0	1	0
z_2	4	2	0	0	-2	0	1	1	1	0	-1	0	1
Cos- te	12	-4	0	0	2	0	-2	0	-3	1	1	0	0

Análisis: u_1, u_3 siguen bloqueados por y_1, y_3 . Seleccionamos x_1 para entrar en la base. El criterio de la razón mínima determina que sale y_1 (pivote = 2).

5.3.1.3. Tabla 3 (Iteración 3)

- **Base:** $\{x_1, x_2, y_3, z_1, z_2\}$
- **Valor del Objetivo:** 8

Ba- se	Sol.	x_1	x_2	y_1	y_2	y_3	u_1	u_2	u_3	v_1	v_2	z_1	z_2
x_1	1	1	0	1/2	-1/2	0	0	0	0	0	0	0	0
x_2	3	0	1	1/2	1/2	0	0	0	0	0	0	0	0
y_3	2	0	0	-5/2	1/2	1	0	0	0	0	0	0	0
z_1	6	0	0	-1	1	0	1	-1	2	-1	0	1	0
z_2	2	0	0	-1	-1	0	1	1	1	0	-1	0	1
Cos- te	8	0	0	2	0	0	-2	0	-3	1	1	0	0

Análisis: Como y_1 ha salido de la base ($y_1 = 0$), se desbloquea la entrada de u_1 . Entra u_1 y sale z_2 (pivote = 1).

5.3.1.4. Tabla 4 (Iteración 4)

- **Base:** $\{x_1, x_2, y_3, z_1, u_1\}$
- **Valor del Objetivo:** 4

Ba- se	Sol.	x_1	x_2	y_1	y_2	y_3	u_1	u_2	u_3	v_1	v_2	z_1	z_2
x_1	1	1	0	1/2	-1/2	0	0	0	0	0	0	0	0
x_2	3	0	1	1/2	1/2	0	0	0	0	0	0	0	0
y_3	2	0	0	-5/2	1/2	1	0	0	0	0	0	0	0
z_1	4	0	0	0	2	0	0	-2	1	-1	1	1	-1
u_1	2	0	0	-1	-1	0	1	1	1	0	-1	0	1
Cos- te	4	0	0	0	-2	0	0	2	-1	1	-1	0	2

Análisis: u_3, v_2 siguen bloqueados por y_3, x_2 . La única variable viable con coste reducido negativo es la holgura primal y_2 . Entra y_2 y sale z_1 (pivote = 2).

5.3.1.5. Tabla Final (Iteración 5)

- **Base:** $\{x_1, x_2, y_3, y_2, u_1\}$
- **Valor del Objetivo:** 0

Ba- se	Sol.	x_1	x_2	y_1	y_2	y_3	u_1	u_2	u_3	v_1	v_2	z_1	z_2
x_1	2	1	0	1/2	0	0	0	-1/2	1/4	-1/4	1/4	1/4	-1/4
x_2	2	0	1	1/2	0	0	0	1/2	-1/4	1/4	-1/4	-1/4	1/4
y_3	1	0	0	-5/2	0	1	0	1/2	-1/4	1/4	-1/4	-1/4	1/4
y_2	2	0	0	0	1	0	0	-1	1/2	-1/2	1/2	1/2	-1/2
u_1	4	0	0	-1	0	0	1	0	3/2	-1/2	-1/2	1/2	1/2
Cos- te	0	0	0	0	0	0	0	0	0	0	0	1	1

El coste del objetivo es cero, lo que significa que todas las variables artificiales han salido de la base. Hemos hallado la solución óptima:

$$x_1^* = 2, \quad x_2^* = 2$$

Con multiplicador activo $u_1 = 4$. El valor mínimo de la distancia al cuadrado es $f(2, 2) = (2 - 4)^2 + (2 - 4)^2 = 8$.

Análisis Geométrico de la Trayectoria de Wolfe

La progresión de las tablas del Símpex de Wolfe dibuja una trayectoria sobre la frontera del politopo factible en el plano $\mathbb{R}^2$:

1. **Iteración 1:** Comienza en el origen $(0, 0)$.
2. **Iteración 2:** La variable x_2 entra en la base, desplazándose sobre el eje de ordenadas hasta el vértice $(0, 2)$, donde se activa la restricción $g_2(x) \leq 0$ ($y_2 = 0$).
3. **Iteración 3:** La variable x_1 entra en la base. El algoritmo avanza por la frontera de la restricción $-x_1 + x_2 \leq 2$ hasta el vértice $(1, 3)$. En este punto se alcanza la intersección con la restricción $x_1 + x_2 \leq 4$, por lo que y_1 sale de la base.
4. **Iteración 4:** Al salir y_1 de la base, se permite la entrada del multiplicador dual u_1 ($u_1 = 2$). El punto físico permanece en $(1, 3)$ mientras se realiza el ajuste de las tensiones duales.
5. **Iteración 5:** Entra la holgura primal y_2 (desactivando la restricción $g_2(x) \leq 0$). El punto se desliza a lo largo del segmento definido por la restricción activa $x_1 + x_2 = 4$ hasta alcanzar el punto más cercano al objetivo $(4, 4)$, que se sitúa en $(2, 2)$, con un multiplicador final de $u_1 = 4$.

5.4. Formulación en Python con CVXPY

Código Python: Resolución de LP con Restricciones Convexas en CVXPY

El siguiente script resuelve en Python el problema de programación lineal con restricciones de tipo elipse:

$$\min\{3x - y \mid x^2 + y^2 \leq 1, 4x + y \geq 1\}$$

```

import cvxpy as cp

# 1. Definir variables de decision
x = cp.Variable()
y = cp.Variable()

# 2. Definir funcion objetivo
objetivo = cp.Minimize(3 * x - y)

# 3. Definir restricciones
restricciones = [
    cp.square(x) + cp.square(y) <= 1,
    4 * x + y >= 1
]

# 4. Formular y resolver el problema
problema = cp.Problem(objetivo, restricciones)
problema.solve()

print(f"Estado del problema: {problema.status}")
print(f"Solucion optima: x = {x.value:.4f}, y = {y.value:.4f}")
print(f"Valor optimo de la funcion: {problema.value:.4f}")

```

💡 Código Python: Resolución del QP de Wolfe con CVXPY

Resolvemos el problema cuadrático resuelto paso a paso por el método de Wolfe para verificar la solución:

```

import cvxpy as cp
import numpy as np

# 1. Definir vector de variables de decision
x = cp.Variable(2)

# 2. Definir matrices del problema cuadrático
#  $f(x) = 1/2 * x^T * P * x + q^T * x + 32$ 
P = np.array([[2.0, 0.0], [0.0, 2.0]])
q = np.array([-8.0, -8.0])

objetivo = cp.Minimize(0.5 * cp.quad_form(x, P) + q.T @ x + 32.0)

# 3. Definir restricciones lineales
restricciones = [
    x[0] + x[1] <= 4,
    -x[0] + x[1] <= 2,
    2 * x[0] + x[1] <= 7,
    x[0] >= 0,
    x[1] >= 0
]

# 4. Resolver
problema = cp.Problem(objetivo, restricciones)
problema.solve()

print(f"Estado del problema: {problema.status}")
print(f"Solucion optima: x1 = {x.value[0]:.4f}, x2 = {x.value[1]:.4f}")
print(f"Valor minimo obtenido: {problema.value:.4f}")

```

6 Optimización en Ciencia de Datos: Regresión y SVM

La optimización matemática es el motor silencioso que impulsa la ciencia de datos y el aprendizaje automático (*machine learning*). Desde el ajuste clásico de una recta hasta el entrenamiento de redes neuronales profundas, el proceso de “aprendizaje” a partir de datos consiste en formular una función de pérdida que mide la discrepancia entre el modelo y la realidad, y resolver el problema de optimización asociado para hallar los parámetros óptimos. En este capítulo, estudiaremos cómo la optimización no lineal y la programación entera mixta dan respuesta a problemas clave: el ajuste de regresiones en diferentes normas, la selección óptima de variables, la formulación de las Máquinas de Vectores de Soporte (SVM) y la generación de explicaciones contrafácticas para modelos de caja negra.

! Objetivos de aprendizaje

Al finalizar este capítulo, serás capaz de:

1. **Formular y resolver** el problema de ajuste de curvas lineales y cuadráticas bajo diferentes normas (ℓ_1 , ℓ_2 , ℓ_∞) usando programación lineal y cuadrática.
2. **Modelar la selección de características** (*feature selection*) en regresión múltiple utilizando variables binarias y restricciones lógicas.
3. **Deducir la formulación de SVM** de margen máximo y margen blando, y comprender la importancia de su problema dual y el truco del kernel.
4. **Comprender la necesidad de explicabilidad** en ciencia de datos y formular el problema biobjetivo para la búsqueda de contrafácticos.
5. **Formular y resolver** el problema de búsqueda de contrafácticos en regresión logística como un modelo de optimización cuadrático entero mixto (MIQP).
6. **Implementar en Python** modelos de regresión multivariante y clasificadores SVM utilizando las librerías CVXPY y `scikit-learn`.

6.1. Introducción a la Optimización en Aprendizaje Automático

El proceso de entrenamiento de un modelo de aprendizaje automático consiste en ajustar los parámetros de una función hipótesis $f(x; w)$ para modelar un patrón en un conjunto de datos,

siguiendo los principios y metodologías analíticas del aprendizaje estadístico clásico (Hastie, Tibshirani, y Friedman 2009). Las principales aplicaciones de la optimización en este ámbito son:

- **Estimación puntual:** Maximizar la función de verosimilitud (MLE) o minimizar el error cuadrático medio.
- **Regularización:** Añadir términos de penalización en la función objetivo (normas ℓ_1 o ℓ_2) para controlar la complejidad del modelo y evitar el sobreajuste (*overfitting*).
- **Clasificación:** Maximizar el margen de separación entre clases o minimizar la pérdida de bisagra (*hinge loss*).

6.2. El Problema de Ajuste de Datos en Diferentes Normas

Dada una muestra de observaciones de dos variables, x e y , el ajuste de datos consiste en encontrar una función $f(x)$ tal que $y_i \approx f(x_i)$ para toda observación $i = 1, \dots, n$.

Consideremos una muestra de $n = 19$ observaciones:

x	0.0	0.5	1.0	1.5	1.9	2.5	3.0	3.5	4.0	4.5	5.0	5.5	6.0	6.6	7.0	7.6	8.5	9.0	10.0
y	1.0	0.9	0.7	1.5	2.0	2.4	3.2	2.0	2.7	3.5	1.0	4.0	3.6	2.7	5.7	4.6	6.0	6.8	7.3

6.2.1. Regresión Lineal Clásica (Norma ℓ_2)

La regresión lineal clásica asume que la relación toma la forma de una recta $y = a + bx + \epsilon$. Los parámetros a (intersección) y b (pendiente) se obtienen minimizando el error cuadrático medio:

$$\min_{a,b} \sum_{i=1}^n (a + bx_i - y_i)^2$$

Las ecuaciones normales obtenidas al anular el gradiente nos proporcionan la conocida recta por mínimos cuadrados ordinarios (MCO):

$$Y = 0.4261 + 0.6108x$$

6.2.2. Ajustes en Normas Robustas (ℓ_1 y ℓ_∞)

Cuando los datos contienen valores atípicos (*outliers*) o errores de medición severos (como el punto (5.0, 1.0) en nuestra muestra), la norma cuadrática ℓ_2 se ve fuertemente distorsionada. Proponemos alternativas más robustas:

- **Norma ℓ_1 (Desviaciones Absolutas)**: Minimiza la suma de los valores absolutos de los residuos:

$$\min_{a,b} \sum_{i=1}^n |a + bx_i - y_i|$$

- **Norma ℓ_∞ (Minimax)**: Minimiza el máximo error cometido:

$$\min_{a,b} \max_{i=1,\dots,n} |a + bx_i - y_i|$$

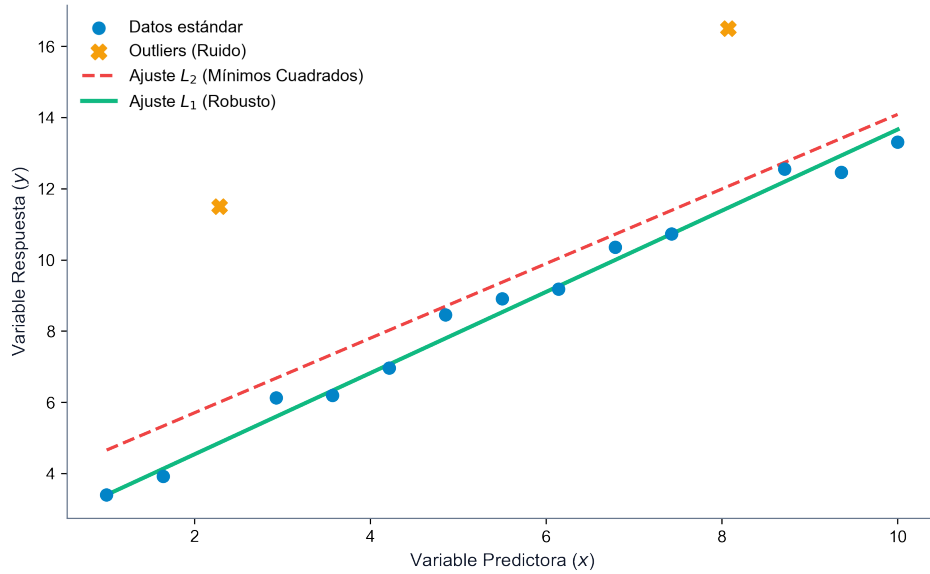


Figura 6.1: Comparativa del ajuste lineal robusto en norma l_1 frente a la distorsión por outliers de la norma l_2

i Formulación en Programación Lineal para Ajustes Robustos

Podemos formular la minimización en normas ℓ_1 y ℓ_∞ como problemas de programación lineal continua equivalentes mediante la introducción de variables auxiliares de error:

- **Formulación para la Norma ℓ_1 :**

$$\begin{aligned} & \min_{a,b,e} \sum_{i=1}^n e_i \\ \text{sujeto a } & a + bx_i - y_i \leq e_i, \quad i = 1, \dots, n \\ & -(a + bx_i - y_i) \leq e_i, \quad i = 1, \dots, n \\ & e_i \geq 0, \quad i = 1, \dots, n \end{aligned}$$

■ **Formulación para la Norma ℓ_∞ :**

$$\begin{aligned} \min_{a,b,z} \quad & z \\ \text{sujeto a} \quad & a + bx_i - y_i \leq z, \quad i = 1, \dots, n \\ & -(a + bx_i - y_i) \leq z, \quad i = 1, \dots, n \\ & z \geq 0 \end{aligned}$$

Al resolver estos modelos sobre los datos de nuestra muestra, se aprecian los siguientes ajustes lineales y cuadráticos:

1. **Ajuste lineal ℓ_1 :** $y = 0.6375x + 0.5812$ (con un error total de 11.46 y un máximo error de 2.77 en $x = 5.0$).
2. **Ajuste lineal ℓ_∞ :** $y = 0.6245x - 0.4000$ (con un error total de 19.95 y un máximo error minimizado de 1.725 en $x = 3.5, 7.0$).
3. **Ajuste cuadrático ℓ_1 :** $y = 0.0337x^2 + 0.2945x + 0.9823$.
4. **Ajuste cuadrático ℓ_∞ :** $y = 0.1250x^2 - 0.6250x + 2.4750$ (máximo error de 1.475).

6.3. Regresión Lineal Múltiple y Selección de Características

Cuando disponemos de m variables independientes $X_1, \dots, X_m$, el modelo es:

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_m X_m + \epsilon$$

6.3.1. Selección de Variables mediante Programación Entera Mixta

En presencia de múltiples variables explicativas, a menudo deseamos construir un modelo disperso (*sparse*) que solo incluya un subconjunto óptimo de características. Definimos variables binarias de decisión $\delta_j \in \{0, 1\}$ para indicar si la característica j participa en el modelo ($\beta_j \neq 0$). Imponemos restricciones de “Gran M” utilizando cotas inferiores $\underline{\beta}_j$ y superiores $\bar{\beta}_j$ sobre el valor físico de los coeficientes:

$$\underline{\beta}_j \delta_j \leq \beta_j \leq \bar{\beta}_j \delta_j, \quad j = 1, \dots, m$$

Esto nos permite plantear restricciones estructurales complejas:

■ **Número máximo de variables seleccionadas:**

$$\sum_{j=1}^m \delta_j \leq M_{\text{máx}}$$

- Relación de exclusión mutua entre variables i y j :

$$\delta_i + \delta_j \leq 1$$

- Inclusión obligatoria de al menos una entre i y j :

$$\delta_i + \delta_j \geq 1$$

El problema resultante es un modelo **Entero Cuadrático Mixto (MIQP)**.

6.4. Máquinas de Vectores de Soporte (SVM)

El objetivo de una SVM es clasificar observaciones binarias $y_i \in \{-1, +1\}$ mediante un hiperplano $w^T x + b = 0$ que maximice el margen de separación (la distancia al punto más cercano de cada clase).

! Formulación Primal de SVM (Caso Linealmente Separable)

La distancia del punto más cercano al hiperplano es $1/\|w\|_2$. Maximizar el margen $2/\|w\|_2$ equivale a minimizar su recíproco al cuadrado, dando lugar al problema cuadrático convexo restringido:

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|_2^2 \\ \text{sujeto a} \quad & y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

! Formulación Primal de SVM (Caso No Separable - Margen Blando)

Si los datos no son perfectamente separables, introducimos variables de holgura $\xi_i \geq 0$ que miden la penetración de un punto en el margen o su mala clasificación. El parámetro de regularización $C > 0$ controla el equilibrio entre la maximización del margen y la penalización de errores:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i \\ \text{sujeto a} \quad & y_i(w^T x_i + b) \geq 1 - \xi_i, \quad i = 1, \dots, n \\ & \xi_i \geq 0, \quad i = 1, \dots, n \end{aligned}$$

! Formulación Dual de SVM y el Truco del Kernel

Planteando las condiciones de optimalidad de la Lagrangiana, podemos reescribir el problema en su forma dual:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j) \\ \text{sujeto a} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

- **Vectores de soporte:** Los puntos con multiplicador óptimo $\alpha_i > 0$ son los únicos que determinan la posición del hiperplano óptimo.
- **El Truco del Kernel:** Dado que los datos de entrada solo intervienen en el problema dual mediante el producto escalar $x_i^T x_j$, podemos reemplazar este término por una función de coincidencia kernel $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$. Esto permite proyectar los datos de forma implícita a un espacio de características infinito o de gran dimensión (como el kernel Gaussiano RBF), logrando fronteras de decisión altamente no lineales en el espacio original sin incurrir en costes computacionales prohibitivos.

6.5. Explicabilidad y Contrafácticos (Counterfactuals)

El auge de los modelos de caja negra (redes neuronales, random forests) ha motivado el desarrollo de técnicas de explicabilidad. Una explicación contrafáctica responde a la pregunta: “¿Qué cambios mínimos debe realizar una persona x^0 clasificada en la clase negativa para cambiar la predicción a la clase positiva?”

6.5.1. Formulación del Problema de Optimización

Dado un clasificador $P : \mathcal{X} \rightarrow [0, 1]$ que devuelve la probabilidad de clase positiva, y un perfil x^0 con $P(x^0) < \tau$ (negativo), buscamos un nuevo perfil factible x que minimice la función de coste del cambio $C(x^0, x)$ bajo la condición de cruzar el umbral de clasificación:

$$\begin{aligned} \min_{x \in \mathcal{X}(x^0)} \quad & C(x^0, x) \\ \text{sujeto a} \quad & P(x) \geq \nu \end{aligned}$$

- **Contrafácticos Endógenos:** El conjunto de búsqueda se restringe a muestras reales de la base de datos ya clasificadas como positivas. Su modelización es discreta (localización de instalaciones). - **Contrafácticos Exógenos:** El contrafáctico generado es un perfil sintético en un espacio continuo, resolviéndose mediante optimización continua o entera mixta.

6.5.2. Formulación Entera Mixta para un Clasificador de Regresión Logística

En la regresión logística, la probabilidad de clase positiva es $\varphi(w^T x - b)$ con $\varphi(t) = 1/(1 + e^{-t})$. La restricción de probabilidad $P(x) \geq \nu$ se linealiza como:

$$w^T x - b \geq \ln \left(\frac{\nu}{1 - \nu} \right)$$

Si la función de coste combina la distancia euclídea (ℓ_2) y una penalización por el número de características modificadas (ℓ_0), el modelo se formula como un problema **MIQP**:

$$\begin{aligned} \min_{x, \delta} \quad & \sum_{k=1}^K (x_k^0 - x_k)^2 + \lambda \sum_{k=1}^K \delta_k \\ \text{sujeto a} \quad & \sum_{k=1}^K w_k x_k - b \geq \ln \left(\frac{\nu}{1 - \nu} \right) \\ & -M_k \delta_k \leq x_k^0 - x_k \leq M_k \delta_k, \quad k = 1, \dots, K \\ & x_k = x_k^0, \quad \forall k \in \mathcal{K}_{\text{inmutable}} \\ & \delta_k \in \{0, 1\}, \quad k = 1, \dots, K \end{aligned}$$

6.6. Implementación Práctica en Python

💡 Código Python: Ajustes en Diferentes Normas y SVM con CVXPY

El siguiente script completo implementa los ajustes de datos univariantes utilizando las normas $\ell_1, \ell_2, \ell_\infty$, y formula el clasificador SVM de margen blando en 2D:

```

import cvxpy as cp
import numpy as np

# 1. Datos del ajuste de curvas
x_data = np.array([0.0, 0.5, 1.0, 1.5, 1.9, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 6.6, 7.0, 7.5, 8.0, 8.5, 9.0, 9.5, 10.0])
y_data = np.array([1.0, 0.9, 0.7, 1.5, 2.0, 2.4, 3.2, 2.0, 2.7, 3.5, 1.0, 4.0, 3.6, 2.7, 5.0, 4.5, 3.8, 3.2, 2.5, 1.8, 1.0])

# Variables de decision para la recta y = a + bx
a = cp.Variable()
b = cp.Variable()

# --- Ajuste L2 (Minimos Cuadrados Ordinarios) ---
cost_l2 = cp.sum_squares(a + b * x_data - y_data)
prob_l2 = cp.Problem(cp.Minimize(cost_l2))
prob_l2.solve()
print("Recta L2 (MCO):")
print(f" y = {a.value:.4f} + {b.value:.4f}x")

# --- Ajuste L1 (Desviaciones Absolutas - Robustez) ---
cost_l1 = cp.sum(cp.abs(a + b * x_data - y_data))
prob_l1 = cp.Problem(cp.Minimize(cost_l1))
prob_l1.solve()
print("\nRecta L1:")
print(f" y = {a.value:.4f} + {b.value:.4f}x")

# --- Ajuste L_infinito (Minimax - Minimo Error Maximo) ---
cost_linf = cp.norm(a + b * x_data - y_data, "inf")
prob_linf = cp.Problem(cp.Minimize(cost_linf))
prob_linf.solve()
print("\nRecta L_infinito:")
print(f" y = {a.value:.4f} + {b.value:.4f}x")

# 2. Formulacion Primal de SVM de Margen Blando
np.random.seed(42)
X_svm = np.vstack([np.random.randn(10, 2) - 2, np.random.randn(10, 2) + 2])
y_svm = np.array([-1]*10 + [1]*10)
m, d = X_svm.shape

w = cp.Variable(d)
intercept = cp.Variable()
xi = cp.Variable(m)
C = 1.0 # Parametro de penalizacion

objetivo_svm = cp.Minimize(0.5 * cp.sum_squares(w) + C * cp.sum(xi))
restricciones_svm = [
    y_svm[i] * (X_svm[i] @ w + intercept) >= 1 - xi[i] for i in range(m)
] + [xi >= 0]

prob_svm = cp.Problem(objetivo_svm, restricciones_svm)
prob_svm.solve()
print("\nHiperplano Separador SVM:")
print(f" w = {w.value}")

```

7 Introducción a Grafos y Árboles Soporte

El **análisis de redes** es una de las disciplinas más transversales y aplicadas de las matemáticas discretas y la optimización. Muchas de las infraestructuras de la sociedad moderna (las redes de transporte eléctrico, el tráfico vial, la fibra óptica, Internet o los mapas metabólicos celulares) se modelan matemáticamente mediante **grafos**. La abstracción espacial que proporciona un grafo permite obviar la geometría física de los sistemas para centrarse en sus propiedades topológicas, conectividad y flujos, siguiendo los enfoques fundamentales de la optimización y algoritmos de redes (Ahuja, Magnanti, y Orlin 1993; Bertsekas 1998; Cormen et al. 2009).

En este capítulo, estudiaremos los conceptos elementales de la teoría de grafos, las formas estándar de representar redes en ordenador, las propiedades analíticas de los grafos planares y bipartitos, las caracterizaciones algebraicas de los árboles y los algoritmos clásicos de Kruskal y Prim para construir el árbol soporte de mínimo peso (MST) con sus aplicaciones en el aprendizaje no supervisado.

! Objetivos de aprendizaje

Al finalizar este capítulo, serás capaz de:

1. **Definir e identificar** los diferentes tipos de grafos (dirigidos, no dirigidos, valorados, bipartitos y planares) y sus elementos constituyentes.
2. **Aplicar el Teorema de Euler para grafos planares** para relacionar el número de vértices, aristas y regiones, y deducir los límites de densidad de aristas.
3. **Utilizar el Teorema de Kuratowski** para demostrar analíticamente si un grafo dado es planar o no planar mediante la búsqueda de subdivisiones de K_5 o $K_{3,3}$.
4. **Caracterizar la estructura de los árboles** empleando sus diversas definiciones equivalentes basadas en conectividad, aciclicidad y recuento de aristas.
5. **Explicar e implementar** los algoritmos de Kruskal y Prim para el cálculo del árbol soporte de mínimo peso (MST).
6. **Modelar problemas de agrupamiento** (*clustering*) jerárquico continuo a través del MST, y programar estas soluciones en Python usando la librería **NetworkX**.

7.1. Origen Histórico y Definiciones Básicas

7.1.1. El Problema de Königsberg (Euler, 1736)

La teoría de grafos nació formalmente en 1736 cuando Leonhard Euler resolvió el problema de los puentes de Königsberg. La ciudad estaba dividida por el río Pregel en cuatro porciones de tierra conectadas por siete puentes. El reto consistía en encontrar una ruta que cruzara cada puente exactamente una vez y regresara al origen. Euler demostró que tal recorrido (denominado hoy ciclo euleriano) era imposible. Lo fundamental de su análisis fue que obvió los detalles de las distancias o la forma de las riberas, modelando el problema como una red de nodos (tierra) y líneas (puentes). Dijo que un nodo solo puede cruzarse de forma completa si cada entrada tiene una salida, lo que exige que todos los nodos tengan un grado par. En Königsberg, los nodos tenían grado impar, cerrando toda posibilidad.

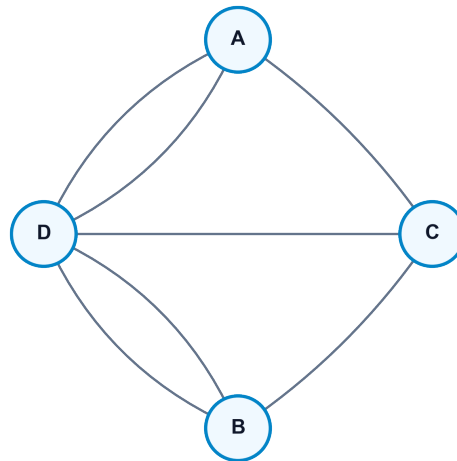


Figura 7.1: Grafo asociado al problema de los puentes de Königsberg

7.1.2. Definición Formal de un Grafo

- **Grafo No Dirigido:** Un par ordenado $G = (V, E)$ donde V es un conjunto no vacío de **vértices** o **nodos** y E es un conjunto de **aristas** formadas por pares no ordenados de vértices distintos $\{u, v\}$.
- **Grafo Dirigido (Digrafo):** Un par ordenado $G = (V, A)$ donde los elementos de A son **arcos** definidos por pares ordenados de vértices (u, v) (donde el flujo va del nodo origen u al nodo destino v).

- **Multigrafo:** Grafo que contiene múltiples aristas conectando el mismo par de nodos.
- **Red o Grafo Valorado:** Un grafo asociado a una función de coste $w : E \rightarrow \mathbb{R}$ que asigna un peso o longitud a cada conexión.

7.1.3. Subgrafos, Conectividad y Grados

- **Subgrafo:** Un grafo $G' = (V', E')$ es subgrafo de $G = (V, E)$ si $V' \subseteq V$ y $E' \subseteq E \cap (V' \times V')$.
- **Grafo Parcial (Spanning Subgraph):** Un subgrafo que contiene a todos los vértices del grafo original ($V' = V$).
- **Grado de un vértice ($d(v)$):** Número de aristas incidentes en v . En digrafos se desglosa en:
 - *Grado de entrada ($d^-(v)$):* Número de arcos entrantes.
 - *Grado de salida ($d^+(v)$):* Número de arcos salientes.
- **Conexión:** Una cadena es una secuencia de nodos y aristas consecutivas. Si no se repiten aristas es *simple*, y si no se repiten nodos es *elemental*. Un ciclo es una cadena cerrada simple.
- **Grafo Conexo:** Existe al menos una cadena entre cualquier pareja de nodos de V . En digrafos, si existe un camino dirigido de ida y vuelta para cualquier par, se denomina **fuertemente conexo**.

7.2. Representación de Grafos

7.2.1. Representación Geométrica y Grafos Planares

Un grafo es **planar** si puede ser dibujado en el plano bidimensional de manera que sus aristas solo se corten en sus vértices.

- **Regiones:** Las caras cerradas (y la región no acotada exterior) delimitadas por la representación en el plano de un grafo planar.

! Teorema: Fórmula Planar de Euler

Sea $G = (V, E)$ un grafo planar conexo con $n = |V|$ vértices y $m = |E|$ aristas. Si r es el número de regiones de su representación planar, entonces se cumple:

$$n + r = m + 2$$

Demostración (por inducción sobre el número de aristas m):

- **Base de la inducción ($m = 1$):**

- Si conecta dos nodos distintos ($n = 2$), define 1 región (exterior): $2 + 1 = 1 + 2$ (correcto).
- Si es un bucle sobre un único nodo ($n = 1$), divide el plano en una región interior y otra exterior ($r = 2$): $1 + 2 = 1 + 2$ (correcto).

■ **Paso inductivo:**

Asumimos que la fórmula se cumple para grafos de k aristas. Sea G un grafo con $m = k + 1$ aristas:

- *Caso 1:* Si G contiene un nodo de grado 1 (vértice pendiente v), al eliminar v y su arista asociada obtenemos un subgrafo conexo con $n - 1$ vértices, k aristas y las mismas r regiones. Por hipótesis: $(n - 1) + r = k + 2 \implies n + r = (k + 1) + 2$.
- *Caso 2:* Si no hay nodos de grado 1, G contiene al menos un ciclo. Al eliminar una arista del ciclo, el número de aristas pasa a k y las dos regiones adyacentes a la arista se unen en una sola ($r - 1$ regiones en el subgrafo). Por hipótesis: $n + (r - 1) = k + 2 \implies n + r = (k + 1) + 2$. ■

! Corolario de la Planaridad

Si G es un grafo simple conexo planar con $n \geq 3$ vértices y m aristas, entonces:

$$3r \leq 2m \quad \text{y} \quad m \leq 3n - 6$$

Demostración: Como el grafo es simple y sin bucles, la frontera de toda región (incluida la exterior) contiene al menos 3 aristas. Sumando las fronteras sobre todas las regiones, cada arista se contabiliza a lo sumo dos veces (una para cada región que delimita). Por tanto:

$$3r \leq \sum_{i=1}^r \text{longitud}(\text{frontera}_i) \leq 2m \implies 3r \leq 2m$$

Sustituyendo el teorema de Euler ($r = m - n + 2$):

$$3(m - n + 2) \leq 2m \implies 3m - 3n + 6 \leq 2m \implies m \leq 3n - 6. \quad \blacksquare$$

Una consecuencia de este corolario es que el grafo completo K_5 (5 nodos, 10 aristas) no es planar, puesto que violaría la cota: $10 \leq 3(5) - 6 = 9$ (falso).

! Teorema de Kuratowski (1930)

Un grafo es planar si y solo si no contiene ningún subgrafo que sea homeomorfo (una subdivisión de aristas) o contenga como menor a los grafos no planares básicos K_5 o $K_{3,3}$.

! Teorema: Caracterización de Grafos Bipartitos

Un grafo es bipartito (sus vértices se dividen en dos conjuntos disjuntos V_1 y V_2 sin conexiones internas) si y solo si **no contiene ningún ciclo de longitud impar**.

7.2.2. Representación en Ordenador

- **Matriz de Adyacencia** ($A \in \mathbb{R}^{n \times n}$): Matriz donde la posición a_{ij} indica la existencia (1) o el coste de la arista que une los nodos i y j .
- **Matriz de Incidencia** ($M \in \mathbb{R}^{n \times m}$): Para digrafos, cada columna representa un arco y tiene un 1 en la fila de origen y un -1 en la fila de destino. Al ser esta matriz **totalmente unimodular (TUM)**, los determinantes de sus submatrices cuadradas son siempre $-1, 0$ o 1 , garantizando soluciones enteras bajo restricciones de balance.
- **Lista de Adyacencia**: Estructura optimizada que asocia a cada nodo un vector dinámico con sus vecinos directos. Es idónea para grafos dispersos.

7.3. Estructura Matemática de los Árboles

Un árbol es un grafo que modela conexiones jerárquicas mínimas sin redundancia.

! Teorema: Caracterizaciones Equivalentes de un Árbol

Sea $G = (V, E)$ un grafo simple no dirigido con n vértices. Las siguientes proposiciones son equivalentes:

1. G es un árbol (conexo y acíclico).
2. G es conexo y tiene exactamente $n - 1$ aristas.
3. G es acíclico y tiene exactamente $n - 1$ aristas.
4. Existe un único camino elemental entre cualquier par de vértices de G .
5. G es acíclico, pero si se añade una arista cualquiera se forma exactamente un ciclo.
6. G es conexo, pero si se elimina una arista cualquiera se desconecta.

7.4. Árbol Soporte de Mínimo Peso (MST)

Dado un grafo conexo no dirigido valorado $G = (V, E, w)$, un **árbol soporte de mínimo peso** es un grafo parcial $T = (V, E_T)$ con $E_T \subseteq E$ que es un árbol y minimiza el coste total de sus conexiones:

$$\min \sum_{e \in E_T} w(e)$$

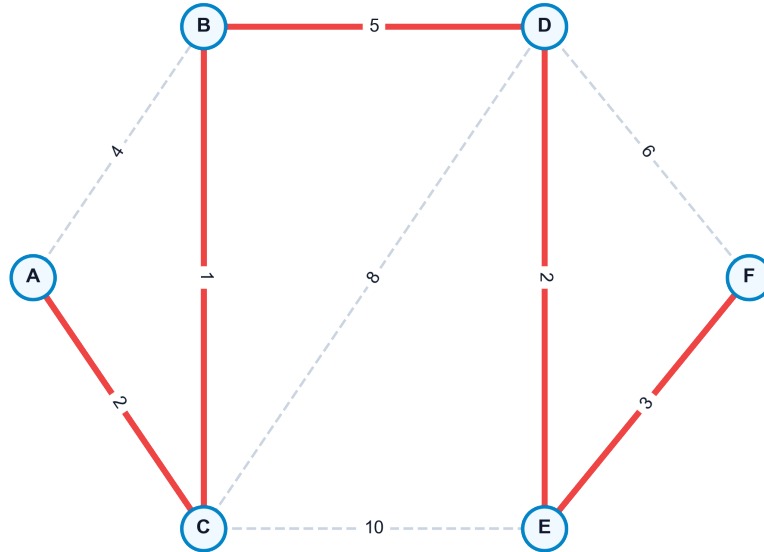


Figura 7.2: Árbol Soporte de Mínimo Peso (MST) destacado en color rojo sobre un grafo valorado

7.4.1. Algoritmo de Kruskal (1956)

Es de naturaleza codiciosa (greedy) y procesa las aristas de forma aislada:

1. Ordenar todas las aristas del grafo de menor a mayor peso.
2. Inicializar cada nodo en su propia componente conexa disjunta.
3. Recorrer las aristas en orden. Si los extremos de la arista actual pertenecen a componentes conexas distintas (no forman ciclo), añadir la arista al árbol y fusionar las componentes. En caso contrario, descartar.
4. Terminar al seleccionar $n - 1$ aristas.

- *Complejidad:* $O(|E| \log |E|)$ debido al coste de la ordenación inicial.

7.4.2. Algoritmo de Prim

Hace crecer una única componente conectada desde un nodo raíz arbitrario s :

1. Inicializar los nodos cubiertos $V_T = \{s\}$ y el árbol de aristas $E_T = \emptyset$.
2. Identificar la arista de coste mínimo $\{u, v\}$ tal que $u \in V_T$ y $v \notin V_T$.
3. Añadir el nodo v a V_T y la arista $\{u, v\}$ a E_T .
4. Repetir hasta cubrir todos los nodos ($V_T = V$).

- *Complejidad:* $O(|E| + |V|\log|V|)$ si se implementa mediante colas de prioridad con montículos de Fibonacci.

Clustering Jerárquico Basado en MST

El MST es una herramienta potente en aprendizaje no supervisado. Para agrupar un conjunto de observaciones en k grupos o clústeres disjuntos:

1. Construir un grafo completo donde los nodos son las observaciones y los pesos de las aristas corresponden a la distancia métrica (euclídea, Manhattan) entre los perfiles.
2. Calcular el MST del grafo.
3. Eliminar las $k - 1$ aristas de mayor peso del MST.
4. Las k componentes conexas resultantes definen los clústeres óptimos bajo el criterio de enlace simple (*Single-Linkage*).

7.5. Implementación en Python

Código Python: Cálculo de MST y Agrupamiento con NetworkX

El siguiente script construye un grafo valorado, extrae su MST usando Kruskal y realiza una partición en $k = 3$ clústeres basados en la topología de pesos:

```

import networkx as nx

# 1. Inicializar el grafo no dirigido con pesos
G = nx.Graph()
edges = [
    ("A", "B", 4), ("A", "H", 8), ("B", "C", 8), ("B", "H", 11),
    ("C", "D", 7), ("C", "F", 4), ("C", "I", 2), ("D", "E", 9),
    ("D", "F", 14), ("E", "F", 10), ("F", "G", 2), ("G", "H", 1),
    ("G", "I", 6), ("H", "I", 7)
]
G.add_weighted_edges_from(edges)

# 2. Calcular el MST usando el algoritmo de Kruskal
mst = nx.minimum_spanning_tree(G, algorithm="kruskal")
peso_mst = mst.size(weight="weight")
print(f"Peso total del MST obtenido: {peso_mst:.2f}")
print("Aristas del arbol soporte:")
for u, v, data in mst.edges(data=True):
    print(f" {u} - {v}: coste = {data['weight']}")

# 3. Clustering basado en MST (k = 3 grupos)
k = 3
# Ordenamos las aristas del MST por peso de forma ascendente
aristas_mst_ordenadas = sorted(mst.edges(data=True), key=lambda x: x[2]['weight'])

# Eliminamos las k-1 aristas mas costosas del arbol
for i in range(k - 1):
    u, v, _ = aristas_mst_ordenadas[-(i + 1)]
    mst.remove_edge(u, v)

# Las componentes conexas resultantes representan los clusters
clusters = list(nx.connected_components(mst))
print(f"\nParticion del Grafo en {k} clusters:")
for idx, cluster in enumerate(clusters):
    print(f" Cluster {idx + 1}: {cluster}")

```

8 El Problema del Camino Mínimo

El problema del **camino mínimo** (Shortest Path Problem) es una de las piedras angulares de la optimización de redes. Su propósito es determinar la ruta de menor coste o distancia acumulada entre un nodo origen s y un nodo destino t , o desde un origen único hacia todos los demás nodos de la red. Este problema no solo cuenta con aplicaciones directas en sistemas de navegación vial y enrutamiento de paquetes de datos en telecomunicaciones, sino que sirve como subproblema crítico en algoritmos de mayor envergadura (como los algoritmos de generación de columnas o los problemas de flujo de coste mínimo), constituyendo un campo fundamental de estudio en la optimización combinatoria (Ahuja, Magnanti, y Orlin 1993; Bertsekas 1998; Cormen et al. 2009).

En este capítulo, estudiaremos el Principio de Optimalidad y las ecuaciones de Bellman, los algoritmos especializados según la naturaleza de los pesos (ordenación topológica para DAGs, Dijkstra para pesos positivos, y Bellman-Ford para pesos negativos), el algoritmo de Floyd-Warshall para caminos entre todos los pares de nodos, y el modelado matemático mediante programación lineal primal-dual destacando la propiedad de total unimodularidad.

! Objetivos de aprendizaje

Al finalizar este capítulo, serás capaz de:

1. **Formular y justificar** las ecuaciones de optimalidad de Bellman para caminos mínimos basándote en el Principio de Optimalidad.
2. **Aplicar la ordenación topológica** para resolver el problema del camino mínimo en redes acíclicas (DAGs) en tiempo lineal $O(|V| + |A|)$.
3. **Ejecutar e implementar** el algoritmo de Dijkstra en redes con pesos no negativos, comprendiendo su carácter ávido y su complejidad temporal.
4. **Resolver problemas** con aristas de coste negativo usando el algoritmo de Bellman-Ford y demostrar analíticamente cómo se detectan los ciclos de coste negativo.
5. **Calcular las distancias** de todos los pares de nodos simultáneamente aplicando la relación de recurrencia de programación dinámica de Floyd-Warshall.
6. **Formular el problema** como un programa lineal continuo, justificando la integridad de sus soluciones mediante la propiedad totalmente unimodular (TUM) de la matriz de incidencia y relacionando las variables duales con los potenciales de los nodos.

8.1. Definición Formal y Condiciones de Existencia

Sea un digrafo conexo valorado $G = (V, A, c)$, donde c_{ij} es el coste o peso asociado al arco dirigido $(i, j) \in A$. Dado un camino dirigido $\mu = (v_0, v_1, \dots, v_k)$ con arcos $(v_{i-1}, v_i) \in A$, la longitud o coste de la ruta es:

$$c(\mu) = \sum_{j=1}^k c_{v_{j-1}, v_j}$$

El problema busca encontrar un camino elemental μ^* de s a t de coste mínimo. Para que exista una solución óptima acotada y bien definida, se deben cumplir dos condiciones:

- **Accesibilidad:** Debe existir al menos un camino dirigido que una el origen s con el destino t .
- **Ausencia de ciclos de coste negativo:** No debe haber ningún ciclo de coste total negativo en la red que sea accesible desde s y desde el cual se pueda alcanzar el destino t . Si existiera tal ciclo, podríamos recorrerlo indefinidamente, reduciendo el coste total acumulado hacia $-\infty$, por lo que el camino mínimo no estaría bien definido.

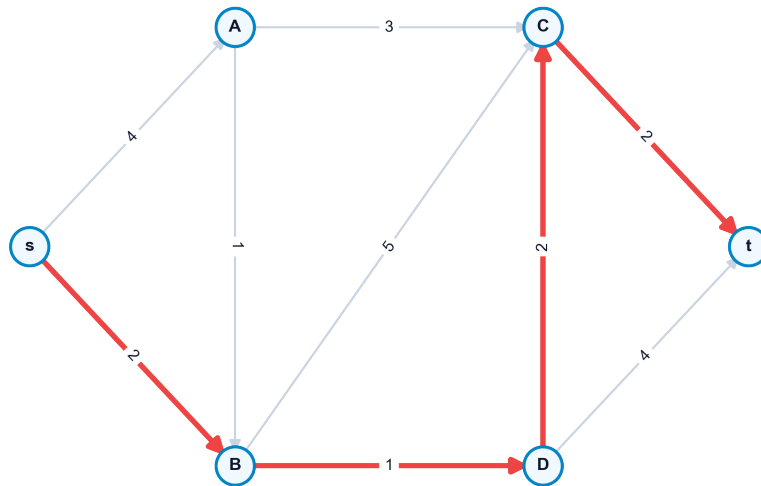


Figura 8.1: Un digrafo con pesos en los arcos y el camino mínimo de origen a destino destacado en rojo

8.2. El Principio de Optimalidad de Bellman

El **Principio de Optimalidad de Bellman** es la base teórica de la programación dinámica aplicada a redes: “Si el camino óptimo para ir de un nodo s a un nodo t pasa por un nodo

intermedio i , entonces la porción de la ruta que une s con i debe ser también el camino óptimo entre estos dos nodos”.

! Las Ecuaciones de Bellman

Sea $d(j)$ la distancia mínima desde el origen fijo s al nodo j . Las distancias mínimas satisfacen de forma garantizada el sistema de ecuaciones:

$$d(s) = 0$$

$$d(j) = \min_{i \in \text{Pred}(j)} \{d(i) + c_{ij}\}, \quad \forall j \neq s$$

Donde $\text{Pred}(j) = \{i \in V \mid (i, j) \in A\}$ representa el conjunto de predecesores directos del nodo j en la red.

8.3. Redes Sin Circuitos (DAGs) y Ordenación Topológica

Si el digrafo no contiene ciclos (Directed Acyclic Graph, DAG), podemos explotar su estructura geométrica ordenando sus nodos para resolver las ecuaciones de Bellman en un único paso secuencial.

8.3.1. Ordenación Topológica

Consiste en etiquetar los nodos con números $t(i) \in \{1, \dots, n\}$ tales que para todo arco dirigido $(i, j) \in A$ se cumpla la relación de orden $t(i) < t(j)$. El algoritmo localiza repetidamente nodos con grado de entrada cero ($d^-(i) = 0$), les asigna la etiqueta correspondiente y los elimina de la red junto con sus arcos salientes.

8.3.2. Algoritmo de Resolución

Una vez reordenados los nodos como $1, 2, \dots, n$ (con el origen s como nodo 1), calculamos las distancias de forma directa:

$$d(1) = 0$$

$$d(j) = \min_{i < j, (i, j) \in A} \{d(i) + c_{ij}\}, \quad j = 2, \dots, n$$

- *Complejidad:* $O(|V| + |A|)$, que es lineal y óptima.

8.4. Redes con Pesos No Negativos: Algoritmo de Dijkstra

Cuando los costes de los arcos son no negativos ($c_{ij} \geq 0$), se utiliza el **Algoritmo de Dijkstra**. Es un método de etiquetado que mantiene un conjunto de distancias provisionales y permanentes:

1. **Inicialización:** Hacer permanente la distancia del origen $d(s) = 0$. Para todo nodo $j \neq s$ adyacente a s , asignar la etiqueta provisional $d(j) = c_{sj}$ (hacer $d(j) = \infty$ si no es adyacente). El conjunto de nodos con distancia permanente es $S = \{s\}$.
2. **Selección del óptimo local:** Localizar el nodo $i \notin S$ que tenga la menor distancia provisional:

$$i = \arg \min_{j \notin S} \{d(j)\}$$

3. **Fijar etiqueta:** Añadir el nodo i al conjunto S (su distancia $d(i)$ se convierte en permanente). Si $S = V$ o $d(i) = \infty$, terminar.
4. **Actualización de distancias:** Para cada vecino $j \notin S$ tal que $(i, j) \in A$, relajar la arista:

$$d(j) = \min\{d(j), d(i) + c_{ij}\}$$

Volver al paso 2.

- *Complejidad:* $O(|A| + |V| \log |V|)$ si se utilizan colas de prioridad con montículos de Fibonacci.

8.5. Redes con Pesos Negativos: Algoritmo de Bellman-Ford

Si existen arcos con pesos negativos, Dijkstra puede fallar debido a que la etiqueta de un nodo fijada como permanente podría abarataarse posteriormente a través de un camino con arcos negativos. En este escenario, aplicamos el **Algoritmo de Bellman-Ford**.

8.5.1. Algoritmo Paso a Paso

1. **Inicialización:** $d(s) = 0$ e $d(j) = \infty$ para todo $j \neq s$.
2. **Relajación sistemática:** Realizar exactamente $|V| - 1$ iteraciones. En cada iteración, recorrer todos los arcos $(i, j) \in A$ del digrafo y actualizar:

$$d(j) = \min\{d(j), d(i) + c_{ij}\}$$

Detección de Ciclos Negativos

Para comprobar si existen ciclos negativos en la red que hagan que las distancias diverjan hacia $-\infty$, se realiza una iteración adicional de relajación sobre todos los arcos $(i, j) \in A$. Si para alguna arista se cumple que:

$$d(j) > d(i) + c_{ij}$$

Entonces existe al menos un **ciclo de coste negativo** accesible desde el origen s , lo que indica que no hay solución óptima acotada.

8.6. Formulación en Programación Lineal Primal-Dual

El problema del camino mínimo puede formularse y resolverse como un programa lineal continuo.

8.6.1. Modelo Primal (Flujo de Coste Mínimo de Una Unidad)

Asumiendo que queremos enviar una unidad de flujo desde el origen s al destino t :

$$\begin{aligned} \min_x \quad & \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \text{sujeto a} \quad & \sum_{j|(i,j) \in A} x_{ij} - \sum_{j|(j,i) \in A} x_{ji} = \begin{cases} 1 & \text{si } i = s \\ -1 & \text{si } i = t \\ 0 & \text{si } i \neq s, t \end{cases} \quad \forall i \in V \\ & x_{ij} \geq 0, \quad \forall (i, j) \in A \end{aligned}$$

La variable continua x_{ij} representa la cantidad de flujo que pasa por el arco (i, j) . - **Total Unimodularidad (TUM)**: La matriz de coeficientes de este sistema es la matriz de incidencia nodo-arco de la red. Al ser esta matriz totalmente unimodular, cualquier base factible del problema de programación lineal es entera. Por tanto, resolver el problema lineal continuo garantiza obtener una solución binaria óptima ($x_{ij}^* \in \{0, 1\}$) que define de forma exacta el camino mínimo.

8.6.2. Modelo Dual (Potenciales Nodales)

Asociando una variable dual no restringida $u_i \in \mathbb{R}$ a la ecuación de balance de cada nodo $i \in V$, el problema dual es:

$$\begin{aligned} & \underset{u}{\text{máx}} && u_t - u_s \\ \text{sujeto a} &&& u_j - u_i \leq c_{ij}, \quad \forall (i, j) \in A \\ &&& u_i \text{ no restringida en signo} \end{aligned}$$

i Interpretación Física y Dual del Potencial Nodal

Las variables duales u_i representan potenciales nodales o tensiones. Si fijamos de forma arbitraria el potencial del origen $u_s = 0$, la variable dual óptima u_j^* coincide exactamente con la distancia mínima $d(j)$ del origen al nodo j . Las restricciones duales $u_j - u_i \leq c_{ij}$ son las condiciones de consistencia de distancias: la diferencia de potencial entre dos nodos no puede superar el coste directo de la arista que los conecta.

8.7. Camino Mínimo Entre Todos los Pares de Nodos: Floyd-Warshall

El **Algoritmo de Floyd-Warshall** es un método de programación dinámica que calcula las distancias mínimas entre cualquier pareja de nodos de forma simultánea.

! Relación de Recurrencia de Floyd-Warshall

Sea $d_{ij}^{(k)}$ la distancia mínima para ir del nodo i al nodo j pudiendo utilizar como nodos intermedios únicamente un subconjunto de nodos $\{1, 2, \dots, k\}$. La relación de recurrencia en la iteración k es:

$$d_{ij}^{(k)} = \min \{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \}$$

Con las condiciones iniciales en $k = 0$ dadas por los pesos directos de los arcos ($d_{ij}^{(0)} = c_{ij}$ si hay arco, $d_{ii}^{(0)} = 0$, y $d_{ij}^{(0)} = \infty$ en caso contrario). - *Complejidad*: $O(|V|^3)$, lo que lo hace muy eficiente para redes densas.

8.8. Implementación en Python con NetworkX

Código Python: Resolución de Caminos Mínimos con NetworkX

El siguiente script define un digrafo que contiene una arista de peso negativo y resuelve el camino mínimo utilizando los algoritmos de Dijkstra y Bellman-Ford, además de comprobar la existencia de ciclos negativos y calcular la matriz de distancias:

```

import networkx as nx

# 1. Inicializar el digrafo con aristas y pesos
G = nx.DiGraph()
edges = [
    ("s", "A", 4), ("s", "B", 3), ("A", "B", -2), ("A", "C", 2),
    ("B", "C", 3), ("B", "t", 6), ("C", "t", 2)
]
G.add_weighted_edges_from(edges)

# 2. Intentar resolver mediante Dijkstra
try:
    path_dij = nx.dijkstra_path(G, "s", "t")
    dist_dij = nx.dijkstra_path_length(G, "s", "t")
    print("Dijkstra:")
    print(f"  Ruta = {path_dij}, Distancia = {dist_dij}")
except ValueError as e:
    print(f"Error en Dijkstra: {e}")

# 3. Resolver de forma robusta con Bellman-Ford
path_bf = nx.bellman_ford_path(G, "s", "t")
dist_bf = nx.bellman_ford_path_length(G, "s", "t")
print("\nBellman-Ford:")
print(f"  Ruta = {path_bf}, Distancia = {dist_bf}")

# 4. Comprobar si el digrafo contiene ciclos de coste negativo
tiene_ciclo_neg = nx.negative_edge_cycle(G)
print(f"\n¿Tiene el digrafo algun ciclo de coste negativo?: {tiene_ciclo_neg}")

# 5. Distancias entre todos los pares (Floyd-Warshall)
dist_pares = dict(nx.floyd_warshall(G))
print("\nDistancias minimas desde el origen 's' a todos los nodos (Floyd-Warshall):")
for destino, dist in dist_pares["s"].items():
    print(f"  s -> {destino}: {dist:.1f}")

```

9 Problemas de Flujo en Redes

La optimización de **flujos en redes** representa una de las ramas más maduras y de mayor impacto práctico de la investigación operativa. Permite modelar y resolver de forma exacta el envío de entidades físicas o lógicas (tales como toneladas de mercancía, litros de crudo, vatios de potencia eléctrica o gigabytes de datos) a través de canales con capacidades limitadas, cuya teoría y aplicaciones algorítmicas se estudian detalladamente en los textos clásicos de flujos y optimización en redes (Ahuja, Magnanti, y Orlin 1993; Bertsekas 1998). En este capítulo, estudiaremos el concepto algebraico de flujo y cociclo, el célebre Lema de Coloración de Arcos de Minty (herramienta teórica cardinal para demostrar optimalidades), el teorema de flujo máximo y corte mínimo, el algoritmo de Ford-Fulkerson y el problema general del flujo de coste mínimo y transbordo.

! Objetivos de aprendizaje

Al finalizar este capítulo, serás capaz de:

1. **Formular algebraicamente** el concepto de flujo y circulación en redes, aplicando las ecuaciones de balance y conservación en los nodos.
2. **Identificar y calcular** cociclos (cortes) en digrafos, distinguiendo entre arcos directos e inversos y calculando su capacidad física.
3. **Comprender y aplicar** el Lema de Coloración de Arcos de Minty, ejecutando el algoritmo constructivo de etiquetado para demostrar la existencia de ciclos y cociclos coloreados.
4. **Demostrar y aplicar** el teorema Max-Flow Min-Cut de Ford-Fulkerson para hallar la capacidad de transporte límite de una red y sus cuellos de botella.
5. **Ejecutar e implementar** el algoritmo de Ford-Fulkerson en la red residual para encontrar caminos aumentantes y actualizar flujos.
6. **Modelar y resolver** problemas de flujo a coste mínimo y transbordo mediante programación lineal y la librería **NetworkX**.

9.1. Estructura Algebraica de Flujos y Cociclos

Sea un digrafo conexo $G = (V, A)$.

9.1.1. Definición de Flujo y Circulación

Un **flujo** en la red G es un vector $x \in \mathbb{R}^{|A|}$ que asigna a cada arco dirigido $(i, j) \in A$ un caudal o cantidad x_{ij} . El flujo debe cumplir con las ecuaciones de conservación en los nodos:

$$\sum_{j|(i,j) \in A} x_{ij} - \sum_{j|(j,i) \in A} x_{ji} = b_i, \quad \forall i \in V$$

Donde el vector $b \in \mathbb{R}^{|V|}$ representa los balances externos:

- **fuentes** ($b_i > 0$): Nodos inyectores de flujo.
- **sumideros** ($b_i < 0$): Nodos extractores de flujo.
- **transbordo** ($b_i = 0$): Nodos intermedios (todo flujo entrante sale).

Para que el sistema tenga solución factible, la red debe estar en equilibrio global: $\sum_{i \in V} b_i = 0$. Si $b_i = 0$ para todo $i \in V$, el flujo x se denomina **circulación**.

Definición de Cociclo (Corte)

Dado un subconjunto de nodos $S \subset V$ (y su complementario $T = V \setminus S$), el **cociclo** $w(S)$ es el conjunto de arcos que conectan nodos de S con nodos de T , independientemente de su dirección:

$$w(S) = w^+(S) \cup w^-(S)$$

Donde: - $w^+(S)$ (**arcos directos**): Arcos orientados desde S hacia T :

$$w^+(S) = \{(i, j) \in A \mid i \in S, j \notin S\}$$

- $w^-(S)$ (**arcos inversos**): Arcos orientados desde T hacia S :

$$w^-(S) = \{(j, i) \in A \mid i \in S, j \notin S\}$$

9.2. El Lema de Coloración de Arcos de Minty (1966)

El lema de coloración de Minty es un teorema fundamental en optimización combinatoria. Permite demostrar de forma directa teoremas de dualidad y existencia de flujos compatibles.

Lema de Coloración de Arcos de Minty

Sea $G = (V, A)$ un digrafo conexo cuyos arcos se pueden colorear de forma arbitraria con tres colores: **negro** (N), **verde** (V), **rojo** (R) o dejarse **sin color** (0). Se exige que al menos un arco $u_0 = (t, s) \in A$ esté coloreado de **negro**.

Entonces, se verifica una, y solo una, de las dos afirmaciones siguientes:

1. El arco negro u_0 pertenece a un **ciclo elemental** coloreado (formado solo por arcos negros, verdes o rojos). Los arcos negros están orientados en sentido directo en el ciclo, los verdes en sentido inverso y los rojos en cualquier sentido.
2. El arco negro u_0 pertenece a un **cociclo elemental** coloreado (formado solo por arcos negros, verdes o sin color). Los arcos negros están orientados en sentido directo en el corte, los verdes en sentido inverso y los arcos sin color en cualquier sentido.

i Demostración y Algoritmo de Etiquetado de Minty

La demostración del lema es constructiva y define el algoritmo de etiquetado:

1. **Etiquetado inicial:** Etiquetar el nodo s (extremo del arco negro de referencia $u_0 = (t, s)$) como $p(s) = s$. Todos los demás nodos se marcan como no etiquetados ($p(j) = 0$).
2. **Propagación:** Buscar un nodo i etiquetado ($p(i) \neq 0$) y un nodo j no etiquetado ($p(j) = 0$) adyacentes en la red. Etiquetar j asignando $p(j) = i$ si se cumple alguna de las siguientes condiciones:
 - Existe el arco dirigido $u = (i, j) \in A$ y u está coloreado de negro o rojo.
 - Existe el arco dirigido $u = (j, i) \in A$ y u está coloreado de verde o rojo.
3. **Criterio de Parada:** Repetir el paso 2 hasta que no se puedan realizar más etiquetas. Al finalizar:
 - *Caso A (El nodo t está etiquetado):* Siguiendo la cadena de predecesores desde t hasta s ($t - p(t) - p(p(t)) - \dots - s$) junto con el arco $u_0 = (t, s)$, se define un ciclo elemental que satisface la primera afirmación del lema.
 - *Caso B (El nodo t no está etiquetado):* Definimos el conjunto $S = \{i \in V \mid p(i) \neq 0\}$. Al no estar t etiquetado, $s \in S$ y $t \notin S$, por lo que el cociclo $w(S)$ contiene al arco u_0 . Por construcción de las etiquetas, este cociclo no puede contener arcos rojos ni arcos con orientaciones prohibidas, satisfaciendo la segunda afirmación. ■

9.3. El Problema del Flujo Máximo

Busca enviar la mayor cantidad posible de flujo desde un nodo fuente s a un nodo sumidero t respetando las capacidades superiores de los arcos u_{ij} :

$$\begin{aligned}
& \underset{x,v}{\text{máx}} \quad v \\
& \text{sujeto a} \quad \sum_{j|(i,j) \in A} x_{ij} - \sum_{j|(j,i) \in A} x_{ji} = \begin{cases} v & \text{si } i = s \\ -v & \text{si } i = t \\ 0 & \text{si } i \neq s, t \end{cases} \quad \forall i \in V \\
& \quad 0 \leq x_{ij} \leq u_{ij}, \quad \forall (i,j) \in A
\end{aligned}$$

9.3.1. Cortes y Capacidad de un Corte

Un corte (S, T) es una partición del conjunto de nodos V tal que el origen $s \in S$ y el destino $t \in T$ (donde $T = V \setminus S$). La **capacidad del corte** es la suma de las capacidades de los arcos que cruzan de S a T :

$$C(S, T) = \sum_{i \in S, j \in T} u_{ij}$$

! Teorema de Flujo Máximo - Corte Mínimo (Ford-Fulkerson, 1956)

- **Acotación débil:** Para cualquier flujo factible v y cualquier corte (S, T) , se cumple que $v \leq C(S, T)$.
- **Igualdad fuerte:** El valor del flujo máximo de s a t coincide exactamente con la capacidad mínima de un corte (S, T) que separe s de t :

$$v^* = \min_{(S,T)} C(S, T)$$

9.3.2. Algoritmo de Ford-Fulkerson (Camino Aumentante)

El algoritmo busca repetidamente rutas donde sea posible añadir flujo de s a t en la **red residual** G_f :

- **Red Residual G_f :** Para cada arco original $(i, j) \in A$ con capacidad u_{ij} y flujo actual x_{ij} , se definen en G_f :
 - Un arco directo (i, j) con capacidad residual $u_{ij} - x_{ij}$ (espacio libre para aumentar flujo).
 - Un arco inverso (j, i) con capacidad residual x_{ij} (capacidad de cancelar o retornar flujo).
- **Pasos del Algoritmo:**
 1. Fijar el flujo inicial $x_{ij} = 0$ para todos los arcos.

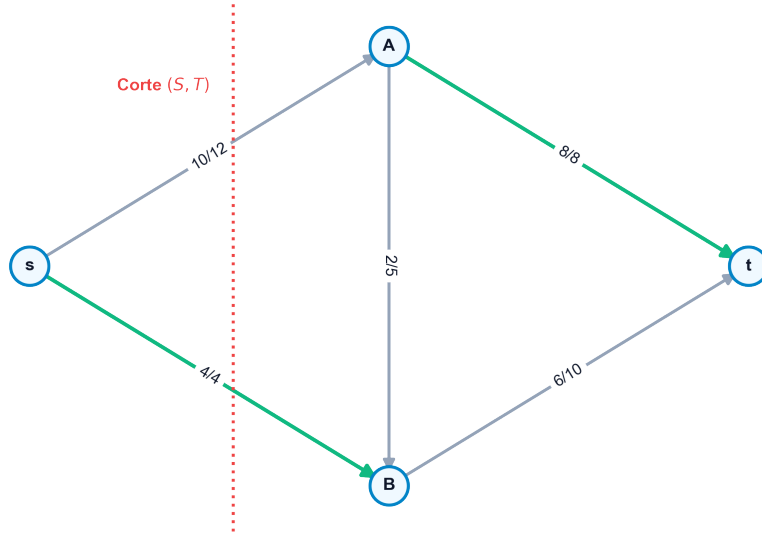


Figura 9.1: Una red de transporte con capacidades y flujo compatible, mostrando un corte mínimo de capacidad 14

2. Buscar un camino simple de s a t en la red residual G_f . Si no existe, terminar (el flujo actual es óptimo).
3. Para el camino aumentante μ hallado, calcular su cuello de botella:

$$\delta = \min_{e \in \mu} \{\text{capacidad residual de } e\}$$

4. Actualizar el flujo original: añadir δ en los arcos directos del camino y restar δ en los arcos inversos.
5. Volver al paso 2.

9.4. Problema de Flujo de Coste Mínimo y Transbordo

El problema de flujo de coste mínimo busca satisfacer las demandas b_i de todos los nodos al menor coste lineal posible, respetando capacidades inferiores y superiores en los arcos:

$$\begin{aligned} & \min_x \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \text{sujeto a } & \sum_{j|(i,j) \in A} x_{ij} - \sum_{j|(j,i) \in A} x_{ji} = b_i, \quad \forall i \in V \\ & l_{ij} \leq x_{ij} \leq u_{ij}, \quad \forall (i,j) \in A \end{aligned}$$

El **problema del transbordo** es una versión donde existen nodos intermedios que no tienen demanda ni oferta externa propia ($b_i = 0$), actuando puramente como puntos de consolidación o bifurcación del flujo. Se formula directamente bajo el esquema del problema de flujo de coste mínimo.

9.5. Implementación en Python con NetworkX

💡 Código Python: Flujo Máximo, Corte Mínimo y Coste Mínimo en NetworkX

El siguiente script construye una red de transporte con capacidades, calcula el flujo máximo y el corte mínimo correspondiente, y resuelve un problema de flujo de coste mínimo compatible usando el algoritmo del Simplex de redes:

```

import networkx as nx

# 1. Resolver el problema de Flujo Maximo y Corte Minimo
G_max = nx.DiGraph()
edges_flow = [
    ("s", "A", 16), ("s", "B", 13), ("A", "B", 10), ("B", "A", 4),
    ("A", "C", 12), ("B", "D", 14), ("C", "B", 9), ("D", "C", 7),
    ("C", "t", 20), ("D", "t", 4)
]
for u, v, cap in edges_flow:
    G_max.add_edge(u, v, capacity=cap)

# Calcular el flujo maximo
valor_flujo, dict_flujo = nx.maximum_flow(G_max, "s", "t")
print(f"Flujo Maximo de s a t: {valor_flujo} unidades")
print("Flujo asignado por arco:")
for u, vecinos in dict_flujo.items():
    for v, val in vecinos.items():
        if val > 0:
            print(f" {u} -> {v}: {val}")

# Calcular el corte minimo asociado
cap_corte, (S, T) = nx.minimum_cut(G_max, "s", "t")
print(f"\nCapacidad del Corte Minimo: {cap_corte}")
print(f"  Nodos del conjunto S (fuente): {S}")
print(f"  Nodos del conjunto T (sumidero): {T}")

# 2. Resolver el problema de Flujo de Coste Minimo (Min-Cost Flow)
G_cost = nx.DiGraph()
# balances de nodo: b_i > 0 oferta, b_i < 0 demanda, b_i = 0 transbordo
G_cost.add_node("s1", demand=-4) # Oferta de 4 unidades
G_cost.add_node("s2", demand=-3) # Oferta de 3 unidades
G_cost.add_node("n1", demand=0)
G_cost.add_node("n2", demand=0)
G_cost.add_node("d1", demand=5) # Demanda de 5 unidades
G_cost.add_node("d2", demand=2) # Demanda de 2 unidades

# Arcos con formato (origen, destino, capacidad, coste unitario)
arcs_cost = [
    ("s1", "n1", 4, 2), ("s1", "n2", 2, 5), ("s2", "n2", 3, 1),
    ("n1", "d1", 3, 4), ("n1", "n2", 2, 2), ("n2", "d1", 2, 3),
    ("n2", "d2", 5, 6), ("d1", "d2", 1, 1)
]
for u, v, cap, cost in arcs_cost:
    G_cost.add_edge(u, v, capacity=cap, weight=cost)

try:
    coste_opt, flujo_opt = nx.network_simplex(G_cost)
    print(f"\nCoste total minimo del flujo: {coste_opt}")
    print("Distribucion optima de flujos:")
    for u, vecinos in flujo_opt.items():
        for v, val in vecinos.items():

```

10 Problemas de Emparejamiento (Matching)

El problema de **emparejamiento** (matching) aborda la selección de un subconjunto de aristas de un grafo de tal manera que ninguna de ellas comparta un vértice común. Esta estructura discreta, en apariencia sencilla, posee una enorme relevancia teórica y práctica en problemas de asignación de tareas a operarios, emparejamiento de alumnos a centros universitarios (como en los algoritmos estables de Gale-Shapley), y enrutamiento en redes complejas, constituyendo una de las áreas más ricas de la optimización combinatoria (Ahuja, Magnanti, y Orlin 1993; Papadimitriou y Steiglitz 1998).

En este capítulo, estudiaremos las definiciones de emparejamientos maximales, máximos y perfectos, la caracterización de optimalidad de Berge basada en cadenas incrementales, la búsqueda mediante árboles alternantes, las relaciones de dualidad de Gallai entre emparejamiento y recubrimiento (*covering*), el teorema de König-Egerváry para grafos bipartitos y el algoritmo Húngaro para emparejamientos con pesos.

! Objetivos de aprendizaje

Al finalizar este capítulo, serás capaz de:

1. **Diferenciar** entre emparejamientos maximales, máximos y perfectos, y determinar las condiciones necesarias para la existencia de estos últimos.
2. **Aplicar el Teorema de Berge** para certificar si un emparejamiento dado es de máxima cardinalidad mediante la búsqueda de cadenas de aumento.
3. **Construir árboles alternantes** para identificar de forma algorítmica caminos incrementales en grafos bipartitos y comprender la dificultad introducida por las “flores” en grafos generales.
4. **Formular y aplicar** las relaciones de dualidad de Gallai que ligán las cardinalidades de emparejamiento, recubrimiento de aristas, recubrimiento de vértices y conjunto independiente.
5. **Utilizar el Teorema de König-Egerváry** para resolver problemas de emparejamiento bipartito mediante su equivalencia con el recubrimiento de vértices mínimo.
6. **Modelar problemas de asignación de máximo coste** en Python, implementando y resolviendo modelos de emparejamiento con pesos usando la librería **NetworkX**.

10.1. Conceptos Fundamentales de Emparejamiento

Sea un grafo simple no dirigido $G = (V, E)$.

10.1.1. Definición de Emparejamiento

Un subconjunto de aristas $M \subseteq E$ es un **emparejamiento** si para todo par de aristas distintas $e_1, e_2 \in M$ se cumple que no comparten ningún nodo ($e_1 \cap e_2 = \emptyset$).

- **Vértice Saturado (o Emparejado):** Un vértice $v \in V$ que pertenece a alguna de las aristas seleccionadas en M .
- **Vértice Insaturado (o Libre):** Un vértice $v \in V$ que no forma parte de ninguna arista de M .
- **Emparejamiento Maximal:** Un emparejamiento M tal que no se le puede añadir ninguna arista de $E \setminus M$ sin violar la condición de emparejamiento.
- **Emparejamiento Máximo:** Un emparejamiento que tiene la mayor cantidad de aristas posible en el grafo G . Su tamaño se denota por la cardinalidad $\alpha_E(G)$.
- **Emparejamiento Perfecto:** Un emparejamiento que satura a todos los vértices del grafo. Si existe, la cardinalidad del emparejamiento es exactamente $|M| = n/2$ (lo que exige que el número total de nodos n sea par).

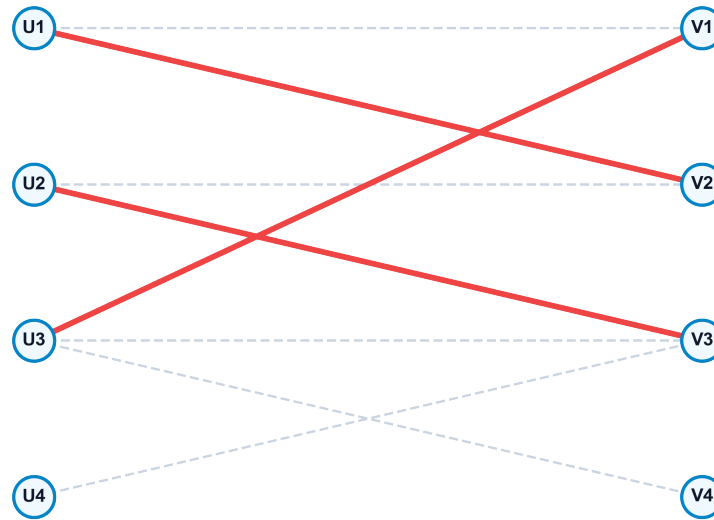


Figura 10.1: Ejemplo de un emparejamiento máximo destacado en color rojo en un grafo bipartito

10.1.2. Cadenas Alternantes e Incrementales

Dada una red con un emparejamiento M :

- **Cadena Alternante:** Una cadena de aristas que pertenecen alternativamente a M y a $E \setminus M$.
- **Cadena Incremental (o de Aumento):** Una cadena alternante cuyos extremos inicial y final son nodos libres (insaturados) con respecto a M .

! Teorema de Berge (1957)

Un emparejamiento M en un grafo G es máximo si y solo si el grafo **no contiene ninguna cadena incremental** con respecto a M .

Demostración (Sentido necesario): Supongamos que existe una cadena incremental $P = (v_0, v_1, \dots, v_k)$ con respecto a M . Al ser incremental, las aristas del camino alternan entre pertenecer a $E \setminus M$ y a M , empezando y terminando con aristas libres (fuera de M). Por tanto, el número de aristas del camino que no están en M es exactamente una más que las que sí están en M . Definimos la diferencia simétrica $M' = M \oplus P = (M \setminus P) \cup (P \setminus M)$. Es directo comprobar que M' constituye un emparejamiento factible en G y su tamaño es $|M'| = |M| + 1$, lo que contradice que M fuera de tamaño máximo. Por tanto, si el emparejamiento es máximo, no pueden existir cadenas incrementales. ■

10.2. Búsqueda mediante Árboles Alternantes

Para localizar de forma algorítmica las cadenas incrementales, construimos una estructura jerárquica de búsqueda denominada **árbol alternante**:

- Se selecciona un vértice libre como nodo raíz.
- Las aristas que descienden desde niveles pares del árbol hacia niveles impares corresponden a aristas no emparejadas ($E \setminus M$).
- Las aristas que van desde niveles impares hacia niveles pares corresponden a aristas emparejadas (M).
- *Resolución:* Si durante la exploración del árbol se encuentra un nodo vecino que está libre, se ha identificado una cadena incremental entre la raíz y dicho nodo.
- *Flores (Blossoms):* En grafos bipartitos, el árbol alternante se construye sin ciclos impares. En grafos generales, la presencia de ciclos de longitud impar (denominados flores) puede bloquear la búsqueda del algoritmo. Jack Edmonds resolvió este problema en 1965 con su algoritmo de contraer las flores en un supernodo durante la búsqueda de caminos y expandirlas posteriormente al actualizar el emparejamiento.

10.3. Problemas de Recubrimiento en Grafos

Estudiamos la relación entre seleccionar aristas independientes y cubrir los componentes del grafo.

10.3.1. Definición de Recubrimientos y Conjuntos Independientes

- **Recubrimiento de Vértices (Vertex Cover):** Un subconjunto de nodos $V_C \subseteq V$ tal que toda arista del grafo tiene al menos uno de sus extremos en V_C :

$$\{u, v\} \cap V_C \neq \emptyset, \quad \forall \{u, v\} \in E$$

El tamaño del recubrimiento de vértices mínimo se denota por $\alpha_V(G)$.

- **Recubrimiento de Aristas (Edge Cover):** Un subconjunto de aristas $E_C \subseteq E$ tal que todo vértice del grafo es incidente en al menos una arista de E_C . Solo existe si el grafo no tiene nodos aislados. Su tamaño mínimo se denota por $\beta_E(G)$.
- **Conjunto de Vértices Independientes:** Un subconjunto de nodos $I \subseteq V$ tal que no existen aristas que unan nodos de I entre sí. Su tamaño máximo se denota por $\beta_V(G)$.

! Teoremas de Dualidad de Gallai (1959)

Para cualquier grafo simple conexo $G = (V, E)$ con n vértices y sin nodos aislados, se cumplen las siguientes relaciones de conservación:

1. Dualidad de Vértices:

$$\alpha_V(G) + \beta_V(G) = n$$

2. Dualidad de Aristas:

$$\alpha_E(G) + \beta_E(G) = n$$

! Teorema de König-Egerváry

Para cualquier **grafo bipartito** $G = (V, E)$, la cardinalidad del emparejamiento máximo es igual al número mínimo de vértices necesarios para recubrir todas las aristas:

$$\alpha_E(G) = \alpha_V(G)$$

10.4. Emparejamiento de Máximo Peso y Algoritmo Húngaro

Cuando cada arista $\{u, v\}$ tiene asociado un beneficio o peso $w_{uv} \geq 0$, el problema de emparejamiento de peso máximo busca maximizar la recompensa total acumulada:

$$\max \sum_{e \in M} w(e) \quad \text{s.t. } M \text{ es un emparejamiento en } G$$

En grafos bipartitos completos donde $|V_1| = |V_2| = n$, este problema se denomina **problema de asignación óptima**. Se resuelve de forma exacta mediante el **Algoritmo Húngaro** (Kuhn-Munkres, 1955), el cual opera sobre el problema dual de programación lineal:

- Asocia un potencial o precio dual u_i a los nodos de la izquierda y v_j a los de la derecha.
- Mantiene la viabilidad dual: $u_i + v_j \leq c_{ij}$ (donde c_{ij} es el coste o peso).
- Busca un emparejamiento perfecto sobre el subgrafo de igualdad formado únicamente por los arcos donde se cumple la igualdad dual con holgura cero: $u_i + v_j = c_{ij}$.
- Si el emparejamiento no es perfecto, modifica de forma óptima los precios duales para activar nuevas aristas en el subgrafo de igualdad y repite.

10.5. Implementación en Python con NetworkX

Código Python: Emparejamientos Máximos con NetworkX

El siguiente script construye un grafo bipartito para hallar su emparejamiento de máxima cardinalidad, y resuelve el emparejamiento de máximo peso sobre un grafo general no bipartito:

```

import networkx as nx

# 1. Emparejamiento de Maxima Cardinalidad en Grafo Bipartito
B = nx.Graph()
nodos_izq = ["U1", "U2", "U3", "U4"]
nodos_der = ["V1", "V2", "V3", "V4"]
# Marcamos la propiedad de biparticion (bipartite=0 o 1)
B.add_nodes_from(nodos_izq, bipartite=0)
B.add_nodes_from(nodos_der, bipartite=1)

# Anadir aristas factibles
aristas_bip = [
    ("U1", "V1"), ("U1", "V2"), ("U2", "V2"), ("U2", "V3"),
    ("U3", "V1"), ("U3", "V2"), ("U4", "V3"), ("U4", "V4")
]
B.add_edges_from(aristas_bip)

# Calcular el emparejamiento maximo bipartito
match_bip = nx.bipartite.maximum_matching(B, top_nodes=nodos_izq)
print("Emparejamiento maximo cardinal en grafo bipartito:")
for u, v in sorted(match_bip.items()):
    if u in nodos_izq: # Evitar duplicados del diccionario bidireccional
        print(f" {u} <-> {v}")

# 2. Emparejamiento de Peso Maximo en Grafo General (No Bipartito)
G = nx.Graph()
aristas_pesos = [
    ("A", "B", 4), ("A", "C", 5), ("B", "C", 6), ("B", "D", 2),
    ("C", "D", 3), ("D", "E", 8), ("C", "E", 4)
]
for u, v, w in aristas_pesos:
    G.add_edge(u, v, weight=w)

# Calcular el emparejamiento de peso maximo
match_peso = nx.max_weight_matching(G, maxcardinality=False)
print("\nEmparejamiento de Peso Maximo (Grafo General):")
peso_total = 0
for u, v in match_peso:
    w = G[u][v]['weight']
    peso_total += w
    print(f" {u} <-> {v} (peso = {w})")
print(f"Peso acumulado del emparejamiento: {peso_total}")

```

11 Rutas Eulerianas, Hamiltonianas y TSP

El diseño de rutas óptimas es uno de los campos más estudiados de la optimización combinatoria y la logística de distribución. Abarca desde problemas solubles en tiempo polinomial, como la determinación de rutas que recorren todas las calles (aristas) de una ciudad al menor coste (el Problema del Cartero Chino), hasta problemas altamente complejos y NP-duros, como la secuenciación de visitas a clientes (vértices) minimizando el kilometraje total sin repetir visitas (el Problema del Viajante o TSP), constituyendo uno de los desafíos más célebres y estudiados de la optimización entera y combinatoria (Papadimitriou y Steiglitz 1998; Wolsey 1998).

En este capítulo, analizaremos la caracterización matemática de las rutas eulerianas y hamiltonianas, la resolución del Problema del Cartero Chino mediante emparejamientos de coste mínimo, la formulación matemática del TSP con restricciones de eliminación de subtours (DFJ y MTZ), las principales heurísticas constructivas y de mejora local (2-opt), y el algoritmo de aproximación de Christofides.

! Objetivos de aprendizaje

Al finalizar este capítulo, serás capaz de:

1. **Diferenciar** entre grafos eulerianos y hamiltonianos, explicando la disparidad en su complejidad computacional inherente (P frente a NP-completo).
2. **Aplicar los teoremas de Euler** para verificar la existencia de ciclos y caminos eulerianos basándote en la paridad del grado de los nodos.
3. **Resolver el Problema del Cartero Chino** en grafos no dirigidos no eulerianos aplicando el cálculo de caminos mínimos y emparejamientos perfectos de peso mínimo.
4. **Evaluar la hamiltonianidad** de un grafo simple a través de las condiciones de suficiencia de Dirac y Ore.
5. **Formular formalmente** el Problema del Viajante (TSP) como un programa lineal entero, comparando las formulaciones DFJ (exponencial) y MTZ (polinomial).
6. **Implementar y comparar** heurísticas constructivas (Vecino más cercano) e incremento de calidad (2-opt), así como el algoritmo de Christofides, resolviendo estos problemas en Python mediante **NetworkX**.

11.1. Grafos Eulerianas

Un grafo conexo es euleriano si admite un recorrido cerrado que recorre cada arista exactamente una vez y regresa al nodo de origen. El origen del estudio de estos grafos se remonta a 1736, cuando **Leonhard Euler** resolvió el famoso problema de los **Puentes de Königsberg**, sentando las bases de la teoría de grafos moderna al demostrar que no existía una ruta que cruzara los siete puentes de la ciudad sin repetir ninguno.

! Teorema: Existencia de Ciclo Euleriano

Un grafo simple conexo no dirigido $G = (V, E)$ contiene un ciclo euleriano si y solo si **todos sus vértices tienen grado par**:

$$d(v) \equiv 0 \pmod{2}, \quad \forall v \in V$$

! Teorema: Existencia de Camino Euleriano Abierto

Un grafo simple conexo no dirigido $G = (V, E)$ contiene un camino euleriano abierto (que empieza en un nodo y termina en otro) si y solo si tiene **exactamente dos vértices con grado impar**. El camino comenzará de forma obligatoria en uno de los vértices impares y terminará en el otro.

11.2. El Problema del Cartero Chino (CPP)

Formulado por el matemático chino **Mei-Ko Kwan** en 1960, el Problema del Cartero Chino (Chinese Postman Problem, CPP) busca la ruta cerrada de longitud mínima que recorra **cada arista del grafo al menos una vez**. Su nombre fue acuñado por Alan Goldman en honor al origen de Kwan.

11.2.1. Estrategia de Resolución en Grafos No Dirigidos

- **Caso A: El grafo es euleriano:** Cualquier ciclo euleriano es solución óptima del problema. La longitud de la ruta óptima es exactamente igual a la suma de los pesos de todas las aristas de la red.
- **Caso B: El grafo no es euleriano:** El grafo posee un conjunto de nodos de grado impar. Por el lema del apretón de manos, el número de estos nodos es par, denotado por V_{impar} (con $|V_{\text{impar}}| = 2k$). Para poder trazar un recorrido euleriano, debemos duplicar ciertos caminos entre estos nodos de grado impar de modo que el coste añadido sea mínimo:

1. Calcular la matriz de distancias mínimas $d(u, v)$ entre todos los pares de nodos pertenecientes a V_{impar} (usando algoritmos como Dijkstra o Floyd-Warshall).
2. Construir un **grafo auxiliar completo** cuyos nodos son únicamente los elementos de V_{impar} , y el peso de cada arista $\{u, v\}$ es la distancia del camino mínimo $d(u, v)$ calculado en el paso anterior.
3. Encontrar un **emparejamiento perfecto de peso mínimo** en este grafo auxiliar completo.
4. Duplicar las aristas correspondientes a las parejas seleccionadas por el emparejamiento perfecto en el grafo original. El multigrafo resultante ahora posee únicamente nodos de grado par, convirtiéndose en un grafo euleriano.
5. Calcular el ciclo euleriano sobre este multigrafo modificado.

11.3. Ciclos Hamiltonianos

Un **ciclo hamiltoniano** es un ciclo simple que visita cada nodo de la red exactamente una vez (excepto el inicial, al que regresa para cerrar el ciclo). Su nombre se debe al matemático irlandés William Rowan Hamilton, quien en 1857 inventó el *Juego Icosian*, consistente en encontrar un camino a lo largo de las aristas de un dodecaedro regular que visitara cada vértice una sola vez.

A diferencia del problema euleriano, que se resuelve de forma inmediata analizando la paridad de los grados en tiempo lineal, determinar si un grafo general contiene un ciclo hamiltoniano es un problema **NP-completo**. No se conocen condiciones necesarias y suficientes sencillas de verificar.

11.3.1. Teoremas de Suficiencia de Hamiltonianidad

Disponemos de condiciones suficientes basadas en la densidad de aristas o en los grados mínimos de los nodos:

! Teorema de Dirac (1952)

Si $G = (V, E)$ es un grafo simple con $n \geq 3$ vértices tal que el grado de todo vértice cumple:

$$d(v) \geq \frac{n}{2}, \quad \forall v \in V$$

Entonces G contiene al menos un ciclo hamiltoniano.

! Teorema de Ore (1960)

Si $G = (V, E)$ es un grafo simple con $n \geq 3$ vértices tal que para cualquier pareja de vértices no adyacentes $u, v \in V$ se cumple:

$$d(u) + d(v) \geq n$$

Entonces G contiene al menos un ciclo hamiltoniano.

11.4. El Problema del Viajante (TSP)

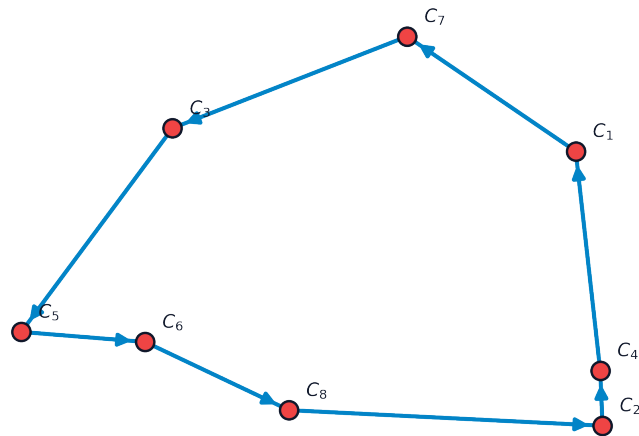


Figura 11.1: Una ruta hamiltoniana óptima que resuelve el problema del viajante (TSP) sobre un conjunto de ciudades

El Problema del Viajante (Traveling Salesperson Problem, TSP) busca encontrar el ciclo hamiltoniano de menor longitud o coste total en un grafo completo valorado.

11.4.1. Formulaci3n Lineal Entera DFJ (Dantzig-Fulkerson-Johnson, 1954)

Sean variables binarias $x_{ij} = 1$ si la ruta 3ptima incluye ir directamente del nodo i al nodo j , y 0 en caso contrario.

$$\begin{aligned}
 \min_x \quad & \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \\
 \text{sujeto a} \quad & \sum_{j=1}^n x_{ij} = 1, \quad \forall i = 1, \dots, n \\
 & \sum_{i=1}^n x_{ij} = 1, \quad \forall j = 1, \dots, n \\
 & \sum_{i \in S, j \in S} x_{ij} \leq |S| - 1, \quad \forall S \subset V \quad \text{con} \quad 2 \leq |S| \leq n - 1 \\
 & x_{ij} \in \{0, 1\}, \quad \forall i, j
 \end{aligned}$$

Las **restricciones de eliminaci3n de subtours** (tercera inecuaci3n) evitan que el modelo genere soluciones compuestas por ciclos independientes inconexos que no recorren la totalidad de los nodos en una sola traza. Como el n3mero de subconjuntos S crece de forma exponencial ($2^n - 2$), en la pr3ctica estas restricciones se a3aden din3micamente mediante algoritmos de corte (*Branch-and-Cut*).

11.4.2. Formulaci3n Lineal Entera MTZ (Miller-Tucker-Zemlin, 1960)

Una alternativa para evitar la formulaci3n exponencial es introducir variables auxiliares continuas $u_i \in \mathbb{R}$ que representan el “orden de visita” de cada nodo i :

$$u_i - u_j + nx_{ij} \leq n - 1, \quad \forall i, j \in \{2, \dots, n\} \text{ con } i \neq j$$

- *Ventaja*: El n3mero de restricciones de eliminaci3n de subtours es polinomial ($O(n^2)$), lo que permite escribir el modelo completo directamente en solvers convencionales.
- *Desventaja*: La relajaci3n lineal del modelo MTZ es significativamente m3s d3bil que la del modelo DFJ. Esto provoca que los solvers tarden mucho m3s tiempo en resolver problemas medianos mediante ramificaci3n y acotaci3n (*Branch-and-Bound*) al no poder acotar el espacio de b3squeda con eficacia.

11.4.3. Heurísticas y Algoritmos de Aproximación

- **Heurística del Vecino más Cercano:** Algoritmo constructivo ávido (*greedy*) que parte de un nodo aleatorio y salta en cada paso al nodo no visitado más cercano. Es extremadamente rápido ($O(n^2)$), pero presenta el riesgo de dejar nodos aislados muy lejanos para el final, generando un último paso de retorno al origen de coste desorbitado.
- **Heurística de Mejora Local 2-Opt:** Dada una ruta hamiltoniana válida, busca parejas de aristas que se crucen o cuya reconexión reduzca la longitud de la ruta. En cada paso, reemplaza el par de aristas $\{(A, B), (C, D)\}$ por el par $\{(A, C), (B, D)\}$, lo que invierte el sentido del recorrido del segmento intermedio. Se aplica iterativamente hasta alcanzar un óptimo local (donde ningún intercambio 2-opt consiga reducir el coste).

Algoritmo de Christofides (1976)

En espacios métricos (donde las distancias cumplen la desigualdad triangular $c_{ij} \leq c_{ik} + c_{kj}$), Christofides proporciona una garantía teórica muy robusta: la solución obtenida no superará el **1.5 veces** el coste de la solución óptima global:

1. Calcular el árbol soporte de mínimo peso (MST) T de la red.
2. Identificar el conjunto de nodos con grado impar en el árbol T , denotado por O (debe ser un número par de nodos).
3. Calcular un emparejamiento perfecto de peso mínimo M sobre los nodos de O utilizando las aristas y distancias del grafo original.
4. Unir las aristas de T y M para formar un multigrafo euleriano H .
5. Encontrar un ciclo euleriano en este multigrafo.
6. Transformar el ciclo euleriano en un ciclo hamiltoniano realizando “atajos” (*shortcuts*), es decir, recorriendo el ciclo y saltando directamente a nodos no visitados omitiendo duplicados.

Demostración: Garantía del Factor 1.5 de Christofides

Demostremos que la ruta obtenida por el algoritmo de Christofides cumple $Cost(C) \leq 1.5Cost(C^*)$, donde C^* es el ciclo óptimo del TSP.

1. **Cota del MST:** Si eliminamos una arista del ciclo óptimo C^* , obtenemos un camino soporte que pasa por todos los nodos del grafo. Dado que este camino es un árbol soporte, y el MST T es por definición el árbol de mínimo peso:

$$Cost(T) \leq Cost(\text{camino}) < Cost(C^*)$$

2. **Cota del Emparejamiento:** Sea O el conjunto de vértices de grado impar en T . Consideremos el ciclo óptimo C^* proyectado únicamente sobre los vértices de O mediante atajos. Por la desigualdad triangular, el coste de este ciclo proyectado

C_O^* cumple $Cost(C_O^*) \leq Cost(C^*)$. El ciclo C_O^* tiene un número par de vértices. Podemos descomponer dicho ciclo en dos emparejamientos perfectos alternos M_1 y M_2 sobre los nodos de O . La suma de sus costes es:

$$Cost(M_1) + Cost(M_2) = Cost(C_O^*) \leq Cost(C^*)$$

Como M es el emparejamiento perfecto de peso mínimo sobre O :

$$Cost(M) \leq \min\{Cost(M_1), Cost(M_2)\} \leq 0.5Cost(C^*)$$

3. **Resultado final:** Combinando ambas cotas en el multigrafo euleriano $H = T \cup M$:

$$Cost(H) = Cost(T) + Cost(M) \leq Cost(C^*) + 0.5Cost(C^*) = 1.5Cost(C^*)$$

Al aplicar atajos para obtener el ciclo hamiltoniano C , la desigualdad triangular garantiza que el coste solo puede disminuir: $Cost(C) \leq Cost(H) \leq 1.5Cost(C^*)$. ■

11.5. Implementación en Python con NetworkX

💡 Código Python: Cartero Chino y TSP con NetworkX

El siguiente script construye un grafo de distancias bidimensionales para aproximar el TSP usando el algoritmo de Christofides, y comprueba la eulerianidad para el problema del cartero chino:

```

import networkx as nx
import math

# 1. Resolver el TSP Metrico usando Christofides
G_tsp = nx.complete_graph(5)
# Coordenadas espaciales de los nodos para garantizar la desigualdad triangular
posiciones = {
    0: (0, 0),
    1: (1, 3),
    2: (4, 4),
    3: (6, 1),
    4: (2, 1)
}

# Asignar distancias euclideas como pesos de las aristas
for u in G_tsp.nodes():
    for v in G_tsp.nodes():
        if u != v:
            x1, y1 = posiciones[u]
            x2, y2 = posiciones[v]
            dist = math.sqrt((x1 - x2)**2 + (y1 - y2)**2)
            G_tsp[u][v]['weight'] = dist

# Resolver mediante la aproximacion de traveling_salesman_problem (Christofides)
from networkx.algorithms.approximation import traveling_salesman_problem
ruta_optima = traveling_salesman_problem(G_tsp, cycle=True)
print("Ruta aproximada del TSP (Christofides):")
print(f" {ruta_optima}")

coste_ruta = 0
for i in range(len(ruta_optima) - 1):
    coste_ruta += G_tsp[ruta_optima[i]][ruta_optima[i+1]]['weight']
print(f"Longitud total de la ruta: {coste_ruta:.4f} unidades")

# 2. Comprobacion de eulerianidad para el Cartero Chino
G_post = nx.Graph()
edges_post = [
    (0, 1, 3), (0, 2, 4), (1, 2, 2), (1, 3, 5), (2, 3, 1), (2, 4, 6), (3, 4, 3)
]
G_post.add_weighted_edges_from(edges_post)

es_euler = nx.is_eulerian(G_post)
print(f"\n¿Es euleriano el grafo de reparto?: {es_euler}")
if not es_euler:
    nodos_impares = [v for v, d in G_post.degree() if d % 2 != 0]
    print(f"Nodos con grado impar que requieren emparejarse: {nodos_impares}")

```

Conclusiones

A lo largo de once capítulos hemos presentado el material de la asignatura de **Optimización y Análisis de Redes** del Grado en **Matemáticas**.

En este capítulo de cierre, te invitamos a reflexionar sobre las principales lecciones aprendidas durante el curso, destacando cómo el rigor matemático y el pensamiento analítico se convierten en herramientas indispensables para dominar la modelización matemática. Además, animamos a los estudiantes a continuar explorando y ampliando sus conocimientos en cursos posteriores, consolidando una base sólida para afrontar los desafíos de la optimización y la ciencia de datos moderna desde una perspectiva analítica, rigurosa y fundamentada.

Al finalizar este recorrido, queda patente que la optimización es mucho más que una colección de algoritmos aislados; es un marco de pensamiento estructurado para tomar decisiones óptimas bajo condiciones de recursos limitados. Hemos transitado desde los fundamentos continuos del análisis convexo y las condiciones de optimalidad hasta la estructura discreta y combinatorial de las redes y grafos, equipando a los futuros matemáticos con herramientas capaces de modelar, resolver e interpretar problemas complejos del mundo real.

Resumen del aprendizaje

A lo largo de este manual, hemos construido un conocimiento progresivo y bidireccional sobre la optimización matemática, cubriendo los siguientes pilares:

1. **Optimización No Lineal y Métodos Numéricos:** Partimos del estudio de problemas no lineales continuos sin restricciones. Analizamos métodos de búsqueda lineal unidimensional y algoritmos multivariantes, comprendiendo las ventajas de velocidad y convergencia del método de Newton y Cuasi-Newton (BFGS) frente al clásico descenso de gradiente.
2. **Análisis Convexo:** Establecimos el marco matemático que garantiza la globalidad de los óptimos. Estudiamos las propiedades de los conjuntos y funciones convexas, los teoremas de separación e hiperplanos de soporte, y la generalización de la diferenciabilidad mediante el subgradiente y el subdiferencial, sentando las bases teóricas de la optimización convexa moderna.

3. **Condiciones de Optimalidad (KKT) y Programación Cuadrática:** Dedujimos formalmente las condiciones necesarias de Fritz-John y las condiciones necesarias y suficientes de Karush-Kuhn-Tucker (KKT) bajo cualificación de restricciones (LICQ). Aplicamos este potente formalismo a la resolución de problemas lineales (conectando multiplicadores KKT con dualidad) y cuadráticos (Símplex de Wolfe).
4. **Optimización en Ciencia de Datos:** Exploramos aplicaciones reales y modernas en el aprendizaje automático. Formulamos regresiones robustas bajo diferentes normas (ℓ_1 , ℓ_2 , ℓ_∞), dedujimos el clasificador de margen máximo (SVM) mediante su problema dual, y planteamos la explicabilidad mediante contrafacticos a través de modelos de optimización cuadrática entera mixta (MIQP).
5. **Estructura y Algoritmos en Redes:** Pasamos al dominio discreto de los grafos. Analizamos la propiedad clave de la total unimodularidad (TUM) de la matriz de incidencia, que actúa como puente matemático permitiendo resolver problemas discretos mediante programación lineal continua sin perder la integridad. Estudiamos algoritmos eficientes para árboles soporte mínimos (Prim y Kruskal), caminos mínimos (Dijkstra, Bellman-Ford, Floyd-Warshall) con su interpretación dual de potenciales nodales, y flujos máximos (Ford-Fulkerson y el teorema del corte mínimo).
6. **Emparejamientos y Rutas Óptimas:** Abordamos problemas combinatorios complejos. Desde el emparejamiento óptimo (bipartito y el algoritmo Húngaro) hasta el diseño de rutas óptimas (el problema del Cartero Chino y el TSP), contrastando la dificultad computacional de los problemas en P frente a los NP-duros y evaluando el diseño de heurísticas (2-opt) y algoritmos de aproximación (Christofides).

Reflexiones finales

El estudio de la **Optimización y el Análisis de Redes** dota al matemático de un puente directo entre la abstracción algebraica y analítica y la resolución de problemas tangibles de toma de decisiones. Hemos visto cómo conceptos teóricos profundos —geometría convexa, dualidad lineal y estructura combinatorial de grafos— se traducen en algoritmos computacionales de alta eficiencia que gestionan la logística, las finanzas o las telecomunicaciones modernas.

Hemos aprendido que modelar no es un proceso mecánico ni una mera aplicación de recetas informáticas. Construir un modelo útil requiere un proceso iterativo de **abstracción, resolución, validación y análisis de sensibilidad**. Entender las hipótesis que garantizan la convergencia de un algoritmo o la integridad de una solución es lo que distingue a un matemático de un aplicador de software a ciegas.

Proyección futura: El valor del rigor matemático

Las competencias adquiridas en esta asignatura son esenciales en el perfil del matemático moderno. Vuestra formación os proporciona una ventaja competitiva fundamental: la capacidad de ir más allá del código, comprendiendo la geometría del espacio factible, la estabilidad de las restricciones y la justificación teórica de los óptimos obtenidos.

Este conocimiento es el lenguaje compartido con la **Investigación Operativa** avanzada, el **Aprendizaje Automático** y la **Ciencia de Datos Cuantitativa**. Mientras que otros perfiles aplican modelos como “cajas negras”, vuestro rigor os capacita para diseñar nuevas formulaciones, diagnosticar fallos de convergencia y optimizar recursos con plenas garantías matemáticas.

Esta sólida base analítica os posiciona de manera idnea para trayectorias profesionales avanzadas en sectores estratégicos como **Ingeniería de Optimización y Logística**, **Científico de Datos Cuantitativo**, **Analista de Riesgos o Quants** en el sector financiero, o para continuar vuestra formación en **estudios de postgrado (Máster y Doctorado)** en optimización, estadística aplicada e investigación operativa.

¡Os deseamos el mayor de los éxitos en vuestra andadura profesional y académica!

Bibliografía

- Ahuja, Ravindra K, Thomas L Magnanti, y James B Orlin. 1993. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.
- Bazaraa, Mokhtar S, John J Jarvis, y Hanif D Sherali. 2011. *Linear Programming and Network Flows*. John Wiley & Sons.
- Bazaraa, Mokhtar S, Hanif D Sherali, y Chitharanjan M Shetty. 2013. *Nonlinear Programming: Theory and Algorithms*. John Wiley & Sons.
- Bertsekas, Dimitri P. 1998. *Network Optimization: Continuous and Discrete Models*. Athena Scientific.
- Boyd, Stephen, y Lieven Vandenberghe. 2004. *Convex Optimization*. Cambridge University Press.
- Cormen, Thomas H, Charles E Leiserson, Ronald L Rivest, y Clifford Stein. 2009. *Introduction to Algorithms*. 3rd ed. MIT Press.
- Hastie, Trevor, Robert Tibshirani, y Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. Springer Science & Business Media.
- Hillier, Frederick S, y Gerald J Lieberman. 2010. *Introducción a la Investigación de Operaciones*. 9na ed. McGraw-Hill.
- Luenberger, David G, y Yinyu Ye. 2015. *Linear and Nonlinear Programming*. 4th ed. Springer.
- Nocedal, Jorge, y Stephen Wright. 2006. *Numerical Optimization*. Springer Science & Business Media.
- Papadimitriou, Christos H, y Kenneth Steiglitz. 1998. *Combinatorial Optimization: Algorithms and Complexity*. Courier Corporation.
- Rockafellar, R Tyrrell. 1970. *Convex Analysis*. Princeton University Press.
- Wolsey, Laurence A. 1998. *Integer Programming*. John Wiley & Sons.